ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



PHẠM THANH BÌNH
MSHV: 230201003

PHÁT TRIỂN MÔ HÌNH DEEP LEARNING CẢI TIẾN DỰA TRÊN MAMBA ĐỂ CHẨN ĐOÁN VÀ DỰ ĐOÁN CÁC LOẠI BỆNH PHỔ BIẾN TRÊN LÁ XOÀI

LUẬN VĂN THẠC SĨ
NGÀNH CÔNG NGHỆ THÔNG TIN
Mã số: 8.48.02.01

NGƯỜI HƯỚNG DẪN KHOA HỌC:
TS. NGUYỄN TẤT BẢO THIỆN

TP HỒ CHÍ MINH - NĂM 2026

LỜI CAM ĐOAN

Tôi xin cam đoan luận văn thạc sĩ với đề tài "PHÁT TRIỂN MÔ HÌNH DEEP LEARNING CẢI TIẾN DỰA TRÊN MAMBA ĐỂ CHẨN ĐOÁN VÀ DỰ ĐOÁN CÁC LOẠI BỆNH PHỔ BIẾN TRÊN LÁ XOÀI" là kết quả nghiên cứu do tôi thực hiện dưới sự hướng dẫn khoa học của TS. Nguyễn Tất Bảo Thiện.

Các số liệu, kết quả và nội dung trình bày trong luận văn là trung thực. Các tài liệu, số liệu và kết quả của các tác giả khác được sử dụng trong luận văn đều được trích dẫn đầy đủ, rõ ràng theo quy định.

Tôi xin chịu trách nhiệm về tính chính xác và trung thực của nội dung luận văn này.

TP. Hồ Chí Minh, ngày .... tháng .... năm ....

Học viên

Phạm Thanh Bình

LỜI CẢM ƠN

Trước hết, tôi xin bày tỏ lòng biết ơn sâu sắc đến TS. Nguyễn Tất Bảo Thiện, người hướng dẫn khoa học đã định hướng, góp ý chuyên môn và đồng hành cùng tôi trong quá trình thực hiện luận văn. Những chỉ dẫn của Thầy là chính cơ sở quan trọng giúp tôi có thể hoàn thành luận văn này.

Tôi trân trọng cảm ơn Quý Thầy/Cô Trường Đại học Công nghệ Thông tin - Đại học Quốc gia Thành phố Hồ Chí Minh đã tận tình giảng dạy và tạo điều kiện thuận lợi trong suốt quá trình học tập, nghiên cứu.

Tôi cũng xin cảm ơn gia đình, bạn bè và những cá nhân đã động viên, hỗ trợ tôi trong quá trình thu thập dữ liệu nghiên cứu cho luận văn này.

Mặc dù đã nỗ lực hoàn thành, luận văn vẫn khó tránh khỏi những hạn chế và sai sót. Tôi xin trân trọng tiếp thu các ý kiến đóng góp của Quý Thầy/Cô và các nhà nghiên cứu để tiếp tục hoàn thiện công trình trong thời gian tới.

TP. Hồ Chí Minh, ngày .... tháng .... năm ....

Học viên

Phạm Thanh Bình

MỤC LỤC

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT 4

DANH MỤC CÁC BẢNG 5

DANH MỤC CÁC HÌNH VẼ, ĐỒ THỊ 6

MỞ ĐẦU 8

CHƯƠNG 1. TỔNG QUAN 9

1.1. Giới thiệu đề tài 9

1.2. Mục tiêu của luận văn này 10

1.3. Nội dung nghiên cứu 11

1.4. Phương pháp nghiên cứu 11

1.5. Cấu trúc của luận văn này 12

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 13

2.1. Cơ sở lý thuyết về học sâu cho thị giác 13

2.1.1. Mạng nơ-ron tích chập (CNN) 13

2.1.2. Attention Mechanism và Vision Transformer 15

2.1.3. Mamba 17

2.1.4. Các kỹ thuật liên quan (Large-Kernel Attention, ECA, GRN, SwiGLU) 20

2.1.5. Học chuyển tiếp và chương trình huấn luyện nhiều giai đoạn 24

2.2. Dữ liệu nghiên cứu 25

2.2.1. Các bộ dữ liệu công bố (MangoLeafBD, Mango Leaf Disease Dataset, MLD24) 25

2.2.2. Phân tích hạn chế của các bộ dữ liệu hiện có 26

2.2.3. Đề xuất bộ dữ liệu mới: VN-MangoLeaf 27

2.2.3.1. Quy trình thu thập và gán nhãn dữ liệu 27

MỤC LỤC

2.2.3.2. Đặc điểm và cách nhận biết các bệnh trong bộ dữ liệu VN-MangoLeaf 29
2.2.4. Tiền xử lý và tăng cường dữ liệu 34
2.3. Các độ đo đánh giá hiệu năng mô hình 36
2.3.1. Các độ đo về độ chính xác (Accuracy, Precision, Recall, F1-Score) 36
2.3.2. Các độ đo về hiệu quả tính toán (Thời gian suy luận, số lượng tham số, kích thước mô hình) 37
CHƯƠNG 3. KIẾN TRÚC MÔ HÌNH VÀ PHƯƠNG PHÁP ĐỀ XUẤT 39
3.1. Kiến trúc MangoMamba 39
3.1.1. Cấu hình phân cấp và luồng dữ liệu 39
3.1.2. PatchEmbed 41
3.1.3. Khối MangoMambaBlock 42
3.1.4. MultiScaleMambaMixer 43
3.1.5. DiseaseAwareMixer 45
3.1.6. Large-Kernel Attention 46
3.1.7. Chuẩn hóa cuối và đầu phân loại 47
3.2. Ba giai đoạn huấn luyện mô hình 48
3.2.1. Thiết lập huấn luyện chung 48
3.2.2. Giai đoạn 1 trên MangoLeafBD 49
3.2.3. Giai đoạn 2 trên Bộ dữ liệu kết hợp 49
3.2.4. Giai đoạn 3 trên VN-MangoLeaf 49
3.2.5. Giả mã lan truyền thuận và chuyển checkpoint 50
3.2.6. Cơ sở lựa chọn và phạm vi kết luận của chương trình ba giai đoạn 51
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ 54

MỤC LỤC

4.1. Ứng dụng chẩn đoán bệnh trên di động 54
4.2. Môi trường và thiết lập thực nghiệm.....55
4.2.1. Cấu hình phần cứng và phần mềm.....55
4.2.2. Các siêu tham số huấn luyện và kỹ thuật điều chuẩn 55
4.3. Đánh giá kết quả phân loại trên các bộ dữ liệu 56
4.3.1. Kết quả trên bộ dữ liệu MangoLeafBD 57
4.3.2. Kết quả trên Bộ dữ liệu kết hợp.....59
4.3.3. Kết quả trên bộ dữ liệu VN-MangoLeaf.....59
4.3.4. Phân tích ma trận nhầm lẫn.....61
4.3.5. Đánh giá chiến lược huấn luyện theo chương trình.....63
4.4. So sánh tham chiếu với các mô hình đã công bố 65
4.4.1. So sánh về độ chính xác.....65
4.4.2. So sánh về hiệu quả tính toán (Tốc độ suy luận, kích thước mô hình) 66
4.5. Diễn giải mô hình và phân tích hạn chế.....67
4.5.1. Trực quan hóa kết quả bằng Grad-CAM 67
4.5.2. Phân tích các trường hợp dự đoán sai 70
CHƯƠNG 5. KẾT LUẬN VÀ KHUYẾN NGHỊ 72
5.1. Kết luận 72
5.2. Hạn chế của luận văn.....72
5.3. Hướng phát triển trong tương lai.....73
DANH MỤC CÔNG BỐ KHOA HỌC CỦA TÁC GIẢ 75
TÀI LIỆU THAM KHẢO 76

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Ký hiệu / viết tắt	Diễn giải
AdamW	Thuật toán tối ưu Adam với phân rã trọng số (Adaptive Moment Estimation with decoupled Weight decay)
AMP	Huấn luyện hỗn hợp độ chính xác tự động (Automatic Mixed Precision)
CAM	Bản đồ kích hoạt lớp (Class Activation Mapping)
CNN	Mạng nơ-ron tích chập (Convolutional Neural Network)
ECA	Cơ chế chú ý kênh hiệu quả (Efficient Channel Attention)
FFN	Mạng truyền thẳng (Feed-Forward Network)
GAP	Gộp trung bình toàn cục (Global Average Pooling)
GLU	Đơn vị tuyến tính có cổng (Gated Linear Unit)
GPU	Bộ xử lý đồ hoạ (Graphics Processing Unit)
Grad-CAM	Gradient-weighted Class Activation Mapping
GRN	Chuẩn hóa phản hồi toàn cục (Global Response Normalization)
LKA	Chú ý kernel lớn (Large-Kernel Attention)
LN	Chuẩn hóa lớp (Layer Normalization)
MLP	Mạng perceptron nhiều lớp (Multi-Layer Perceptron)
ONNX	Định dạng trao đổi mô hình mạng nơ-ron mở (Open Neural Network Exchange)
ReLU	Hàm kích hoạt tuyến tính chỉnh lưu (Rectified Linear Unit)
SE	Cơ chế Squeeze-and-Excitation
SiLU	Hàm kích hoạt Sigmoid Linear Unit
SSM	Mô hình không gian trạng thái (State-Space Model)
SwiGLU	Hàm kích hoạt Swish Gated Linear Unit
VAN	Mạng chú ý thị giác (Visual Attention Network)

DANH MỤC CÁC BẢNG

Bảng 2.1. So sánh các bộ dữ liệu bệnh lá xoài công bố (MangoLeafBD, MLD24, Mango Leaf Disease Dataset) với bộ dữ liệu đề xuất VN-MangoLeaf 26
Bảng 2.2. Tham số tăng cường và tiền xử lý dữ liệu 35
Bảng 2.3. Phân bố ảnh trên các tập Huấn luyện, Kiểm định và Kiểm tra cho từng bộ dữ liệu thực nghiệm 36
Bảng 3.1. Vai trò của từng giai đoạn trong chương trình huấn luyện 53
Bảng 4.1. Cấu hình phần cứng và phần mềm cho thực nghiệm 55
Bảng 4.2. Siêu tham số huấn luyện và kỹ thuật điều chuẩn 56
Bảng 4.3. Kết quả trên bộ dữ liệu MangoLeafBD (tập kiểm tra 400 ảnh) 57
Bảng 4.4. Kết quả khảo sát thành phần trên MangoLeafBD (đầu vào 224 × 224, tập kiểm tra 400 ảnh, n = 4) 58
Bảng 4.5. Kết quả trên Bộ dữ liệu kết hợp (tập kiểm tra 4.000 ảnh) 59
Bảng 4.6. Kết quả trên bộ dữ liệu VN-MangoLeaf (tập kiểm tra 700 ảnh) 59
Bảng 4.7. Chỉ số theo từng lớp trên VN-MangoLeaf của checkpoint seed 42 60
Bảng 4.8. So sánh học theo chương trình ba giai đoạn và tinh chỉnh trực tiếp trên VN-MangoLeaf từ checkpoint giai đoạn 1 (độ chính xác tập kiểm tra, %) 64
Bảng 4.9. So sánh với các mô hình đã công bố trên MangoLeafBD 66

DANH MỤC CÁC HÌNH VẼ, ĐỒ THỊ

Hình 2.1. Minh hoạ phép tích chập (kernel 3×3 trượt với stride 2 trên ảnh đầu vào 7×7 đã padding) và phép gộp cực đại (max pooling 2×2) trong CNN (sơ đồ hoá theo [20]) 14
Hình 2.2. Minh hoạ self-attention với các thành phần Query, Key, Value (theo [43]) 16
Hình 2.3. Minh hoạ kiến trúc Vision Transformer (ViT): chia ảnh thành các patch, embedding vị trí và xử lý qua Transformer Encoder (theo [10]) 16
Hình 2.4. Mô hình không gian trạng thái (SSM) rời rạc hóa trong Mamba: trạng thái ẩn $h_{t-1} \rightarrow h_t$ qua A, B_t và đầu ra y_t qua C_t (theo [12]) 18
Hình 2.5. Kiến trúc khối Mamba (kết hợp khối H3 và Gated MLP); khung đỏ đánh dấu khối SSM chọn lọc thực hiện cơ chế quét chọn lọc (selective scan) (theo [12]) 19
Hình 2.6. Kiến trúc MambaVision của NVIDIA, sơ đồ hóa theo [15]. 20
Hình 2.7. Minh hoạ cơ chế phân rã kernel lớn trong Large-Kernel Attention (LKA) (theo [13]) 21
Hình 2.8. Minh hoạ kiến trúc Efficient Channel Attention (ECA) (theo [44]) 22
Hình 2.9. Minh hoạ ảnh hưởng của chuẩn hóa phản hồi toàn cục (Global Response Normalization - GRN) lên độ đa dạng kích hoạt kênh (theo [46]) 23
Hình 2.10. Bản đồ các địa điểm thu thập dữ liệu của bộ VN-MangoLeaf tại khu vực Nam Bộ, Việt Nam (Nguồn nền: Google Maps; vị trí thu thập; địa danh theo phân chia hành chính Việt Nam năm 2025) 28
Hình 2.11. Mẫu lá xoài nhiễm bệnh rỉ sắt (Red Rust) 28
Hình 2.12. Triệu chứng Thán thư - Anthracnose trên lá xoài 30
Hình 2.13. Triệu chứng Rỉ sắt - Red Rust trên lá xoài 30
Hình 2.14. Triệu chứng Bồ hóng - Sooty Mold trên lá xoài 31
Hình 2.15. Triệu chứng Khô đọt - Dieback trên lá xoài 32
Hình 2.16. Triệu chứng Ghẻ xoài - Gall Midge trên lá xoài 32
Hình 2.17. Triệu chứng Phấn trắng - Powdery Mildew trên lá xoài 33
Hình 2.18. So sánh mẫu ảnh giữa các bộ dữ liệu công bố (MangoLeafBD, Mango Leaf Disease Dataset, MLD24) và bộ dữ liệu đề xuất VN-MangoLeaf 34
Hình 3.1. Kiến trúc tổng thể MangoMamba. 40
Hình 3.2. Cấu trúc một MangoMambaBlock. 43
Hình 3.3. Quy trình huấn luyện theo chương trình ba giai đoạn. 48

DANH MỤC CÁC HÌNH VẼ, ĐỒ THỊ

Hình 3.4. Giải thuật huấn luyện ba giai đoạn và lan truyền thuận của MangoMamba (điều chỉnh từ Algorithm 1 trong [27]).....51
Hình 4.1. Giao diện ứng dụng Android trên thiết bị thật: (a) kết quả nhận diện Anthracnose; (b) bản đồ CAM của lớp Anthracnose; (c) danh mục bảy lớp bệnh; (d) trang thông tin bệnh kèm ảnh mẫu từ VN-MangoLeaf.....54
Hình 4.2. Đường cong huấn luyện của MangoMamba trên VN-MangoLeaf (giai đoạn 3, seed 42).....60
Hình 4.3. Ma trận nhầm lẫn tập kiểm tra MangoLeafBD của checkpoint seed 3333.....62
Hình 4.4. Ma trận nhầm lẫn tập kiểm tra Bộ dữ liệu kết hợp của checkpoint seed 123.....62
Hình 4.5. Ma trận nhầm lẫn tập kiểm tra VN-MangoLeaf của checkpoint seed 42.....63
Hình 4.6. Độ chính xác tập kiểm tra theo seed trên ba bộ dữ liệu của lộ trình ba giai đoạn (trục tung bắt đầu từ 98%).....64
Hình 4.7. Bản đồ Grad-CAM cho bảy lớp của VN-MangoLeaf (checkpoint giai đoạn 3, seed 42). Mỗi hàng gồm ảnh gốc, bản đồ nhiệt Grad-CAM và ảnh phủ.....70
Hình 4.8. Ba trường hợp dự đoán sai đại diện trên VN-MangoLeaf (seed 42); mỗi khối gồm ảnh gốc, bản đồ CAM của lớp thật và bản đồ CAM của lớp dự đoán kèm độ tin cậy.....71

MỞ ĐẦU

Bệnh hại trên lá xoài gây ảnh hưởng đến sinh trưởng, năng suất và chất lượng nông sản. Việc nhận diện sớm qua ảnh có thể hỗ trợ người trồng theo dõi vườn cây; tuy nhiên, mô hình học sâu chỉ có giá trị thực tế khi duy trì được độ chính xác đồng thời phù hợp với giới hạn tính toán của thiết bị triển khai và dữ liệu phản ánh bối cảnh sử dụng.

Luận văn tập trung xây dựng bộ dữ liệu VN-MangoLeaf và phát triển MangoMamba, kiến trúc lai sử dụng Mamba cùng Large-Kernel Attention để phân loại ảnh lá xoài. Mô hình được huấn luyện theo ba giai đoạn trên MangoLeafBD, Bộ dữ liệu kết hợp và VN-MangoLeaf.

Phạm vi nghiên cứu là phân loại ảnh tĩnh thành bảy lớp bệnh trong VN-MangoLeaf. Luận văn không thay thế quy trình chẩn đoán bệnh học tại thực địa; các kết quả nhằm cung cấp cơ sở kỹ thuật cho công cụ hỗ trợ nhận diện ngoại tuyến.

Đóng góp của luận văn gồm bộ dữ liệu VN-MangoLeaf có lớp rỉ sắt, mô hình MangoMamba được đánh giá ở cấu hình chính 320 × 320 và các khảo sát thành phần ở 224 × 224, cùng với phân tích kết quả và giới hạn của các so sánh tham chiếu. Các chương tiếp theo trình bày bối cảnh, cơ sở lý thuyết, phương pháp, kết quả và hạn chế của nghiên cứu.

CHƯƠNG 1. TỔNG QUAN

1.1. Giới thiệu đề tài

Xoài (tên khoa học: Mangifera indica L.) là một trong những cây ăn quả nhiệt đới có giá trị kinh tế cao trên thế giới. Tại Việt Nam, xoài là cây trồng quan trọng ở nhiều địa phương và có tiềm năng đóng góp cho ngành nông nghiệp và xuất khẩu. Vì vậy, việc phát hiện sớm bệnh trên lá xoài có ý nghĩa đối với công tác chăm sóc và quản lý vườn cây.

Tuy nhiên, năng suất và chất lượng xoài có thể bị ảnh hưởng đáng kể bởi các loại bệnh hại. Mức độ thiệt hại phụ thuộc vào loại bệnh, giống cây, điều kiện canh tác và áp lực dịch bệnh tại từng khu vực.

Hiện nay, phương pháp chẩn đoán bệnh truyền thống tại Việt Nam và nhiều quốc gia đang phát triển chủ yếu dựa vào việc kiểm tra trực quan của các chuyên gia nông nghiệp hoặc nông dân giàu kinh nghiệm. Phương pháp này bộc lộ nhiều hạn chế cơ bản: tốn thời gian, mang nặng tính chủ quan và có thể bị nhầm lẫn, đặc biệt khi các triệu chứng bệnh ở giai đoạn đầu hoặc có biểu hiện tương đồng. Hơn nữa, sự thiếu hụt các chuyên gia bệnh học thực vật có trình độ cao tại các địa phương thường dẫn đến việc chẩn đoán chậm trễ, khiến nông dân lạm dụng thuốc bảo vệ thực vật, gây tăng chi phí sản xuất, ô nhiễm môi trường và nguy cơ kháng thuốc.

Sự phát triển của công nghệ, đặc biệt là trong thị giác máy tính và nông nghiệp thông minh đã tạo điều kiện cho các ứng dụng chẩn đoán bệnh cây trồng. Nhiều nghiên cứu đã áp dụng các mô hình như mạng nơ-ron tích chập (CNN), Vision Transformer, mạng capsule (ví dụ CropCapsNet của Qin và cộng sự [31]) và các kiến trúc lai để trích xuất đặc trưng từ ảnh lá bệnh và phân loại triệu chứng.

Mặc dù đạt kết quả khả quan trên các bộ dữ liệu thực nghiệm, việc triển khai các mô hình hiện đại này vào thực tế sản xuất nông nghiệp gặp phải hai rào cản chính:

1. Các mô hình Transformer và CNN cỡ lớn có thể có độ phức tạp tính toán và nhu cầu bộ nhớ cao, đặc biệt khi xử lý ảnh độ phân giải lớn. Chi phí suy luận phụ thuộc vào kiến trúc, kích thước đầu vào, phần cứng, framework và độ chính xác số học; do đó cần được đo trên một cấu hình cụ thể trước khi kết luận về khả năng triển khai trên thiết bị di động hoặc thiết bị biên.

2. Một hạn chế lớn khác là sự thiên vị về mặt địa lý của các bộ dữ liệu hiện có. Phần lớn các bộ dữ liệu phổ biến và được sử dụng rộng rãi (như MangoLeafBD, MLD24) đều được thu thập tại một khu vực địa lý duy nhất là Bangladesh. Việc huấn luyện mô hình trên các bộ dữ liệu này làm giảm khả năng tổng quát hóa khi áp dụng tại các khu vực khác, như Việt Nam, nơi có các chủng bệnh, giống cây trồng, điều kiện ánh sáng và môi trường riêng biệt. Ví dụ, bệnh đốm rong, còn gọi là rỉ sắt (Red Rust), do tảo Cephaleuros virescens gây ra [30], được ghi nhận là bệnh phổ biến trên nhiều loại cây ăn trái, trong đó có xoài, vào mùa mưa tại Việt Nam [1], nhưng lại vắng mặt trong các bộ dữ liệu kể trên.

Từ hai thách thức trên, luận văn này tập trung vào việc phát triển một mô hình học sâu gọn nhẹ cho bài toán chẩn đoán và phân loại bệnh trên lá xoài, đồng thời xây dựng một bộ dữ liệu ảnh phù hợp hơn với điều kiện nông nghiệp ở Việt Nam.

Cụ thể, luận văn này nghiên cứu và đề xuất MangoMamba, một kiến trúc gọn nhẹ dựa trên Mamba [12], kiến trúc mô hình không gian trạng thái (State-Space Model - SSM) có cơ chế quét chọn lọc. Đồng thời, tôi cũng xây dựng bộ dữ liệu VN-MangoLeaf từ các hình ảnh lá xoài thu thập tại khu vực miền Nam Việt Nam, trong đó có lớp bệnh Red Rust để phản ánh phạm vi bệnh được lựa chọn cho dữ liệu mục tiêu.

1.2. Mục tiêu của luận văn này

Mục tiêu tổng quát của luận văn này là giải quyết những thách thức còn tồn đọng của các mô hình học sâu hiện đại khi áp dụng vào bài toán chẩn đoán bệnh lá xoài trong thực tiễn, cụ thể là vấn đề về chi phí tính toán cao và sự thiên vị về mặt địa lý của dữ liệu huấn luyện.

Để thực hiện mục tiêu tổng quát đó, luận văn này đề ra các mục tiêu cụ thể:

1. Luận văn thực hiện nghiên cứu chuyên sâu về cơ sở lý thuyết của các kiến trúc học sâu tiên tiến, tập trung vào CNN, ViT và đặc biệt là mô hình Mamba để nắm bắt ưu nhược điểm của từng cách tiếp cận.
2. Đề xuất và xây dựng kiến trúc mô hình **MangoMamba**, một kiến trúc lai gọn

nhẹ mới, nhằm khai thác hiệu quả tính toán của Mamba, với chi phí tăng tuyến tính theo độ dài chuỗi token.

3. Giải quyết vấn đề thiếu hụt dữ liệu đặc thù bằng cách tiến hành thu thập, gán nhãn và công bố bộ dữ liệu mới **VN-MangoLeaf**, vốn bao gồm các loại bệnh phổ biến tại Việt Nam, bao gồm 7.000 hình ảnh chụp lá xoài với 6 lớp bệnh và 1 lớp bình thường.
4. Đánh giá thực nghiệm hiệu năng của **MangoMamba** và đối chiếu các chỉ số với các mô hình đã được báo cáo. Các so sánh tham chiếu được diễn giải trong bối cảnh khác biệt về phần cứng và cách chia dữ liệu giữa các nghiên cứu. Về triển khai, luận văn đặt yêu cầu thời gian suy luận dưới một giây mỗi ảnh khi chạy ngoại tuyến trên điện thoại phổ thông.

1.3. Nội dung nghiên cứu

Phạm vi và nội dung nghiên cứu của luận văn này được giới hạn trong bài toán phân loại bệnh trên lá xoài dựa trên ảnh tĩnh. Nội dung chính tập trung vào việc nghiên cứu và ứng dụng các kỹ thuật học sâu trong lĩnh vực thị giác máy tính. Luận văn không đi sâu vào các khía cạnh khác của bệnh học thực vật như phân tích gene hay các yếu tố môi trường (đất, nước, không khí), mà tập trung vào việc trích xuất đặc trưng thị giác từ hình ảnh lá bệnh.

Về mặt dữ liệu, nội dung nghiên cứu bao gồm việc phân tích các bộ dữ liệu phổ biến và xây dựng một bộ dữ liệu mới cho nông nghiệp Việt Nam, trong đó bổ sung một lớp bệnh chưa có trong các bộ dữ liệu trước đó. Về mặt mô hình, luận văn tập trung nghiên cứu, thiết kế và tối ưu hóa kiến trúc lai Mamba - Large-Kernel Attention cho bài toán phân loại ảnh. Nội dung nghiên cứu cũng khảo sát hiệu quả của chương trình huấn luyện ba giai đoạn trên dữ liệu từ nhiều nguồn trước khi tinh chỉnh trên VN-MangoLeaf.

1.4. Phương pháp nghiên cứu

- Phương pháp nghiên cứu chính bắt đầu bằng việc tổng quan và phân tích các công trình khoa học đã công bố. Tôi tập trung vào các kiến trúc học sâu như CNN, ViT và Mamba, đồng thời phân tích các bộ dữ liệu bệnh lá xoài để xác

định hạn chế về chi phí tính toán và sự đa dạng địa lý của dữ liệu.

- Từ những hạn chế phát hiện được, tôi thu thập dữ liệu thực địa bằng cách chụp ảnh lá xoài tại các vườn trồng ở Việt Nam để xây dựng bộ dữ liệu VN-MangoLeaf. Quy trình gồm thu thập, làm sạch, tiền xử lý và gán nhãn dữ liệu.
- Tiếp theo, tôi thiết kế và đề xuất kiến trúc MangoMamba dựa trên các phân tích lý thuyết. Kiến trúc này kết hợp các khối Mamba với Large-Kernel Attention nhằm cân bằng độ chính xác phân loại và hiệu quả tính toán, được đánh giá thông qua tốc độ suy luận và kích thước mô hình.
- Cuối cùng, tôi thiết kế các kịch bản thí nghiệm, huấn luyện MangoMamba cùng các cấu hình đối chứng nội bộ trên các bộ dữ liệu được lựa chọn, sau đó so sánh Accuracy, F1-Score, thời gian suy luận và kích thước mô hình; các baseline công bố được đối chiếu theo số liệu tham chiếu.

1.5. Cấu trúc của luận văn này

Ngoài phần Mở đầu, luận văn được tổ chức thành năm chương như sau:

- **Chương 1 (Tổng quan)** trình bày bối cảnh, mục tiêu, nội dung và phương pháp nghiên cứu của luận văn này.
- **Chương 2 (Cơ sở lý thuyết)** trình bày tổng quan về các kiến thức nền tảng liên quan đến học sâu cho thị giác máy tính, phân tích các công trình liên quan, đồng thời giới thiệu chi tiết về các bộ dữ liệu được sử dụng, bao gồm cả quá trình xây dựng bộ dữ liệu mới VN-MangoLeaf.
- **Chương 3 (Kiến trúc và Phương pháp đề xuất)** đi sâu vào việc mô tả kiến trúc mô hình MangoMamba, giải thích từng thành phần cấu thành và cơ chế hoạt động, cùng với đó là trình bày phương pháp luận huấn luyện 3 giai đoạn.
- **Chương 4 (Thực nghiệm và Đánh giá kết quả)** mô tả môi trường cài đặt thực nghiệm, các kịch bản thí nghiệm, trình bày kết quả đạt được và đối chiếu hiệu năng của mô hình đề xuất với các mô hình đã công bố.
- **Chương 5 (Kết luận và khuyến nghị) tổng kết** kết quả nghiên cứu, nêu các hạn chế và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Cơ sở lý thuyết về học sâu cho thị giác

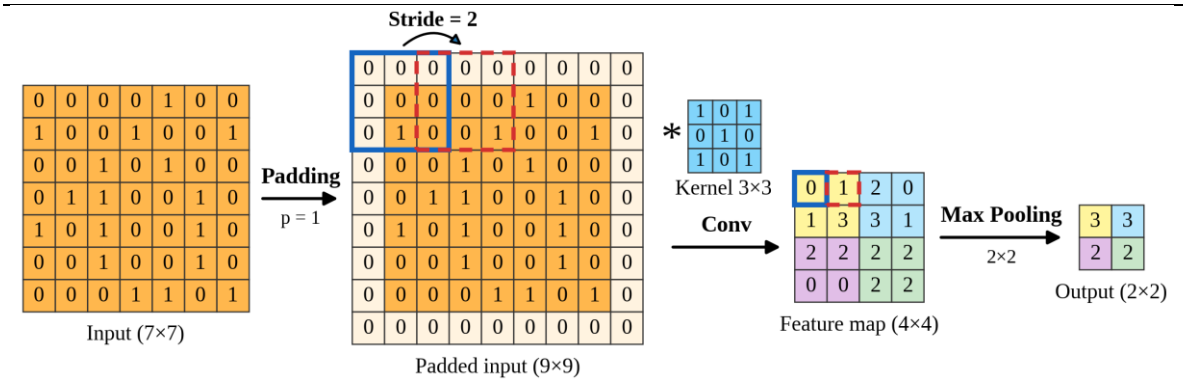
2.1.1. Mạng nơ-ron tích chập (CNN)

Mạng nơ-ron tích chập (Convolutional Neural Network - CNN) là kiến trúc học sâu nền tảng cho các bài toán thị giác máy tính nói chung và phân loại bệnh lá cây nói riêng. Về bản chất toán học, một lớp tích chập thực hiện phép tích chập rời rạc hai chiều giữa đặc trưng đầu vào X và một bộ lọc (kernel) W có kích thước nhỏ (ví dụ 3×3), cộng thêm hệ số điều chỉnh (bias) b, rồi đưa qua một hàm kích hoạt phi tuyến f (ví dụ ReLU):

$$Y[i,j] = f(\Sigma m,n\ X[i+m, j+n] \cdot W[m,n] + b) \quad (2.1)$$

Phép toán này được lặp lại tại mọi vị trí (i, j) trên toàn bộ ảnh, với cùng một bộ trọng số W - đây chính là nguyên lý chia sẻ trọng số (weight sharing) tạo nên hai đặc tính quan trọng của CNN: (i) số lượng tham số của lớp tích chập không phụ thuộc vào chiều cao và chiều rộng của ảnh đầu vào, nhưng phụ thuộc vào kích thước kernel, số kênh đầu vào, số kênh đầu ra và thành phần bias nếu có; với lớp tích chập hai chiều thông thường, số tham số là Kh × Kw × Cin × Cout, cộng thêm Cout nếu dùng bias, trong đó Kh, Kw là kích thước kernel, Cin và Cout lần lượt là số kênh đầu vào và số kênh đầu ra; đặc tính này giúp mô hình gọn nhẹ hơn so với mạng kết nối đầy đủ (fully-connected); và (ii) tính tương đương với phép tịnh tiến (translation equivariance) - một đốm bệnh xuất hiện ở bất kỳ vị trí nào trên lá đều được nhận diện bởi cùng một bộ lọc, phù hợp với bản chất của ảnh bệnh lý thực vật vốn không có vị trí tổn thương cố định.

Sau lớp tích chập, một phép gộp (pooling - thường là lấy giá trị lớn nhất max-pooling hoặc giá trị trung bình average-pooling) được áp dụng trên từng vùng cục bộ nhỏ (ví dụ 2×2) nhằm hai mục đích: giảm kích thước không gian của bản đồ đặc trưng (feature map), qua đó giảm khối lượng tính toán ở các lớp sau, và tăng tính ổn định trước các dịch chuyển nhỏ và nhiễu cục bộ - những yếu tố thường gặp khi chụp ảnh lá cây trong điều kiện thực địa (góc chụp, khoảng cách, độ rung tay).



Hình 2.1. Minh hoạ phép tích chập (kernel 3×3 trượt với stride 2 trên ảnh đầu vào 7×7 đã padding) và phép gộp cực đại (max pooling 2×2) trong CNN (sơ đồ hoá theo [20])

Khi xếp chồng nhiều lớp tích chập và gộp liên tiếp, CNN xây dựng được một hệ thống đặc trưng phân cấp (hierarchical feature representation): các lớp nông (gần đầu vào) học được các đặc trưng cấp thấp và tổng quát như cạnh, góc, gradient màu sắc; trong khi các lớp sâu hơn tổng hợp các đặc trưng cấp thấp này thành các khái niệm ngữ nghĩa cao hơn, chẳng hạn hình dạng đốm bệnh, kết cấu bề mặt lá bị tổn thương hay các mảng màu bất thường đặc trưng cho từng loại bệnh. Nhờ cơ chế học đặc trưng tự động và phân cấp này, CNN đã trở thành lựa chọn mặc định cho hầu hết các nghiên cứu nhận diện bệnh lá xoài giai đoạn đầu, tiêu biểu như LeafNet [34], các mô hình CNN nhẹ của Mahmud và cộng sự [22] hay nghiên cứu học chuyển tiếp (transfer learning) của Varma và cộng sự [42], đều đạt độ chính xác cao trên các bộ dữ liệu chuẩn như MangoLeafBD [3].

Tuy nhiên, hạn chế cơ bản của CNN nằm ở chính cơ chế cục bộ tạo nên sức mạnh của nó: trường tiếp nhận (receptive field) - tức vùng không gian đầu vào mà một điểm trên bản đồ đặc trưng đầu ra có thể nhìn thấy được - của mỗi lớp tích chập chỉ giới hạn trong kích thước kernel. Để một điểm đặc trưng có thể tổng hợp thông tin từ hai đốm bệnh nằm cách xa nhau trên cùng một lá, mạng buộc phải xếp chồng rất nhiều lớp tích chập liên tiếp để trường tiếp nhận mở rộng dần theo cấp số cộng (hoặc cấp số nhân nếu dùng tích chập giãn nở - dilated convolution). Điều này làm tăng độ sâu mạng, số lượng tham số, thời gian huấn luyện và nguy cơ suy giảm gradient (vanishing gradient) khi lan truyền ngược qua nhiều lớp. Hạn chế mang tính cấu trúc này chính là động lực trực tiếp thúc đẩy sự ra đời của các kiến trúc có khả

năng mô hình hóa ngữ cảnh toàn cục ngay từ các lớp đầu tiên, được trình bày lần lượt ở mục 2.1.2 và 2.1.3.

2.1.2. Attention Mechanism và Vision Transformer

Để khắc phục hạn chế về trường tiếp nhận cục bộ của CNN, Attention mechanism - cụ thể là self-attention - cho phép mỗi phần tử trong chuỗi đầu vào (token) tính toán mối liên hệ trực tiếp với mọi phần tử khác trong chuỗi, bất kể khoảng cách giữa chúng, qua đó thiết lập ngữ cảnh toàn cục ngay từ lớp đầu tiên.

Về mặt toán học, với chuỗi đầu vào X (mỗi hàng biểu diễn một token), self-attention trước tiên chiếu X thành ba ma trận Truy vấn (Query), Khoá (Key) và Giá trị (Value) thông qua ba ma trận trọng số học được:

$$Q = XWQ, \quad K = XWK, \quad V = XWV \tag{2.2}$$

Sau đó, mức độ "liên quan" giữa mỗi cặp token được đo bằng tích vô hướng giữa Truy vấn và Khoá, chuẩn hóa theo căn bậc hai của số chiều dk để ổn định phương sai, rồi đưa qua hàm softmax để thu được attention weights có tổng bằng 1, cuối cùng dùng các trọng số này để tính tổng có trọng số của các Giá trị:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V \tag{2.3}$$

Multi-head attention thực hiện nhiều phép chiếu Query, Key và Value song song trên các không gian con khác nhau, sau đó ghép các kết quả lại.

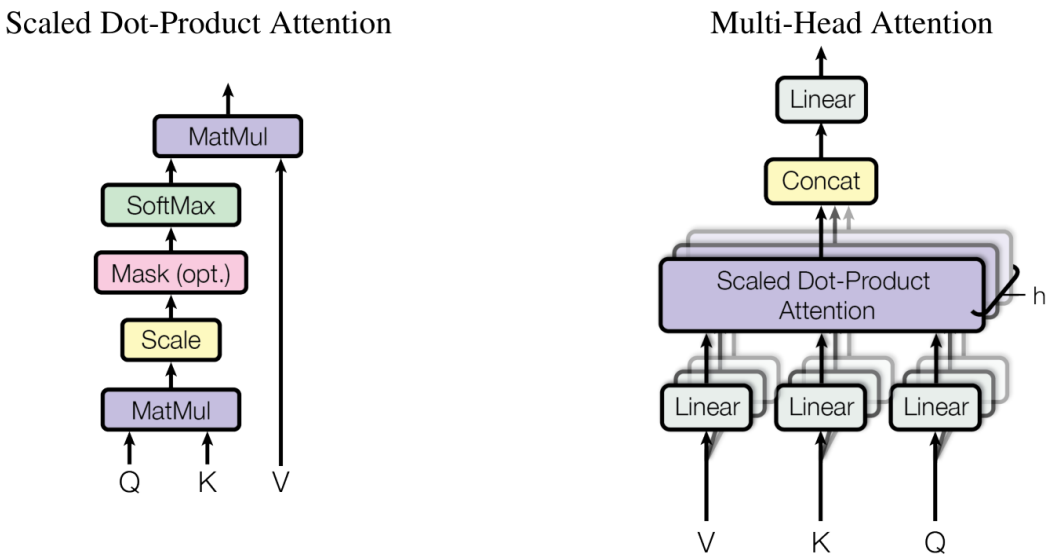
Vision Transformer (ViT) áp dụng self-attention cho dữ liệu ảnh bằng cách chia ảnh đầu vào thành một lưới các mảnh nhỏ không chồng lấp (patch), làm phẳng từng mảnh và chiếu tuyến tính thành một token, sau đó thêm một token phân loại đặc biệt (class token) và embedding vị trí (position embedding) E_{pos} để bù đắp việc mất thông tin thứ tự không gian vốn có của phép self-attention:

$$z0 = [xclass ; xp^1E ; ... ; xp^NE] + Epos \tag{2.4}$$

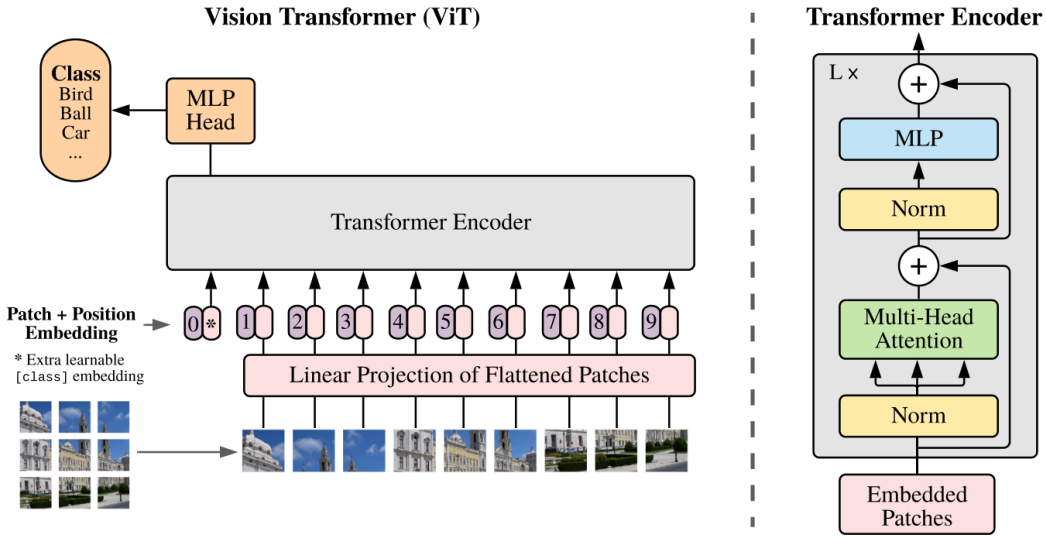
Ở công thức (2.4), N là số lượng patch, E là phép chiếu tuyến tính học được và Epos là embedding vị trí; ký hiệu N ở mục 2.1.2 vì thế khác với ký hiệu N chỉ số chiều trạng thái ẩn ở mục 2.1.3.

Chuỗi token thu được sau đó được đưa qua nhiều lớp mã hoá Transformer (Transformer Encoder), mỗi lớp gồm một khối multi-head attention và một mạng

truyền thẳng, để cuối cùng trạng thái của class token được dùng cho tác vụ phân loại.



Hình 2.2. Minh hoạ self-attention với các thành phần Query, Key, Value (theo [43])



Hình 2.3. Minh hoạ kiến trúc Vision Transformer (ViT): chia ảnh thành các patch, embedding vị trí và xử lý qua Transformer Encoder (theo [10])

Nhiều nghiên cứu đã ứng dụng ViT cho bài toán bệnh lá xoài: Hossain và cộng sự [17] khảo sát ViT trên bộ MangoLeafBD; Alamri và cộng sự [4] đề xuất kiến trúc lai Fused ViT-ConvNeXt kết hợp ưu điểm của cả hai hướng tiếp cận. Tuy nhiên, chính việc mỗi token phải so khớp với tất cả N token còn lại khiến độ phức tạp tính toán và bộ nhớ của self-attention chuẩn tăng theo bậc hai với số lượng token, $O(N^2)$ - hệ quả trực tiếp từ việc ma trận QK^T trong công thức (2.3) có kích thước $N \times N$. Khi

độ phân giải ảnh tăng, số lượng patch (token) tăng theo bình phương, kéo theo chi phí tính toán tăng rất nhanh, gây khó khăn nghiêm trọng cho việc triển khai trên thiết bị biên nơi tài nguyên tính toán, bộ nhớ và năng lượng đều bị giới hạn chặt chẽ.

Để giảm chi phí này, Swin Transformer [21] giới thiệu windowed/shifted-window attention: thay vì tính attention trên toàn bộ ảnh, đặc trưng được chia thành các cửa sổ cục bộ không chồng lấp có kích thước cố định, phép self-attention chỉ được tính trong nội bộ từng cửa sổ, kéo độ phức tạp về gần tuyến tính theo số token, đồng thời tổ chức các lớp theo cấu trúc phân cấp nhiều giai đoạn giống CNN để dần dần mở rộng ngữ cảnh qua các giai đoạn.

DynamicViT [33] tiếp cận vấn đề theo hướng khác, cắt giảm động các token ít quan trọng ngay trong quá trình suy luận nhằm giảm số lượng phép tính mà không cần thay đổi cơ chế attention nền tảng. Các ý tưởng xử lý theo cửa sổ và phân cấp nhiều giai đoạn của Swin Transformer được kế thừa trực tiếp trong thiết kế khối MangoMambaBlock trình bày ở Chương 3; bên trong mỗi cửa sổ, MangoMamba dùng Mamba và Large-Kernel Attention (mục 2.1.3, 2.1.4) làm bộ trộn token thay cho self-attention, đồng thời bổ sung một bước trộn liên cửa sổ ở giai đoạn đầu để trao đổi ngữ cảnh giữa các cửa sổ trên toàn ảnh.

2.1.3. Mamba

Mamba được xây dựng trên mô hình không gian trạng thái (SSM), một khái niệm có nguồn gốc từ lý thuyết điều khiển tuyến tính. SSM mô tả cách một hệ động chuyển đổi tín hiệu đầu vào liên tục $x(t) \in \mathbb{R}$ thành tín hiệu đầu ra $y(t) \in \mathbb{R}$ thông qua một biến trạng thái ẩn $h(t) \in \mathbb{R}^N$, tuân theo hệ phương trình vi phân tuyến tính bậc nhất:

$$h'(t) = Ah(t) + Bx(t), \qquad y(t) = Ch(t) \tag{2.5}$$

Trong đó, $A \in \mathbb{R}N\times N$ là ma trận chuyển trạng thái, $B \in \mathbb{R}N\times 1$ xác định cách đầu vào tác động lên trạng thái và $C \in \mathbb{R}1\times N$ ánh xạ trạng thái thành đầu ra. Trạng thái ẩn $h(t)$ tổng hợp ảnh hưởng của các đầu vào trước đó theo động lực do A và B quy định.

Do dữ liệu thực tế (ảnh, chuỗi token) luôn ở dạng rời rạc, hệ liên tục trên cần

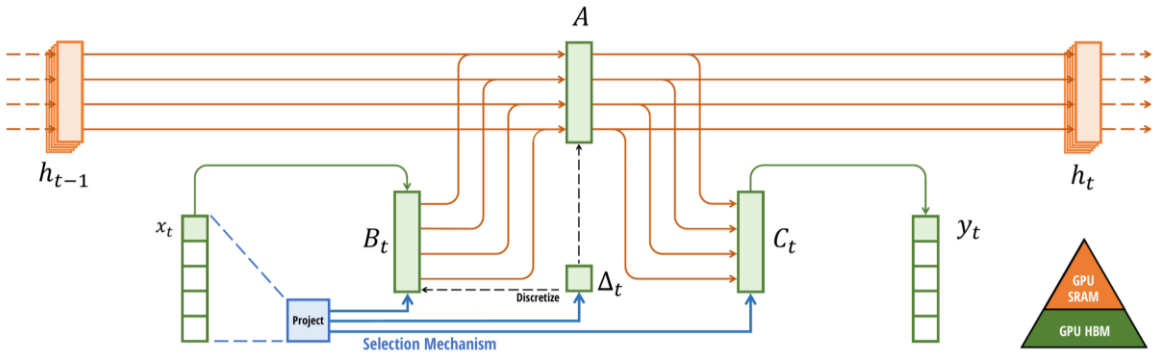
được rời rạc hóa theo một bước thời gian Δ. Phương pháp rời rạc hóa phổ biến nhất - giữ bậc không (zero-order hold) - biến đổi cặp ma trận liên tục (A, B) thành cặp ma trận rời rạc ($\bar{A}$, $\bar{B}$):

$$\bar{A} = \exp(\Delta A), \qquad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)\cdot \Delta B \tag{2.6}$$

Trong cài đặt thực tế, A được tham số hóa dạng đường chéo qua A = -exp(A_log) và Δ thay đổi theo từng token. Hàm selective scan của thư viện mamba-ssm tính $\bar{A} = \exp(\Delta A)$ theo đúng (2.6), còn số hạng đầu vào được tính gọn là $\bar{B}x = \Delta Bx$, tương ứng với xấp xỉ bậc nhất $(\exp(\Delta A) - I)(\Delta A)^{-1} \approx I$ của công thức (2.6). Công thức (2.6) trình bày dạng tổng quát với một bước Δ cố định.

Khi $\bar{A}$, $\bar{B}$ và C không đổi theo bước thời gian, hệ có thể được tính theo dạng đệ quy tuyến tính $h_t = \bar{A}h_{t-1} + \bar{B}x_t$, $y_t = Ch_t$, hoặc tương đương dưới dạng một phép tích chập toàn cục với một kernel dài bằng độ dài chuỗi - cho phép huấn luyện song song hiệu quả trên GPU tương tự CNN. Tính tương đương với tích chập này không còn đúng với Mamba, vì các tham số thay đổi theo đầu vào (xem bên dưới); khi đó phép tính được thực hiện bằng thuật toán quét song song.

Ở cả hai dạng, suy luận (inference) có thể thực hiện tuần tự với bộ nhớ trạng thái hằng số. Nhờ công thức đệ quy tuyến tính này, Mamba xử lý một chuỗi dài L phần tử với độ phức tạp tuyến tính O(L), thay vì bậc hai $O(L^2)$ như self-attention ở mục 2.1.2, vì mỗi bước chỉ cần một phép cập nhật trạng thái, không cần so khớp từng cặp token. Ở đây L là độ dài chuỗi, còn N vẫn là kích thước trạng thái ẩn như trong (2.5).



Hình 2.4. Mô hình không gian trạng thái (SSM) rời rạc hóa trong Mamba: trạng thái ẩn $h_{t-1} \rightarrow h_t$ qua A, B_t và đầu ra y_t qua C_t (theo [12])

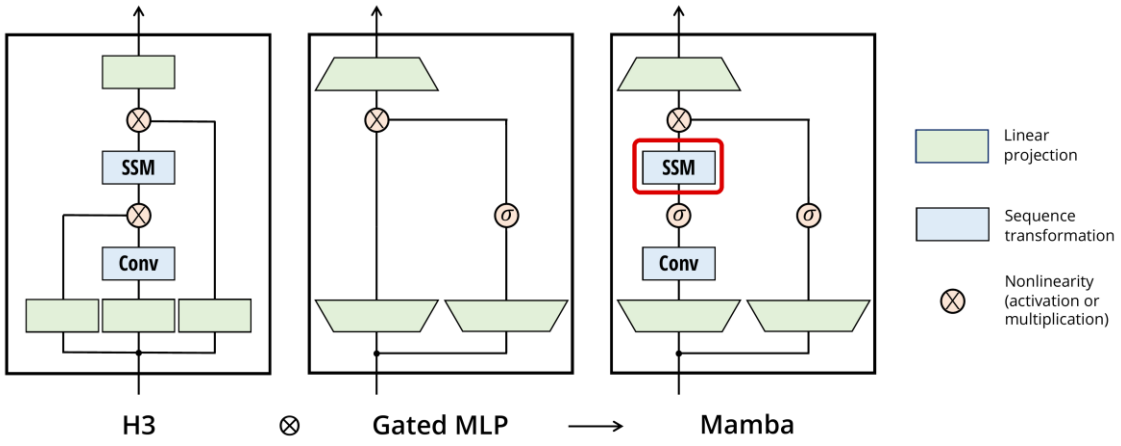
Ở các mô hình không gian trạng thái tuyến tính bất biến theo thời gian như S4

(Structured State Space Sequence Model, mô hình chuỗi không gian trạng thái có cấu trúc) [11], các ma trận A, B, C và tham số rời rạc hóa được học nhưng giữ cố định theo vị trí token trong một chuỗi. Vì vậy, cùng một quy tắc cập nhật trạng thái được áp dụng cho mọi token, không điều chỉnh theo nội dung riêng của từng token.

Kiến trúc Mamba do Gu và Dao [12] đề xuất giải quyết hạn chế này bằng cơ chế quét chọn lọc (selective scan): thay vì cố định, các tham số rời rạc hóa Δ, B, C tại mỗi bước thời gian t được sinh ra động, là hàm số của chính đầu vào xt tại bước đó, thông qua các phép chiếu tuyến tính đơn giản:

$$\Delta t = s\Delta(xt), \quad Bt = sB(xt), \quad Ct = sC(xt) \tag{2.7}$$

Trong đó sΔ, sB và sC là các phép chiếu tuyến tính học được. Mamba làm cho các tham số Δ, B và C phụ thuộc vào đầu vào tại từng bước, nên cập nhật trạng thái có thể thay đổi theo token. Vì các tham số thay đổi theo đầu vào, phép tính không còn là một tích chập cố định như S4; Mamba dùng parallel scan để thực hiện quét hiệu quả trên GPU.

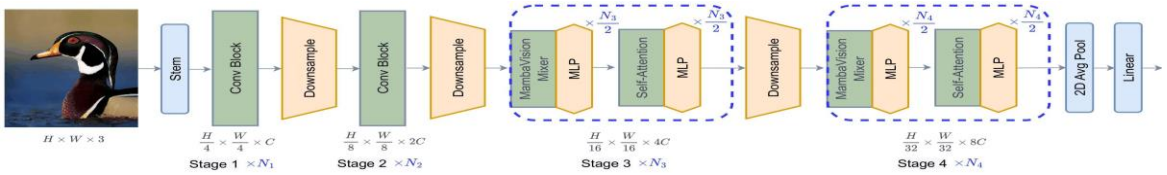


Hình 2.5. Kiến trúc khối Mamba (kết hợp khối H3 và Gated MLP); khung đỏ đánh dấu khối SSM chọn lọc thực hiện cơ chế quét chọn lọc (selective scan) (theo [12])

Nhiều nghiên cứu đã mở rộng Mamba sang lĩnh vực thị giác máy tính:

- Vision Mamba [51] giới thiệu cơ chế quét hai chiều (bidirectional scan), quét chuỗi token đồng thời theo hai hướng thuận - nghịch để khắc phục hạn chế quét một chiều vốn phù hợp hơn với dữ liệu tuần tự (văn bản, âm thanh) hơn là dữ liệu không gian hai chiều như ảnh.
- MambaVision [15] của Nvidia đề xuất kiến trúc lai Mamba-Transformer bốn

giai đoạn, trong đó hai giai đoạn đầu (độ phân giải cao) dùng các khối tích chập để trích xuất đặc trưng nhanh, còn giai đoạn 3 và 4 dùng khối trộn MambaVision cho nửa đầu số lớp và khối self-attention cho nửa sau; self-attention được đặt ở các lớp cuối, nơi số token đã nhỏ, để nắm bắt phụ thuộc không gian tầm xa. MambaVision cũng thiết kế lại bộ trộn Mamba cho dữ liệu ảnh: thay tích chập nhân quả bằng tích chập thông thường, bổ sung một nhánh đối xứng không qua SSM và ghép kênh đầu ra của hai nhánh trước lớp chiếu cuối.



Hình 2.6. Kiến trúc MambaVision của NVIDIA, sơ đồ hóa theo [15].

- Trong nông nghiệp, Zhang và cộng sự [50] áp dụng mô hình VMamba để nhận diện bệnh lá cây trồng và kết hợp với học chuyển tiếp cho tập dữ liệu nhỏ (mục 2.1.5); ngoài phạm vi thị giác, Mamba còn được áp dụng cho các bài toán chuỗi khác, chẳng hạn chẩn đoán lỗi vòng bi công nghiệp bằng học ít mẫu (few-shot learning) của Truong và cộng sự [41].

Trong thị giác máy tính, Vision Mamba [51] và MambaVision [15] báo cáo độ chính xác cạnh tranh với các mô hình Transformer tham chiếu cùng hiệu quả tính toán tốt hơn trên ImageNet; các kết quả này là cơ sở cho việc lựa chọn Mamba làm bộ trộn token chính trong mỗi cửa sổ của kiến trúc MangoMamba ở Chương 3, kết hợp với bước trộn liên cửa sổ để tổng hợp ngữ cảnh trên toàn ảnh, thay vì sử dụng khối Transformer như các kiến trúc lai truyền thống.

2.1.4. Các kỹ thuật liên quan (Large-Kernel Attention, ECA, GRN, SwiGLU)

Ngoài hai dòng kiến trúc chính đã trình bày (CNN và Mamba/Attention), khối MangoMambaBlock (Chương 3) còn tích hợp bốn kỹ thuật bổ trợ nhằm cân bằng giữa hiệu năng và chi phí tính toán. Mục này trình bày cơ chế hoạt động và cơ sở toán học của từng kỹ thuật.

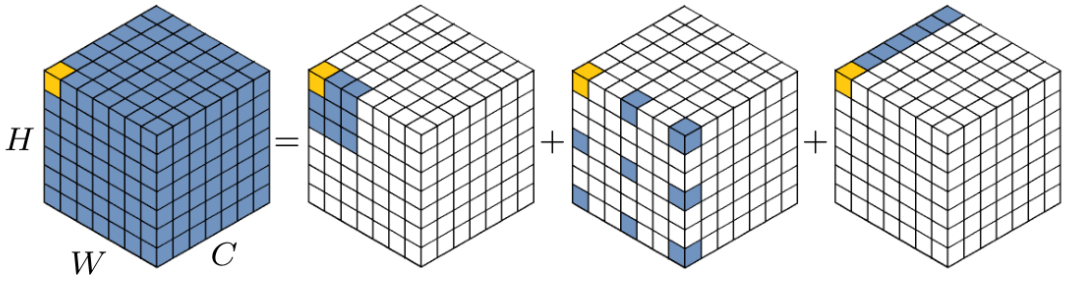
- **Large-Kernel Attention (LKA)**

Được Guo và cộng sự [13] đề xuất trong Visual Attention Network, LKA ra đời từ quan sát rằng một phép tích chập với kernel kích thước lớn (ví dụ 21×21) có thể mô phỏng trường tiếp nhận toàn cục tương tự self-attention, nhưng chi phí tham số và tính toán của một kernel lớn duy nhất tăng theo bình phương kích thước kernel, gây tốn kém. LKA giải quyết vấn đề này bằng cách phân rã một tích chập kernel lớn thành tích của ba phép toán nhỏ hơn nhiều: một tích chập cục bộ theo chiều sâu (depthwise convolution, kernel nhỏ) để nắm bắt phụ thuộc cục bộ, một tích chập giãn nở theo chiều sâu (dilated depthwise convolution) để mở rộng trường tiếp nhận mà không tăng số tham số, và một tích chập 1×1 để trộn thông tin giữa các kênh:

$$\text{Attention} = \text{Conv1}\times1(\ \text{DWDConv}(\ \text{DWConv}(F)\ )\ ) \tag{2.8.1}$$

$$F' = \text{Attention} \otimes F \tag{2.8.2}$$

trong đó F là đặc trưng đầu vào, $\otimes$ là phép nhân từng phần tử (element-wise). Bằng cách phân rã này, LKA tạo ra trường tiếp nhận lớn với chi phí thấp hơn so với việc sử dụng trực tiếp một kernel lớn. LKA không hình thành ma trận tương tác có kích thước bình phương theo số token.



Hình 2.7. Minh hoạ cơ chế phân rã kernel lớn trong Large-Kernel Attention (LKA) (theo [13])

- **Efficient Channel Attention (ECA)**

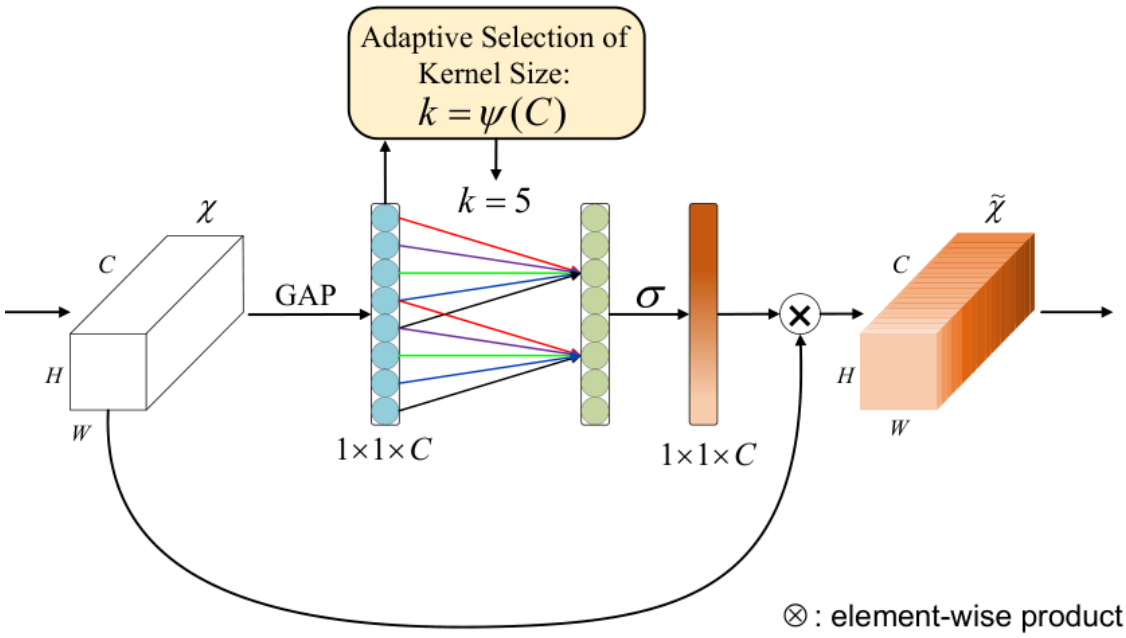
ECA do Wang và cộng sự [44] đề xuất nhằm cải tiến cơ chế channel attention Squeeze-and-Excitation (SE-Net) trước đó. SE-Net sinh channel attention weights bằng hai lớp kết nối đầy đủ có giảm chiều (dimensionality reduction) rồi khôi phục chiều, làm tăng số tham số và - theo phân tích của Wang và cộng sự - việc giảm chiều gây mất mát thông tin, làm suy giảm chất lượng dự đoán trọng số kênh. ECA loại bỏ

hoàn toàn bước giảm chiều, thay vào đó sinh channel attention weights bằng một phép tích chập 1 chiều nhẹ áp dụng trực tiếp trên vector đặc trưng đã gộp trung bình toàn cục (Global Average Pooling - GAP):

$$s = \sigma(\text{Conv1Dk}(\text{GAP}(x))), \quad \hat{x} = x \odot s \tag{2.9.1}$$

$$k = \psi(C) = |\log_2(C)/\gamma + b/\gamma|_{odd} \tag{2.9.2}$$

trong đó σ là hàm sigmoid, k là kích thước kernel của phép tích chập 1 chiều - được xác định thích ứng theo số kênh C theo công thức (2.9.2), trong đó $|t|_{odd}$ là số lẻ gần t nhất; Wang và cộng sự chọn $\gamma = 2$ và $b = 1$ [44], cho phép mô hình hóa tương tác cục bộ giữa các kênh lân cận mà không cần tương tác đầy đủ giữa mọi cặp kênh. Nhờ vậy, ECA gần như không làm tăng thêm số tham số so với mạng gốc, trong khi vẫn giúp mô hình tập trung vào các kênh đặc trưng mang nhiều thông tin bệnh lý.



Hình 2.8. Minh hoạ kiến trúc Efficient Channel Attention (ECA) (theo [44])

- **Chuẩn hóa phản hồi toàn cục (Global Response Normalization - GRN)**

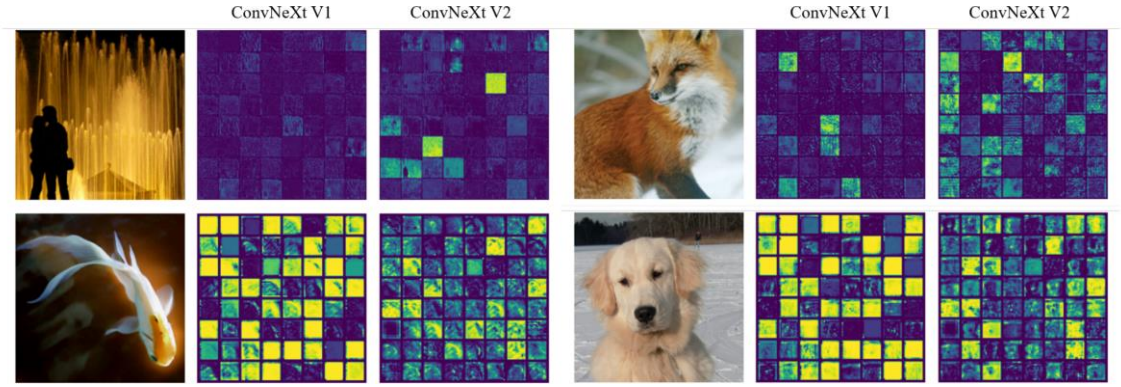
GRN do Woo và cộng sự [46] đề xuất trong ConvNeXt V2. GRN tính phản hồi toàn cục (global response) bằng chuẩn L2 trên hai chiều không gian H và W của từng kênh, rồi chuẩn hóa độ lớn phản hồi này theo giá trị trung bình giữa các kênh (bài báo [46] viết phép chia cho tổng các kênh, còn mã nguồn chính thức dùng trung

bình; hai cách chỉ khác nhau một hằng số C). Cuối cùng áp dụng một phép biến đổi affine có thể học được cùng với kết nối phần dư:

$$nx = \|x\|2 \,/\, (\text{meanC}(\|x\|2) + \varepsilon) \quad (2.10.1)$$

$$GRN(x) = x + \gamma \cdot (x \odot nx) + \beta \quad (2.10.2)$$

Bằng cách khuếch đại các kênh có độ lớn phản hồi cao hơn mức trung bình và làm suy giảm các kênh có phản hồi thấp hơn, GRN thúc đẩy cạnh tranh giữa các kênh và khuyến khích mạng học các đặc trưng đa dạng hơn. Khi mô tả GRN trong MangoMamba, Phụ lục A.2 của bài báo [27] gọi chuẩn L2 này là energy (năng lượng); thuật ngữ đó chỉ cường độ tổng hợp của kích hoạt trong bản đồ đặc trưng, không phải điện năng tiêu thụ. Để tránh nhầm lẫn, luận văn dùng cụm "độ lớn phản hồi" cho đại lượng này.



Hình 2.9. Minh hoạ ảnh hưởng của chuẩn hóa phản hồi toàn cục (Global Response Normalization - GRN) lên độ đa dạng kích hoạt kênh (theo [46])

- **Hàm kích hoạt SwiGLU**

SwiGLU do Shazeer [37] đề xuất là một biến thể của đơn vị tuyến tính có cổng (Gated Linear Unit - GLU), được thiết kế để thay thế các hàm kích hoạt tiêu chuẩn (ReLU, GELU) trong mạng truyền thẳng (Feed-Forward Network - FFN) của các khối Transformer/Mamba. Thay vì áp dụng một hàm phi tuyến đơn giản lên một phép chiếu tuyến tính duy nhất, SwiGLU chiếu đầu vào x qua hai phép chiếu tuyến tính song song V và W, một nhánh được kích hoạt bởi hàm SiLU (Sigmoid Linear Unit, còn gọi là Swish) rồi nhân từng phần tử (cổng - gating) với nhánh còn lại:

$$SiLU(z) = z \cdot \sigma(z) \quad (2.11.1)$$

$$SwiGLU(x, W, V, Wo) = [SiLU(xW) \odot (xV)]Wo \quad (2.11.2)$$

SwiGLU dùng một nhánh đã qua SiLU để điều biến nhánh chiếu còn lại theo

phép nhân từng phần tử. Shazeer đánh giá cơ chế này trên các tác vụ ngôn ngữ; trong MangoMamba, cơ chế được dùng trong nhánh MLP của mỗi block. Ảnh hưởng của việc bỏ SwiGLU được báo cáo ở Bảng 4.4 cho cấu hình thử nghiệm 224 × 224 trên MangoLeafBD.

SwiGLU cũng được dùng trong khối FFN của các mô hình ngôn ngữ lớn (LLM) như PaLM [8] và LLaMA [40]. Tác động thực nghiệm của SwiGLU trong MangoMamba được đánh giá riêng ở Bảng 4.4.

Bốn kỹ thuật LKA, ECA, GRN và SwiGLU trình bày ở trên, cùng với Mamba (mục 2.1.3), là các thành phần cấu thành trực tiếp của khối MangoMambaBlock được mô tả chi tiết ở Chương 3, trong đó mỗi kỹ thuật được cấu hình với các siêu tham số cụ thể (kích thước kernel, hệ số mở rộng, …) phù hợp với đặc thù của bài toán nhận diện bệnh lá xoài.

2.1.5. Học chuyển tiếp và chương trình huấn luyện nhiều giai đoạn

Học chuyển tiếp (transfer learning) sử dụng tri thức đã học trên một miền nguồn để cải thiện dự đoán trên miền đích. Hai miền có thể khác nhau về phân bố ảnh hoặc về tập nhãn. Trong luận văn này, MangoLeafBD và Bộ dữ liệu kết hợp là các miền nguồn; VN-MangoLeaf là miền đích. Cả ba đều gồm ảnh RGB lá xoài, nhưng khác nhau về địa điểm, giống cây, điều kiện chiếu sáng, nền ảnh, thiết bị thu ảnh và số lớp. Do miền đích có dữ liệu gán nhãn, bài toán thuộc nhóm chuyển tiếp quy nạp, dùng trọng số ở giai đoạn trước làm điểm khởi tạo cho giai đoạn sau.

Các tầng nông thường học cạnh, màu sắc và kết cấu tổng quát, còn các tầng sâu hơn mã hóa đặc trưng gắn với lớp mục tiêu [49]. Tái sử dụng trọng số có thể giúp mô hình bắt đầu từ một biểu diễn đã học, nhưng tính hiệu quả phụ thuộc vào độ tương đồng giữa miền nguồn và miền đích [26]. Ở giai đoạn 3, tập nhãn đổi từ tám lớp sang bảy lớp nên đầu phân loại được khởi tạo mới, còn phần trích xuất đặc trưng được nạp từ checkpoint trước.

Lý thuyết thích ứng miền của Ben-David và cộng sự [5] chỉ ra rằng sai số trên miền đích phụ thuộc vào sai số trên miền nguồn, khoảng cách giữa hai miền và khả năng tồn tại một quy tắc dự đoán phù hợp cho cả hai. Học theo chương trình

(curriculum learning) sắp xếp dữ liệu huấn luyện từ dễ đến khó để mô hình tiếp nhận nhiệm vụ dần dần [6, 45]. Luận văn vận dụng tinh thần này ở mức bộ dữ liệu: huấn luyện bắt đầu từ MangoLeafBD, mở rộng sang dữ liệu đa nguồn rồi tinh chỉnh trên VN-MangoLeaf.

Các nghiên cứu trước cũng cho thấy miền trung gian hoặc tinh chỉnh theo từng bước có thể hỗ trợ chuyển tiếp trong một số bài toán [9, 35, 47, 48]. Trong nhận dạng bệnh cây, suy giảm khi chuyển từ dữ liệu có điều kiện thu thập ổn định sang ảnh thực địa cho thấy dịch chuyển miền có thể đáng kể [23]; các nghiên cứu chuyển tiếp theo bước và học trên dữ liệu thực vật cũng gợi ý rằng miền nguồn gần hơn có thể hữu ích [2, 42, 50]. Tuy nhiên, chuyển tiếp âm vẫn có thể xảy ra khi hai miền khác biệt [29], còn tinh chỉnh toàn bộ mạng có thể làm biến đổi đặc trưng đã học [19]. Do đó, tác dụng của giai đoạn trung gian cần được đánh giá bằng đối chứng trực tiếp trên cùng dữ liệu và giao thức, như tại mục 4.3.5.

2.2. Dữ liệu nghiên cứu

Mục này trình bày các bộ dữ liệu ảnh bệnh lá xoài được sử dụng trong luận văn này, bao gồm ba bộ dữ liệu công bố có nguồn gốc từ Bangladesh và bộ dữ liệu VN-MangoLeaf do tôi tự thu thập tại Việt Nam nhằm phục vụ trực tiếp cho chiến lược học theo chương trình (curriculum learning) được trình bày ở Chương 3.

2.2.1. Các bộ dữ liệu công bố (MangoLeafBD, Mango Leaf Disease Dataset, MLD24)

Ba bộ dữ liệu công bố dưới đây có vai trò như miền nguồn (source domain) trong chiến lược học chuyển tiếp của luận văn này, cung cấp lượng lớn dữ liệu đa dạng để mô hình học các đặc trưng tổn thương lá mang tính tổng quát:

1. **MangoLeafBD [3]:** bộ dữ liệu do Ahmed và cộng sự công bố, thu thập tại các vườn xoài thuộc Sher-e-Bangla Agricultural University, Jahangir Nagar University và một số vườn tại Bangladesh, gồm 4.000 ảnh định dạng JPG, độ phân giải 240×320, chia thành 8 lớp: Healthy (khỏe mạnh), Anthracnose, Bacterial Canker, Cutting Weevil, Dieback, Gall Midge, Powdery Mildew và

Sooty Mold, mỗi lớp khoảng 500 ảnh (gồm cả ảnh gốc và ảnh tăng cường).

2. **MLD24 [36]:** bộ dữ liệu của Shakib và cộng sự, thu thập tại khu vực Kushtia và Dhaka (Bangladesh), gồm 6.400 ảnh gốc (không tăng cường), độ phân giải 240×240, 8 lớp như MangoLeafBD, mỗi lớp khoảng 800 ảnh.
3. **Mango Leaf Disease Dataset [32]** do Rahman và cộng sự công bố trên Mendeley Data ngày 13/11/2024, thu thập tại Kashinathpur (Pabna) và Changao (Savar, Dhaka), Bangladesh, bằng camera điện thoại trong 20 ngày (15/10 - 04/11/2024), gồm 2.336 ảnh đã xử lý và 12.730 ảnh tăng cường, chia thành 8 lớp giống với MangoLeafBD; số ảnh đã xử lý của mỗi lớp dao động từ 206 (Powdery Mildew) đến 343 (Bacterial Canker). Mô tả công bố không nêu độ phân giải ảnh.

Bảng 2.1. So sánh các bộ dữ liệu bệnh lá xoài công bố (MangoLeafBD, MLD24, Mango Leaf Disease Dataset) với bộ dữ liệu đề xuất VN-MangoLeaf

Bộ dữ liệu	Độ phân giải	Số ảnh	Số lớp	Ảnh/lớp	Khu vực thu thập
VN-MangoLeaf	240×320 (ảnh gốc 3000×4000)	7.000 (4.900 gốc + 2.100 tăng cường)	7	1.000	Giồng Trôm (Bến Tre); Cao Lãnh (Đồng Tháp); Quận 12, Quận 9, Bình Thạnh (TP.HCM)
MangoLeafBD	240×320	4.000 (1.800 gốc + 2.200 tăng cường)	8	500	Sher-e-Bangla Agricultural University; Jahangir Nagar University; làng Udaypur; làng Itakhola (Bangladesh)
MLD24	240×240	6.400 (toàn bộ gốc)	8	800	Kushtia; Dhaka (Bangladesh)
Mango Leaf Disease Dataset	Không nêu trong mô tả	2.336 ảnh đã xử lý và 12.730 ảnh tăng cường	8	206 - 343 (ảnh đã xử lý)	Kashinathpur (Pabna); Changao (Savar, Dhaka) (Bangladesh)

2.2.2. Phân tích hạn chế của các bộ dữ liệu hiện có

Các bộ dữ liệu công bố nêu trên bộc lộ ba hạn chế chính khi áp dụng cho bối cảnh Việt Nam, mỗi hạn chế đều có ảnh hưởng cụ thể đến chất lượng và độ tin cậy của mô hình khi triển khai thực tế:

1. Thứ nhất, MangoLeafBD, MLD24 và Mango Leaf Disease Dataset đều được thu thập tại Bangladesh. Các đặc trưng hình ảnh như giống xoài, khí hậu, ánh sáng và thiết bị chụp có thể không đại diện cho vườn xoài Việt Nam. Trong học máy, khác biệt giữa phân bố dữ liệu nguồn - đích này được gọi là dịch chuyển miền dữ liệu (domain shift).
2. Thứ hai, khi hợp nhất ba bộ dữ liệu Bangladesh thành Bộ dữ liệu kết hợp (Combined Dataset), số ảnh của mỗi lớp phụ thuộc vào quy mô từng nguồn và có thể không đồng đều. Phân bố này có thể giảm recall của các lớp thiểu số.
3. Thứ ba, Cutting Weevil và Bacterial Canker có trong dữ liệu Bangladesh nhưng hiếm gặp trong các vườn Việt Nam được khảo sát. Ngược lại, Red Rust do Cephaleuros virescens gây ra được đưa vào VN-MangoLeaf nhưng vắng mặt trong các bộ dữ liệu nguồn.

Ba hạn chế này mang tính hệ thống, không thể khắc phục chỉ bằng kỹ thuật tăng cường dữ liệu (data augmentation) thông thường. Đây là động lực trực tiếp để luận văn xây dựng một bộ dữ liệu mới thu thập tại Việt Nam.

2.2.3. Đề xuất bộ dữ liệu mới: VN-MangoLeaf

2.2.3.1. Quy trình thu thập và gán nhãn dữ liệu

Để khắc phục các hạn chế nêu trên, luận văn này (dựa trên công trình đã công bố [27]) giới thiệu bộ dữ liệu VN-MangoLeaf https://www.kaggle.com/datasets/binhpt310/viet-nam-mango-diseases-leaves-dataset, được thu thập tại năm địa điểm thuộc ba tỉnh/thành khu vực Nam Bộ, Việt Nam. Ảnh được chụp bằng điện thoại thông minh Xiaomi Redmi Note 11S ở độ phân giải gốc 3000×4000 pixel, dưới điều kiện ánh sáng tự nhiên. Các địa điểm thu thập được chọn ở nhiều tỉnh/thành nhằm tăng tính đa dạng của điều kiện canh tác và môi trường vườn cây, gồm Quận Bình Thạnh, Quận 12, Quận 9 (Thành phố Hồ Chí Minh), Thành phố Cao Lãnh (tỉnh Đồng Tháp) và huyện Giồng Trôm (tỉnh Bến Tre), như minh hoạ ở Hình 2.10. Địa danh trong luận văn được ghi theo phân chia hành chính của Việt Nam năm 2025, trước đợt sáp nhập tỉnh, phường, xã.



Hình 2.10. Bản đồ các địa điểm thu thập dữ liệu của bộ VN-MangoLeaf tại khu vực Nam Bộ, Việt Nam (Nguồn nền: Google Maps; vị trí thu thập; địa danh theo phân chia hành chính Việt Nam năm 2025)

Tôi gán nhãn ban đầu dựa trên quan sát triệu chứng. Sau đó, các chủ vườn có kinh nghiệm xác minh chéo để phát hiện các trường hợp nhầm bệnh hoặc ảnh không đủ chất lượng. Quy trình này loại bỏ khoảng 2.000 ảnh không đạt chuẩn.



Hình 2.11. Mẫu lá xoài nhiễm bệnh rỉ sắt (Red Rust)

Bộ dữ liệu hoàn chỉnh sau xử lý gồm 7.000 ảnh (đã cân bằng), với tối thiểu 700 ảnh gốc đã xác minh cho mỗi lớp trước khi tăng cường, chia thành 7 lớp: Healthy, Anthracnose, Gall Midge, Powdery Mildew, Sooty Mold, Dieback và Red Rust. So

với các bộ dữ liệu Bangladesh, VN-MangoLeaf không gồm hai lớp hiếm gặp tại các vườn được khảo sát (Cutting Weevil, Bacterial Canker) và bổ sung lớp Red Rust. Lược đồ nhãn này phản ánh các bệnh quan sát được tại các vườn khảo sát.

Phạm vi số liệu xác nhận trong [27] như sau: tổng số ảnh chụp ban đầu trước kiểm định chưa được công bố chi tiết; khoảng 2.000 ảnh đã bị loại và mỗi lớp có tối thiểu 700 ảnh gốc đã xác minh trước khi tăng cường.

2.2.3.2. Đặc điểm và cách nhận biết các bệnh trong bộ dữ liệu VN-MangoLeaf

Phần này mô tả các dấu hiệu hình thái thường gặp của sáu lớp bệnh được sử dụng trong bộ dữ liệu VN-MangoLeaf. Các dấu hiệu trên ảnh chỉ có giá trị nhận biết sơ bộ, vì triệu chứng có thể thay đổi theo giai đoạn bệnh, giống xoài và điều kiện môi trường. Việc xác định tác nhân gây bệnh cần kết hợp với kiểm tra thực địa hoặc đánh giá của chuyên gia khi cần thiết.

- **Thán thư - Anthracnose**

Đặc điểm: Thán thư là bệnh nấm thường liên quan đến các loài thuộc chi Colletotrichum. Trên lá non, bệnh thường tạo các đốm nâu đến đen, hình dạng không đều, có thể lan rộng và liên kết thành vùng cháy lá khi điều kiện ẩm ướt thuận lợi.

Cách nhận biết: Có thể nhận biết sơ bộ bằng các tổn thương sẫm màu, bờ không đều trên lá non hoặc mô lá đang phát triển. Cần phân biệt với Rỉ sắt, vốn thường tạo các đốm nổi có màu đỏ nâu, và Phấn trắng, vốn tạo lớp bột trắng trên bề mặt.

Nguồn giải thích: Mango Anthracnose (Colletotrichum gloeosporioides) - University of Hawai‘i at Mānoa [25]



Hình 2.12. Triệu chứng Thán thư - Anthracnose trên lá xoài

- **Rỉ sắt - Red Rust**

Đặc điểm: Rỉ sắt trên xoài thường liên quan đến tảo Cephaleuros [30]. Tổn thương có thể bắt đầu bằng các đốm tròn màu xám xanh hoặc xanh nhạt, hơi nổi và có bề mặt dạng nhung; về sau chuyển sang màu đỏ nâu hoặc màu gỉ sắt.

Cách nhận biết: Dấu hiệu nhận biết chính là các đốm màu đỏ nâu, hơi nổi hoặc có bề mặt dạng bụi nhung trên lá. Khi nhiều đốm hợp lại, vùng tổn thương có thể lan rộng; không nên nhầm với lớp phủ đen bề mặt của Bồ hóng.

Nguồn giải thích: TNAU Agritech Portal :: Crop Protection - Tamil Nadu Agricultural University [39]



Hình 2.13. Triệu chứng Rỉ sắt - Red Rust trên lá xoài

- **Bồ hóng - Sooty Mold**

Đặc điểm: Bồ hóng là lớp nấm màu đen phát triển trên dịch mật do các côn trùng chích hút tiết ra, chẳng hạn rệp, rệp sáp hoặc rệp vảy. Lớp phủ thường nằm trên bề mặt lá và không nhất thiết là tổn thương trực tiếp do nấm xâm nhập vào mô lá.

Cách nhận biết: Có thể nhận biết bằng lớp phủ đen giống muội than trên mặt lá. Lớp phủ có thể lau hoặc bong một phần, đồng thời thường đi kèm dấu hiệu côn trùng chích hút hoặc bề mặt dính dịch mật. Cần phân biệt với các đốm bệnh nằm trong mô lá như Thán thư.

Nguồn giải thích: The Plant Doctor: Sooty Mold - Mississippi State University Extension Service [16]



Hình 2.14. Triệu chứng Bồ hóng - Sooty Mold trên lá xoài

- **Khô đọt - Dieback**

Đặc điểm: Khô đọt là biểu hiện suy thoái của chồi, cành hoặc nhánh, trong đó mô bị khô thường lan từ phần ngọn về phía gốc. Lá trên cành bị ảnh hưởng có thể héo, cháy mép, chuyển nâu và rụng; vỏ hoặc mô gỗ cũng có thể đổi màu và xuất hiện nhựa.

Cách nhận biết: Dấu hiệu quan trọng là hiện tượng khô chết của chồi hoặc cành đi kèm với triệu chứng trên lá, thay vì chỉ có các đốm riêng lẻ trên phiến lá.

Nguồn giải thích: Disease Control in Mango (Mangifera indica) - University of Florida IFAS [38]



Hình 2.15. Triệu chứng Khô đọt - Dieback trên lá xoài

- **Ghẻ xoài - Gall Midge**

Đặc điểm: Gall Midge là côn trùng gây hại, không phải bệnh do nấm hoặc vi khuẩn. Con trưởng thành đẻ trứng trên mô lá non; ấu trùng sau đó gây các u hoặc nốt phồng nhỏ trên lá, làm biến dạng mô lá và có thể khiến lá non suy yếu hoặc rụng sớm.

Cách nhận biết: Có thể nhận biết bằng các nốt u nổi, vết phồng hoặc vùng biến dạng trên lá non, đôi khi có lỗ thoát của côn trùng. Dấu hiệu này khác với đốm phẳng của Thán thư hoặc lớp phủ bề mặt của Bồ hóng.

Nguồn giải thích: Mango leaf gall midge - Business Queensland [7]



Hình 2.16. Triệu chứng Ghẻ xoài - Gall Midge trên lá xoài

- **Phấn trắng - Powdery Mildew**

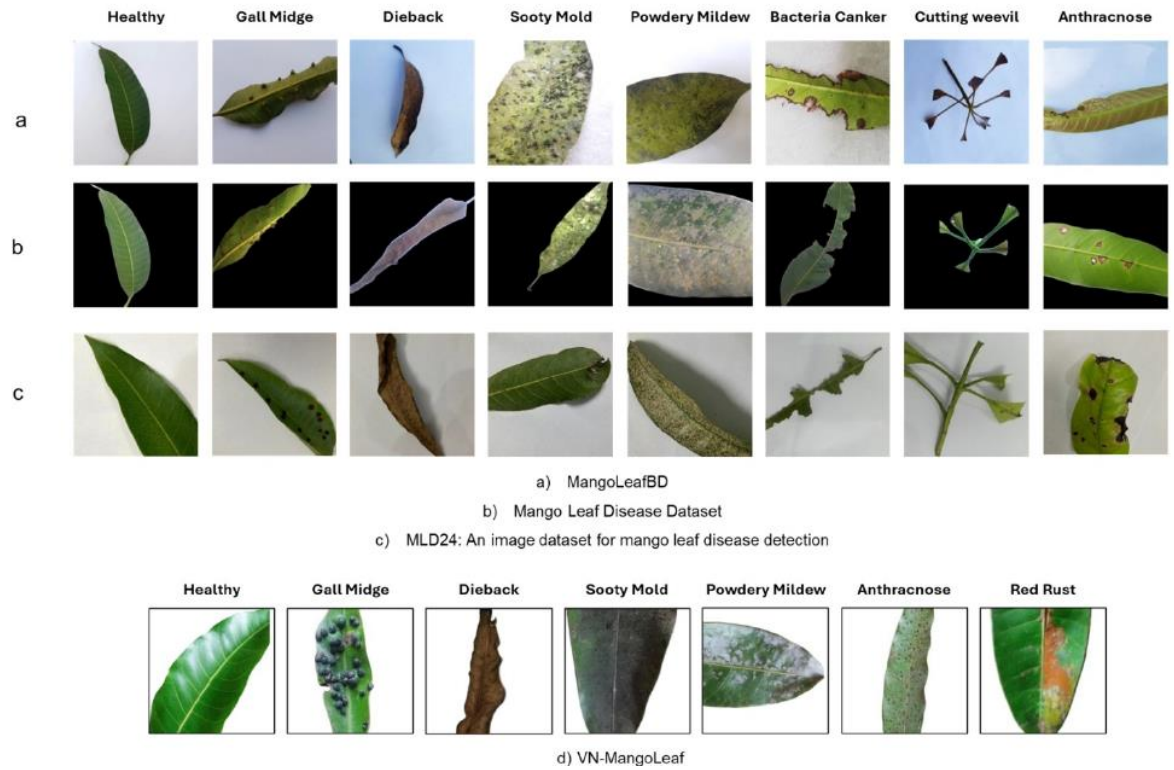
Đặc điểm: Phấn trắng là bệnh nấm tạo lớp sợi hoặc bột màu trắng trên mô non, lá non, chùm hoa và đôi khi trên quả non. Khi bệnh phát triển mạnh, mô non có thể biến dạng, hoa bị ảnh hưởng và tỷ lệ đậu quả giảm.

Cách nhận biết: Dấu hiệu nổi bật là lớp bột trắng có thể nhìn thấy trên bề mặt mô non hoặc cụm hoa. Cần phân biệt với Bồ hóng, vốn có màu đen và thường liên quan đến dịch mật của côn trùng chích hút.

Nguồn giải thích: Mango: Diseases - National Horticulture Board (Ấn Độ) [24]



Hình 2.17. Triệu chứng Phấn trắng - Powdery Mildew trên lá xoài



Hình 2.18. So sánh mẫu ảnh giữa các bộ dữ liệu công bố (MangoLeafBD, Mango Leaf Disease Dataset, MLD24) và bộ dữ liệu đề xuất VN-MangoLeaf

Bộ dữ liệu VN-MangoLeaf được giới thiệu theo giấy phép Creative Commons Attribution 4.0 International (CC BY 4.0). Việc sử dụng, phân phối hoặc tạo phiên bản phái sinh phải tuân thủ điều kiện ghi công và các điều khoản của giấy phép tại nguồn công bố chính thức.

2.2.4. Tiền xử lý và tăng cường dữ liệu

Để cân bằng phân bố lớp của Bộ dữ liệu kết hợp, luận văn áp dụng tăng cường dữ liệu cho các lớp thiểu số, tham khảo các nhóm phép biến đổi (xoay, lật, điều chỉnh độ sáng, co giãn) được Noman và cộng sự sử dụng [28]; khoảng tham số được điều chỉnh riêng như trong Bảng 2.2. Xoay và lật mô phỏng thay đổi về góc chụp và hướng lá; điều chỉnh độ sáng mô phỏng thay đổi ánh sáng; co giãn mô phỏng thay đổi khoảng cách chụp.

Bảng 2.2. Tham số tăng cường và tiền xử lý dữ liệu

Bước xử lý	Mô tả / Tham số
1. Tăng cường (áp dụng ngẫu nhiên để đạt số lượng mục tiêu)	
Xoay ảnh (Rotation)	Khoảng [-45°, 45°]
Lật ảnh (Flipping)	Lật dọc và lật ngang
Độ sáng (Brightness)	Hệ số trong khoảng [0,8; 1,2]
Co giãn (Scaling)	Hệ số trong khoảng [0,5; 1,5]
2. Đổi kích thước (Resizing)	Đưa đầu vào về 320 × 320 khi nạp ảnh; file JPG giữ độ phân giải nêu tại Bảng 2.1.
3. Chuẩn hóa (Normalization)	Chia giá trị điểm ảnh cho 255 (Min-Max về khoảng [0, 1]), sau đó chuẩn hóa theo trung bình và độ lệch chuẩn ImageNet (mục 3.2.1)

Trước tăng cường, Bộ dữ liệu kết hợp gồm khoảng 25.500 ảnh: MangoLeafBD 4.000 ảnh (500 ảnh/lớp) [3], MLD24 6.400 ảnh (800 ảnh/lớp) [36] và Mango Leaf Disease Dataset khoảng 15.000 ảnh, gồm cả ảnh đã xử lý và ảnh tăng cường do nhóm tác giả công bố [32]. Phân bố giữa các lớp không đồng đều: theo Hình 4 của [27], Powdery Mildew là lớp ít ảnh nhất (khoảng 2.300 ảnh), còn Bacterial Canker là lớp nhiều ảnh nhất (khoảng 4.200 ảnh). Sau tăng cường, mỗi lớp có khoảng 5.000 ảnh, tổng cộng 40.000 ảnh.

Mỗi giá trị điểm ảnh x được chuẩn hóa Min-Max về x' theo công thức:

$$x' = (x - xmin) \ / \ (xmax - xmin) \qquad (2.12)$$

Trong đó xmax và xmin lần lượt là cường độ điểm ảnh lớn nhất (255) và nhỏ nhất (0). Chuẩn hóa về khoảng [0, 1] đưa các giá trị đầu vào về cùng thang đo. Bước tăng cường trong Bảng 2.2 được thực hiện để tạo dữ liệu; đổi kích thước và chuẩn hóa được thực hiện khi nạp ảnh. Bài báo [27] báo cáo đổi kích thước ảnh thống nhất về 224 × 224. Trong luận văn này, ảnh nguồn giữ độ phân giải nêu tại Bảng 2.1; cấu hình huấn luyện tạo ảnh đầu vào 320 × 320 bằng phép cắt ngẫu nhiên rồi đổi kích thước (RandomResizedCrop), còn đánh giá dùng Resize cạnh ngắn về 366 pixel rồi CenterCrop về 320 × 320. Vì vậy, 320 × 320 là cấu hình thực nghiệm của luận văn, không phải kích thước đầu vào được báo cáo trong bài báo [27]. Nếu một ảnh đã được

lưu ở 224 × 224 rồi phóng lên 320 × 320, phép biến đổi chỉ nội suy điểm ảnh và không khôi phục chi tiết đã mất. Giá trị pixel được chia cho 255 theo (2.12) khi chuyển sang tensor, sau đó chuẩn hóa theo trung bình và độ lệch chuẩn ImageNet như trình bày tại mục 3.2.1.

Mỗi bộ dữ liệu được chia phân tầng theo tỷ lệ 80:10:10 cho huấn luyện (train), kiểm định (validation) và kiểm tra (test); tập kiểm tra chỉ được dùng để đánh giá cuối cùng và không tham gia quá trình tối ưu.

Bảng 2.3. Phân bố ảnh trên các tập Huấn luyện, Kiểm định và Kiểm tra cho từng bộ dữ liệu thực nghiệm

Bộ dữ liệu	Train (80%)	Validation (10%)	Test (10%)
MangoLeafBD	3.200	400	400
Bộ dữ liệu kết hợp	32.000	4.000	4.000
VN-MangoLeaf	5.600	700	700

2.3. Các độ đo đánh giá hiệu năng mô hình

Luận văn dùng hai nhóm độ đo. Nhóm độ đo phân loại gồm Accuracy, Precision, Recall và F1-score; nhóm độ đo tính toán gồm thời gian suy luận, số tham số và kích thước file trọng số. Các kết quả thực nghiệm được trình bày ở Chương 4.

2.3.1. Các độ đo về độ chính xác (Accuracy, Precision, Recall, F1-Score)

Bốn độ đo phổ biến được sử dụng để đánh giá độ chính xác phân loại: Accuracy (độ chính xác tổng thể), Precision (độ chuẩn xác), Recall (độ bao phủ) và F1-Score, dựa trên bốn đại lượng cơ bản của ma trận nhầm lẫn (confusion matrix): True Positive (TP - số mẫu dương được dự đoán đúng), True Negative (TN - số mẫu âm được dự đoán đúng), False Positive (FP - số mẫu âm bị dự đoán nhầm thành dương) và False Negative (FN - số mẫu dương bị dự đoán nhầm thành âm). Trong bài toán phân loại đa lớp như nhận diện bệnh lá xoài, các đại lượng này được tính riêng cho từng lớp theo chiến lược một-so-với-tất-cả (one-vs-rest); Precision, Recall và F1-Score sau đó được tổng hợp trên các lớp theo trung bình macro (mỗi lớp đóng góp như nhau) hoặc

trung bình có trọng số weighted (theo tỷ lệ số mẫu của mỗi lớp); cách tổng hợp được dùng trong luận văn được ghi rõ ở mục 4.2.

Accuracy đo tỷ lệ số mẫu được phân loại đúng trên tổng số mẫu, cho một cái nhìn tổng quát về hiệu năng nhưng có thể gây hiểu lầm khi dữ liệu mất cân bằng giữa các lớp:

$$\text{Accuracy} = (\Sigma i\ Cii) / (\Sigma i\ \Sigma j\ Cij) \quad (2.13)$$

trong đó C là ma trận nhầm lẫn của bài toán K lớp với K là số lớp, phần tử Cij cho biết số mẫu thuộc lớp i được dự đoán thành lớp j; các phần tử đường chéo Cii là số mẫu được dự đoán đúng. Công thức dựa trên TP, TN, FP và FN chỉ áp dụng khi tính Accuracy riêng cho từng lớp theo cách tiếp cận one-vs-rest.

Precision đo tỷ lệ dự đoán dương chính xác trong số tất cả các dự đoán dương, phản ánh khả năng hạn chế báo động giả (false positive) - quan trọng khi chi phí của việc chẩn đoán nhầm một lá khỏe mạnh thành có bệnh (dẫn đến xử lý thuốc không cần thiết) cần được kiểm soát:

$$\text{Precision} = TP / (TP + FP) \quad (2.14)$$

Recall (hay độ nhạy - sensitivity) đo khả năng mô hình phát hiện đầy đủ các mẫu dương thực sự, phản ánh khả năng phát hiện mọi trường hợp mắc một loại bệnh cụ thể - quan trọng hơn trong bối cảnh nông nghiệp vì bỏ sót một trường hợp bệnh thực sự (false negative) có thể dẫn đến bệnh lây lan không được can thiệp kịp thời:

$$\text{Recall} = TP / (TP + FN) \quad (2.15)$$

F1-Score là trung bình điều hoà (harmonic mean) giữa Precision và Recall, cung cấp một độ đo cân bằng duy nhất, đặc biệt hữu ích khi dữ liệu mất cân bằng giữa các lớp - tình huống thường gặp với các bộ dữ liệu bệnh lá như đã phân tích ở mục 2.2.2:

$$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \quad (2.16)$$

2.3.2. Các độ đo về hiệu quả tính toán (Thời gian suy luận, số lượng tham số, kích thước mô hình)

Độ chính xác không phản ánh đầy đủ khả năng triển khai. Với ứng dụng trên thiết bị di động và thiết bị biên, luận văn còn báo cáo các độ đo chi phí tính toán sau:

1. Thời gian suy luận là thời gian mô hình xử lý một ảnh và trả về dự đoán, thường đo bằng mili giây trên một cấu hình phần cứng cụ thể.
2. Số lượng tham số là tổng số trọng số có thể học của mô hình, thường tính bằng triệu tham số. Đại lượng này phản ánh mức độ phức tạp của mô hình và nhu cầu bộ nhớ khi suy luận.
3. Kích thước mô hình được báo cáo theo hai đại lượng: dung lượng tham số ở dạng FP32, bằng số tham số nhân 4 byte, và dung lượng file mô hình dùng khi triển khai. Đây là giới hạn cần xét khi đóng gói mô hình trong ứng dụng di động.

Ba độ đo hiệu quả tính toán này được tổng hợp để so sánh với các kiến trúc đối chứng (DeiT, Swin Transformer, MobileNetV2, ViX-MangoEFormer [28], ...) trong Chương 4 nhằm chứng minh mức đánh đổi giữa độ chính xác và chi phí triển khai của mô hình đề xuất.

CHƯƠNG 3. KIẾN TRÚC MÔ HÌNH VÀ PHƯƠNG PHÁP ĐỀ XUẤT

3.1. Kiến trúc MangoMamba

Chương này mô tả kiến trúc MangoMamba theo mã nguồn và cấu hình dùng trong thực nghiệm. Mô hình nhận ảnh RGB 320 × 320, gồm backbone phân cấp bốn giai đoạn. Trong mỗi giai đoạn, các khối đầu dùng MultiScaleMambaMixer và các khối sau dùng Large-Kernel Attention (LKA); riêng giai đoạn 1 có thêm một bước trộn liên cửa sổ (inter-window Mamba) đặt giữa khối Mamba và khối LKA (mục 3.1.4). Một số lựa chọn thiết kế được kế thừa từ MambaVision [15]: khối nhúng gồm hai tích chập 3 × 3 bước 2, bộ trộn Mamba dùng tích chập thông thường cùng một nhánh đối xứng không qua SSM, và ý tưởng đặt khối Mamba ở phần đầu mỗi giai đoạn còn khối trộn khác ở phần sau. MangoMamba thay khối self-attention của MambaVision bằng LKA, áp dụng cấu trúc lai này cho cả bốn giai đoạn thay vì chỉ giai đoạn 3 và 4, và bổ sung bước trộn liên cửa sổ.

Về độ phức tạp, selective scan trong từng cửa sổ và tích chập cục bộ của LKA không tạo phép so khớp giữa mọi cặp token trên toàn bộ ảnh. Bước trộn liên cửa sổ ở giai đoạn 1 chạy selective scan trên chuỗi 100 vector đại diện cho 100 cửa sổ, nên chi phí của bước này cũng tăng tuyến tính theo độ dài chuỗi.

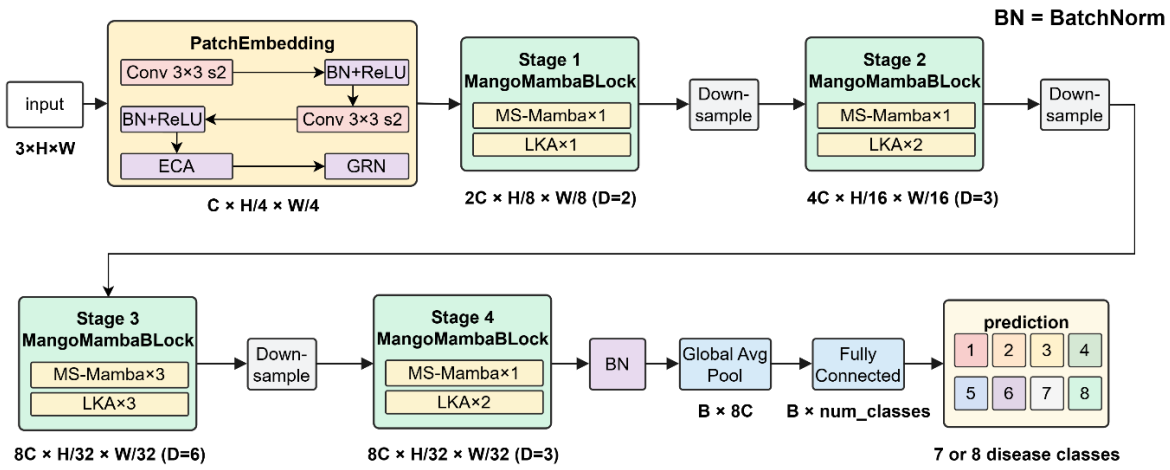
3.1.1. Cấu hình phân cấp và luồng dữ liệu

Sau PatchEmbed, tensor đặc trưng có dạng 48 × 80 × 80. Bốn giai đoạn lần lượt có số kênh [48, 96, 192, 384], độ sâu [2, 3, 6, 3] và kích thước cửa sổ [8, 8, 5, 5]. Với ảnh đầu vào 320 × 320, các bản đồ đặc trưng đều chia hết cho kích thước cửa sổ nên không cần đệm.

Với batch B, đầu vào có dạng B × 3 × 320 × 320. PatchEmbed tạo đặc trưng B × 48 × 80 × 80. Ba giai đoạn đầu lần lượt giảm độ phân giải xuống 40 × 40, 20 × 20 và 10 × 10, đồng thời tăng số kênh lên 96, 192 và 384. Giai đoạn 4 giữ kích thước 10 × 10 để tạo bản đồ đặc trưng cuối B × 384 × 10 × 10. Cách tổ chức này cho phép các giai đoạn đầu giữ chi tiết không gian, còn các giai đoạn sâu làm việc trên biểu diễn

có số kênh lớn hơn.

Kích cỡ cửa sổ [8, 8, 14, 7] được dùng trong MambaVision [15] cho ảnh đầu vào 224 × 224, khi bản đồ đặc trưng qua bốn giai đoạn có kích thước 56 × 56, 28 × 28, 14 × 14 và 7 × 7; cửa sổ 14 × 14 và 7 × 7 ở hai giai đoạn cuối trùng với toàn bộ bản đồ. Nếu áp dụng kích cỡ này cho cả bốn giai đoạn chia cửa sổ của MangoMamba ở 224 × 224, giai đoạn 2 (28/8) cần đệm. Với ảnh 320 × 320, bản đồ đặc trưng ở hai giai đoạn cuối có kích thước 20 × 20 và 10 × 10, không chia hết cho 14 và 7. MangoMamba vì vậy dùng kích cỡ [8, 8, 5, 5] để cả bốn giai đoạn đều chia hết (80/8, 40/8, 20/5, 10/5) và không cần đệm. Cấu hình 224 × 224 với kích cỡ [8, 8, 14, 7] chỉ được dùng trong các thí nghiệm khảo sát ở mục 4.3.1. Luận văn không dùng chênh lệch giữa hai cấu hình để quy kết hiệu quả cho riêng kích cỡ cửa sổ, vì hai cấu hình còn khác nhau ở độ phân giải đầu vào, bước trộn liên cửa sổ và phiên bản kịch bản huấn luyện. Hình 3.1 minh hoạ kiến trúc tổng thể.



Hình 3.1. Kiến trúc tổng thể MangoMamba.

Trong mỗi giai đoạn, đặc trưng được chia thành các cửa sổ không chồng lấp và đổi từ B × C × H × W sang chuỗi B_win × S^2 × C. Mọi khối trong giai đoạn xử lý cùng cách chia cửa sổ này; ngoài bước trộn liên cửa sổ ở giai đoạn 1, thông tin chỉ được trao đổi giữa các cửa sổ qua các khối giảm mẫu, vì tích chập 3 × 3 bước 2 của chúng hoạt động trên toàn bộ bản đồ đặc trưng sau khi các cửa sổ được ghép lại. Sau khối cuối, các cửa sổ được ghép lại thành bản đồ đặc trưng hai chiều và qua GRN. Ba khối giảm mẫu giữa bốn giai đoạn gồm tích chập theo chiều sâu 3 × 3 với stride

2, tích chập điểm 1 × 1 để tăng gấp đôi số kênh, BatchNorm, ECA và GRN.

Ở giai đoạn 1, bản đồ 80 × 80 được chia thành 10 × 10 cửa sổ kích thước 8 × 8; mỗi cửa sổ có 64 token và đi qua hai khối. Sau giảm mẫu, giai đoạn 2 có 5 × 5 cửa sổ 8 × 8 và ba khối. Giai đoạn 3 có 4 × 4 cửa sổ 5 × 5 với sáu khối, còn giai đoạn 4 có 2 × 2 cửa sổ 5 × 5 với ba khối. Như vậy, số token trong một cửa sổ giảm từ 64 ở hai giai đoạn đầu xuống 25 ở hai giai đoạn cuối, trong khi số kênh tăng dần từ 48 lên 384.

Việc chia cửa sổ chỉ thay đổi cách tổ chức tensor trước khi đi vào các khối xử lý. Sau mỗi giai đoạn, WindowReverse khôi phục tensor bốn chiều B × C × H × W; GRN của giai đoạn hoạt động trên tensor này trước khi khối giảm mẫu được gọi. Trong cấu hình dùng cho ảnh 320 × 320, mọi H và W đều chia hết cho kích thước cửa sổ tương ứng. Mã nguồn vẫn có nhánh đệm và cắt phần đệm để hỗ trợ đầu vào khác kích thước, nhưng nhánh này không được kích hoạt trong các thực nghiệm báo cáo.

Các khối của MangoMamba không thực hiện multi-head attention; mọi phép trộn chính đều do MultiScaleMambaMixer, LKA hoặc bước trộn liên cửa sổ ở giai đoạn 1 thực hiện.

3.1.2. PatchEmbed

PatchEmbed gồm hai lớp tích chập 3 × 3 với bước 2; mỗi lớp đi kèm BatchNorm và ReLU. Mô-đun giảm kích thước không gian của ảnh xuống 1/4 trước khi đưa vào backbone.

Lớp tích chập thứ nhất biến đổi 3 kênh RGB thành 16 kênh với kernel 3 × 3, stride 2 và bias = False. Lớp thứ hai tiếp tục biến đổi 16 kênh thành 48 kênh với cùng kernel và stride. Hai lớp đều dùng BatchNorm2d với epsilon 10^{-4} và ReLU tại chỗ. Vì vậy, PatchEmbed vừa thực hiện giảm mẫu hai lần vừa tạo biểu diễn 48 kênh cho giai đoạn đầu; đây là điểm duy nhất đưa ảnh RGB vào backbone.

ECA [44] được áp dụng ngay sau hai lớp tích chập. Mô-đun này gộp trung bình toàn cục theo không gian, dùng tích chập một chiều kernel 3 trên mô tả kênh, đưa kết quả qua sigmoid rồi nhân lại vào từng kênh của đặc trưng. GRN [46] tiếp tục chuẩn

hóa mức đáp ứng giữa các kênh.

ECA không thay đổi kích thước tensor. Với mỗi mẫu, mô-đun gộp mỗi kênh thành một giá trị, lọc chuỗi C giá trị bằng tích chập một chiều kernel 3 và tạo trọng số trong khoảng (0, 1) bằng sigmoid. Trọng số này được phát theo hai chiều không gian rồi nhân với đặc trưng đầu vào. Sau đó, GRN đo chuẩn L2 của từng kênh trên toàn bộ H × W, chuẩn hóa các chuẩn này theo trung bình kênh và dùng hai tham số học được để điều chỉnh đáp ứng. Với đặc trưng đầu vào X, các phép tính của GRN trong mã nguồn được biểu diễn như sau:

$$g_x = \|X\|_{2,(H,W)}, \quad n_x = g_x / (\text{mean}C(g_x) + \varepsilon) \tag{3.1}$$

$$\text{GRN}(X) = X + \gamma \odot (X \odot n_x) + \beta \tag{3.2}$$

Trong đó gamma và beta có kích thước 1 × C × 1 × 1 và được khởi tạo bằng 0. Do đó, GRN ban đầu là ánh xạ đồng nhất và chỉ thay đổi đáp ứng khi các tham số này được cập nhật trong huấn luyện.

3.1.3. Khối MangoMambaBlock

Một MangoMambaBlock áp dụng chuẩn hóa trước hai nhánh phần dư. Nhánh thứ nhất là bộ trộn theo cấu hình của giai đoạn; nhánh thứ hai là MLP dùng SwiGLU. Hai hệ số γ1 và γ2 được học theo từng kênh và cùng khởi tạo bằng 10^{-5}.

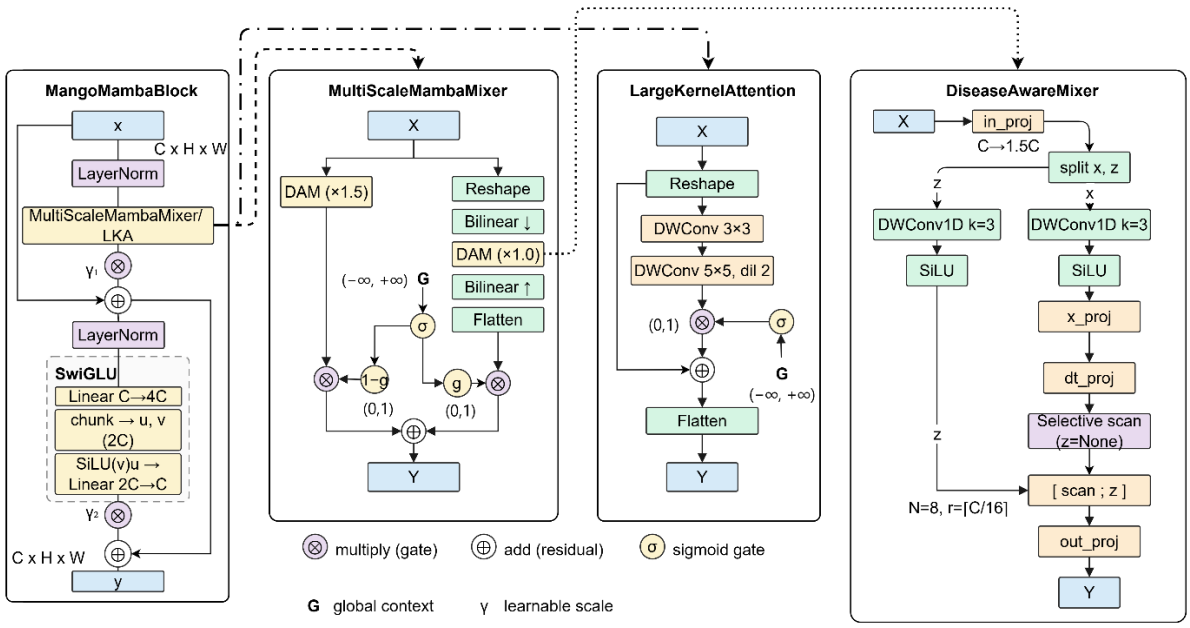
Đầu vào của mỗi khối có dạng B_win × S^2 × C, trong đó B_win là số cửa sổ của toàn batch. LayerNorm chuẩn hóa trên chiều kênh của từng token. Sau đó, đầu ra của bộ trộn được nhân theo kênh với gamma_1, đi qua DropPath và cộng với nhánh phần dư. Nhánh SwiGLU lặp lại cùng nguyên tắc với gamma_2 trên tensor đã cập nhật. Vì hai phép cộng phần dư đều giữ nguyên kích thước B_win × S^2 × C, các khối trong cùng một giai đoạn có thể ghép nối tuần tự mà không cần phép chiếu đổi chiều.

$$x' = x + \text{DropPath}(\gamma_1 \odot \text{Mixer}(\text{LayerNorm}(x))) \tag{3.3}$$

$$y = x' + \text{DropPath}(\gamma_2 \odot \text{SwiGLU}(\text{LayerNorm}(x'))) \tag{3.4}$$

Cấu hình huấn luyện đặt mlp_ratio = 2,5. Mã nguồn nhân tỷ lệ này với 0,8 trước khi tạo tầng ẩn, nên số chiều ẩn của SwiGLU bằng 2C. Phép chiếu đầu tiên sinh hai tensor u và v; đầu ra là SiLU(v) ⊙ u rồi được chiếu về C chiều. DropPath tăng tuyến tính theo độ sâu từ 0 đến 0,01 trong quá trình huấn luyện. Luận văn không khảo sát

riêng ảnh hưởng của giá trị này.



Hình 3.2. Cấu trúc một MangoMambaBlock.

Trong cấu hình hybrid, khác biệt giữa các khối chỉ nằm ở bộ trộn của nhánh thứ nhất. LayerNorm, hai nhánh phần dư, hệ số layer scale, DropPath và SwiGLU được dùng thống nhất. Mã nguồn tạo xác suất DropPath tăng tuyến tính trên toàn bộ 14 khối của backbone; khối đầu có giá trị 0 và khối sâu nhất đạt 0,01. Nhờ đó, biến thiên ngẫu nhiên trong huấn luyện được phân bố theo độ sâu thay vì tập trung ở một giai đoạn riêng lẻ.

Với giai đoạn có độ sâu d, các khối có chỉ số nhỏ hơn floor(d/2) dùng MultiScaleMambaMixer; các khối còn lại dùng LKA. Tương ứng, số khối Mamba và LKA là 1-1 ở giai đoạn 1, 1-2 ở giai đoạn 2, 3-3 ở giai đoạn 3 và 1-2 ở giai đoạn 4. Quy tắc này dựa trên cách MambaVision [15] đặt khối Mamba ở phần đầu và khối self-attention ở phần sau của các giai đoạn 3 và 4; với độ sâu lẻ, MangoMamba dùng floor(d/2) khối Mamba và dành các khối còn lại cho LKA.

3.1.4. MultiScaleMambaMixer

MultiScaleMambaMixer xử lý song song chuỗi token của mỗi cửa sổ ở hai tỷ lệ. Nhánh chính giữ lưới S × S. Nhánh thấp chuyển chuỗi thành bản đồ đặc trưng, nội suy song tuyến tính xuống S_l × S_l, với S_l = max(1, floor(S/2)), xử lý bằng

DiseaseAwareMixer rồi nội suy về S × S. Với cửa sổ 8, nhánh thấp có 16 token; với cửa sổ 5, nhánh thấp có 4 token. Hai nhánh trả về cùng số token và số kênh để có thể hợp nhất.

Hai nhánh được hợp nhất bằng cổng theo kênh. DiseaseAwareMixer của nhánh chính có hệ số mở rộng 1,5, còn nhánh thấp là 1,0. Tham số cổng được khởi tạo bằng 0; sau sigmoid, hai nhánh có cùng trọng số 0,5 tại thời điểm khởi tạo.

Cổng hợp nhất có kích thước 1 × 1 × C nên chỉ thay đổi theo kênh, không thay đổi theo token hay theo mẫu trong cùng một lượt lan truyền. Trong quá trình huấn luyện, từng kênh có thể tăng đóng góp của nhánh chính hoặc nhánh thấp mà không cần một lớp chiếu 2C → C bổ sung.

$$g = \sigma(G) \tag{3.5}$$

$$Y = (1 - g) \odot Yhigh + g \odot Upsample(Ylow) \tag{3.6}$$

Mã nguồn dùng lại MultiScaleMambaMixer cho bước trộn liên cửa sổ ở giai đoạn 1. Bước này được bật mặc định (inter_window_mamba_stages = (0,)) và có mặt trong mọi lượt huấn luyện được báo cáo ở Chương 4, trừ các cấu hình thử nghiệm ở mục 4.3.1 được ghi rõ là không có bước này. Bước trộn được đặt sau khối thứ $\text{floor}(D_1/2) = 1$ của giai đoạn 1, tức sau khối MultiScaleMambaMixer và trước khối LKA. Gọi X_w là chuỗi S^2 token (mỗi token C chiều) của cửa sổ w, với $w = 1, ..., N_w$; ở giai đoạn 1, $N_w = 10 \times 10 = 100$ cửa sổ được xếp theo thứ tự hàng. Mỗi cửa sổ được tóm tắt bằng trung bình các token của nó, rồi chuỗi các vector tóm tắt được xử lý theo hai chiều quét:

$$c_w = (1/S^2) \Sigma_t X_w[t], \quad c = [c_1, ..., c_{Nw}] \tag{3.7}$$

$$\hat{c} = ½ [M_{iw}(LN(c)) + Flip(M_{iw}(LN(Flip(c))))] \tag{3.8}$$

$$X_w \leftarrow X_w + \gamma_{iw} \odot \hat{c}_w, \quad w = 1, ..., N_w \tag{3.9}$$

trong đó M_{iw} là một MultiScaleMambaMixer riêng của bước trộn liên cửa sổ, dùng chung trọng số cho hai chiều quét; LN là LayerNorm; Flip đảo thứ tự chuỗi cửa sổ; γ_{iw} là vector C chiều được khởi tạo bằng 10^{-5}. Vector $\hat{c}_w$ được cộng vào mọi token của cửa sổ w, nên mỗi token nhận thông tin tóm tắt từ các cửa sổ khác trên toàn ảnh trước khi đi vào khối LKA. Với chuỗi 100 vector, nhánh chính của M_{iw} làm việc trên lưới 10 × 10 và nhánh thấp trên lưới 5 × 5. Bước này có 13.992 tham số: mô hình 8

lớp có 5.824.420 tham số khi có bước này và 5.810.428 tham số khi bỏ bước này.

3.1.5. DiseaseAwareMixer

DiseaseAwareMixer áp dụng selective scan của Mamba [12] cho chuỗi token trong từng cửa sổ, theo thiết kế bộ trộn của MambaVision [15]: tích chập nhân quả của Mamba gốc được thay bằng tích chập thông thường, có thêm một nhánh đối xứng không qua SSM, và đầu ra của hai nhánh được ghép kênh rồi chiếu về C chiều.

Cấu hình dùng d_state = 8, d_conv = 3 và dt_rank = max(ceil(C/16), 1). Đầu vào được chiếu lên d_inner chiều qua W_in, rồi tách thành hai nhánh x và z. Cả hai nhánh lần lượt qua tích chập một chiều theo chiều sâu và SiLU. Tích chập này dùng đệm đối xứng (padding = 1 với kernel 3), nên mỗi vị trí nhận thông tin từ cả token liền trước và token liền sau trong chuỗi. Chuỗi token của mỗi cửa sổ được quét theo một chiều, theo thứ tự hàng của lưới S × S; khác với Vision Mamba [51], bên trong cửa sổ không có lượt quét ngược chiều, và mỗi lượt quét chỉ bao phủ 64 token (S = 8) hoặc 25 token (S = 5).

Với nhánh chính, d_inner = 1,5C; lớp chiếu đầu vào không bias biến đổi C chiều thành d_inner chiều rồi tách đều thành hai tensor x và z. Chẳng hạn, tại giai đoạn 1 với C = 48, d_inner = 72 và mỗi tensor có 36 kênh. Nhánh thấp đặt hệ số mở rộng 1,0 nên tại cùng giai đoạn d_inner = 48 và mỗi tensor có 24 kênh. Hai tensor đều đi qua tích chập một chiều theo chiều sâu với kernel 3, sau đó qua SiLU.

$$\begin{aligned} [x, z] &= \text{Split}(\text{Transpose}(\text{Win } u)), \\ x_c &= \text{SiLU}(\text{DWConv1D}(x)), \\ z_c &= \text{SiLU}(\text{DWConv1D}(z)) \end{aligned} \tag{3.10}$$

Từ x_c, một phép chiếu tuyến tính sinh Δ, B và C phụ thuộc vào đầu vào. A được tham số hóa là −exp(A_log), trong đó A_log khởi tạo bằng log của các số nguyên từ 1 đến d_state; D là tham số bỏ qua, khởi tạo bằng 1. Sau khi xây dựng các khối, mô hình khởi tạo lại mọi lớp tuyến tính, gồm cả x_proj và dt_proj, bằng hàm _init_weights: trọng số theo phân phối chuẩn cắt cụt với độ lệch chuẩn 0,02 và bias bằng 0; A_log và D giữ cách khởi tạo nêu trên, còn các tích chập một chiều dùng khởi tạo mặc định của PyTorch. Hàm selective_scan_fn áp dụng softplus cho Δ và trả về

chuỗi s.

Từ tensor x sau tích chập, x_proj sinh ba nhóm tham số có kích thước dt_rank, d_state và d_state cho từng token. dt_proj biến nhóm thứ nhất thành bước thời gian Δ theo từng kênh trong. B và C thay đổi theo đầu vào, còn A và D là các tham số trạng thái được học. selective_scan_fn dùng x, Δ, A, B, C và D để sinh chuỗi trạng thái; lời gọi trong mã nguồn đặt đối số z = None. Điều này phân biệt vai trò của z trong cài đặt này với cơ chế nhân cổng thường gặp ở một số biến thể Mamba.

Sau selective scan, đầu ra s được ghép kênh với z đã qua tích chập rồi mới chiếu về C chiều bằng out_proj không bias. Vì vậy, z là một thành phần của vector đầu ra trước phép chiếu, không phải hệ số nhân trực tiếp với s. Cách viết này cũng giải thích vì sao biểu thức (3.12) dùng phép ghép kênh thay cho phép nhân từng phần tử.

$$s = \text{SelectiveScan}(x_c, \Delta, A, B, C, D) \tag{3.11}$$

$$\text{DAM}(u) = W_{out}\,\text{Concat}(s, z_c) \tag{3.12}$$

3.1.6. Large-Kernel Attention

Các khối LKA mở rộng trường tiếp nhận bằng tích chập theo chiều sâu, dựa trên phép phân rã kernel lớn của VAN [13]. Chuỗi token trong mỗi cửa sổ được đổi thành bản đồ S × S, sau đó qua tích chập 3 × 3 và tích chập 5 × 5 với độ giãn 2. Cài đặt này khác các biểu thức gốc (2.8.1) và (2.8.2) ở hai điểm. Thứ nhất, không dùng tích chập điểm 1 × 1; việc bỏ lớp này giảm số tham số ở các giai đoạn sâu, còn việc trộn thông tin giữa các kênh do các lớp chiếu tuyến tính khác trong khối, như nhánh SwiGLU, đảm nhiệm. Thứ hai, đầu ra a của hai lớp tích chập không được nhân từng phần tử với đặc trưng đầu vào như F' = Attention $\otimes$ F trong (2.8.2), mà được nhân với một cổng sigmoid theo kênh rồi cộng với đầu vào như ở (3.14). Vì vậy, về bản chất đây là một khối tích chập theo chiều sâu kernel lớn có cổng và phần dư; tên LKA được giữ để chỉ nguồn gốc của phép phân rã kernel.

Trong MangoMambaBlock, LKA được khởi tạo với tích chập theo chiều sâu 3 × 3, tiếp theo là tích chập theo chiều sâu 5 × 5 với độ giãn 2. Chuỗi B_win × S^2 × C được đổi thành B_win × C × S × S để thực hiện hai tích chập, rồi đổi ngược về chuỗi token. Không có các phép chiếu Q, K, V hoặc ma trận tương quan token × token trong

đường đi này. Tham số cổng của LKA có kích thước 1 × C × 1 × 1 và được khởi tạo bằng 0; vì sigmoid(0) = 0,5, nhánh tích chập đóng góp với hệ số 0,5 trước khi tham số này được học.

$$a = \mathrm{DWConv}_{5\times5,d=2}(\mathrm{DWConv}_{3\times3}(x)) \tag{3.13}$$

$$\mathrm{LKA}(x) = x + \sigma(G_LKA) \odot a \tag{3.14}$$

G_LKA là tham số riêng của khối LKA, độc lập với tham số cổng G ở mục 3.1.4. LKA làm việc độc lập trong từng cửa sổ, vì vậy nó không tạo ma trận tương quan giữa mọi cặp token như self-attention. Trường tiếp nhận lý thuyết của hai tích chập nối tiếp là 3 + (5 - 1) × 2 = 11 điểm theo mỗi chiều, lớn hơn kích thước cửa sổ 8 × 8 hoặc 5 × 5; do tích chập được thực hiện trên từng cửa sổ với đệm bằng 0, các vị trí nằm ngoài cửa sổ nhận giá trị 0 và LKA không nhìn thấy token của cửa sổ lân cận.

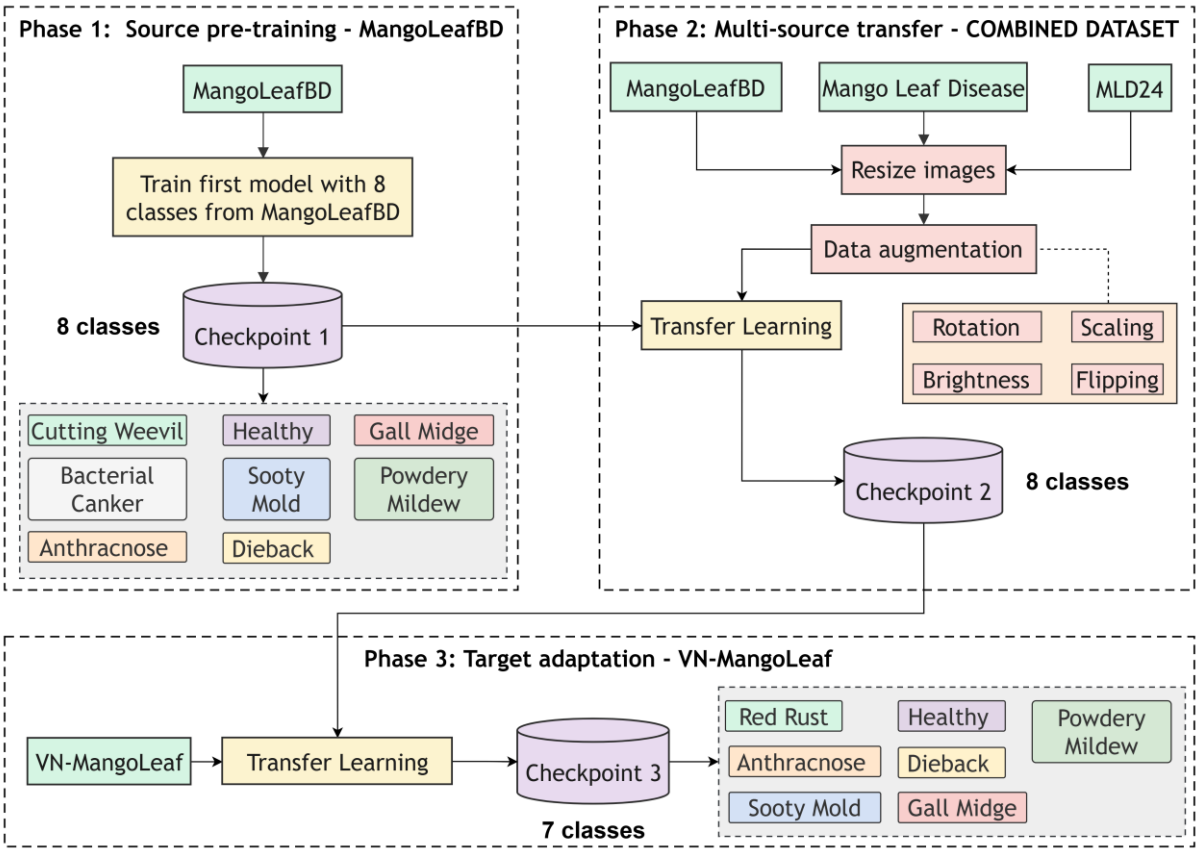
3.1.7. Chuẩn hóa cuối và đầu phân loại

Đầu ra giai đoạn 4 có kích thước 384 × 10 × 10. Mô hình áp dụng BatchNorm2d, gộp trung bình thích nghi về 1 × 1, làm phẳng thành vector 384 chiều và đưa qua lớp tuyến tính. Lan truyền thuận trả về cả logits và vector đặc trưng. Ở hai giai đoạn đầu của chương trình huấn luyện, đầu phân loại có 8 nút. Ở giai đoạn cuối trên VN-MangoLeaf, đầu phân loại có 7 nút và toàn mô hình có 5.824.035 tham số có thể học.

BatchNorm2d cuối cùng hoạt động trên bản đồ B × 384 × 10 × 10. AdaptiveAvgPool2d gộp mỗi kênh thành một giá trị, do đó vector features có kích thước B × 384. Lớp head ánh xạ vector này sang B × 8 ở hai giai đoạn đầu hoặc B × 7 ở giai đoạn mục tiêu. Sự thay đổi từ 8 sang 7 lớp làm số tham số giảm đúng 385, gồm 384 trọng số và một bias của đầu phân loại; vì vậy các lượt chạy 8 lớp có 5.824.420 tham số, còn các lượt chạy 7 lớp có 5.824.035 tham số. Hàm forward trả về đồng thời logits và features để phục vụ phân loại và các bước diễn giải hoặc đóng gói mô hình sau đó.

3.2. Ba giai đoạn huấn luyện mô hình

MangoMamba được huấn luyện qua ba giai đoạn. Giai đoạn 1 dùng MangoLeafBD với 8 lớp. Giai đoạn 2 tinh chỉnh trên Bộ dữ liệu kết hợp và giữ đầu phân loại 8 lớp. Ở giai đoạn 3, backbone được chuyển sang VN-MangoLeaf, đầu phân loại được thay bằng đầu 7 lớp và toàn bộ tham số tiếp tục tinh chỉnh. Mỗi giai đoạn bắt đầu một phiên huấn luyện mới từ checkpoint tốt nhất ở giai đoạn trước.



Hình 3.3. Quy trình huấn luyện theo chương trình ba giai đoạn.

3.2.1. Thiết lập huấn luyện chung

Cả ba giai đoạn dùng ảnh 320 × 320, kích thước cửa sổ [8, 8, 5, 5], chế độ trộn hybrid, ECA, GRN và SwiGLU. AdamW có tốc độ học ban đầu 5×10^{-4}, weight decay 0,01, epsilon 10^{-8} và ngưỡng chuẩn gradient 1,0. Tốc độ học giảm theo lịch cosine đến 5×10^{-6}, sau 5 epoch warm-up, với tối đa 100 epoch. Dừng sớm dựa trên loss kiểm định, với patience = 20 epoch.

Hàm mất mát huấn luyện dùng label smoothing 0,01; kiểm định và kiểm tra

dùng cross-entropy không smoothing. Batch size thực là 64, tích lũy gradient qua 4 bước, nên batch hiệu dụng là 256 khi đủ bốn bước. Huấn luyện sử dụng automatic mixed precision. Tăng cường dữ liệu gồm random resized crop với tỷ lệ diện tích [0,8; 1,0], tỷ lệ khung [3/4; 4/3], lật ngang với xác suất 0,5, lật dọc 0,2, color jitter 0,05 và random erasing 0,05. Ảnh được chuẩn hóa theo mean [0,485; 0,456; 0,406] và std [0,229; 0,224; 0,225].

Các thực nghiệm chính được lặp với bốn seed 42, 123, 2024 và 3333. Tất cả tham số của mô hình đều được cập nhật trong cả ba giai đoạn; mã huấn luyện không đóng băng các tầng thấp.

3.2.2. Giai đoạn 1 trên MangoLeafBD

Ở giai đoạn 1, mô hình được khởi tạo với đầu phân loại 8 lớp và huấn luyện từ đầu trên MangoLeafBD. Checkpoint có loss kiểm định thấp nhất được chọn để chuyển sang giai đoạn 2.

3.2.3. Giai đoạn 2 trên Bộ dữ liệu kết hợp

Ở giai đoạn 2, mô hình vẫn có 8 lớp đầu ra và nạp toàn bộ state_dict của checkpoint giai đoạn 1 với strict = True. Do cấu trúc đầu phân loại không thay đổi, cả backbone và đầu phân loại đều được kế thừa. Checkpoint chuyển sang giai đoạn 3 cũng được chọn theo loss kiểm định, cùng tiêu chí với giai đoạn 1. Optimizer AdamW được tạo mới cho lượt huấn luyện này, sau đó toàn bộ 5.824.420 tham số của mô hình 8 lớp được tinh chỉnh trên Bộ dữ liệu kết hợp.

3.2.4. Giai đoạn 3 trên VN-MangoLeaf

Ở giai đoạn 3, mô hình được khởi tạo với 7 lớp đầu ra rồi nạp checkpoint 8 lớp của giai đoạn 2. Khi phát hiện sai khác kích thước ở head.weight và head.bias, mã huấn luyện loại hai tensor này khỏi checkpoint, nạp các trọng số còn lại với strict = False và giữ đầu phân loại 7 lớp ở trạng thái khởi tạo mới. Toàn bộ backbone và đầu

phân loại mới tiếp tục được tối ưu trên VN-MangoLeaf. Cách nạp này chuyển các biểu diễn đã học nhưng không giả định ánh xạ trực tiếp giữa hai tập nhãn. Checkpoint tốt nhất của giai đoạn 3, dùng cho đánh giá và đóng gói, cũng được chọn theo loss kiểm định.

3.2.5. Giả mã lan truyền thuận và chuyển checkpoint

Giải thuật trình bày ở Hình 3.4 kế thừa cấu trúc của Algorithm 1 trong bài báo Mango-Mamba and VN-MangoLeaf [27] và được trình bày thành hai phần: Phần A mô tả chương trình huấn luyện ba giai đoạn, Phần B mô tả lan truyền thuận. Các ký hiệu được đặt theo vai trò: model là mô hình đang huấn luyện, ckpt1 đến ckpt3 là checkpoint tốt nhất theo loss kiểm định của từng giai đoạn. Các bước được điều chỉnh theo mã nguồn MangoMamba dùng trong luận văn: PatchEmbed đã bao gồm ECA và GRN; mỗi nhánh phần dư được nhân với hệ số layer scale $\gamma 1$ hoặc $\gamma 2$ và đi qua DropPath như ở (3.3) và (3.4); bước trộn liên cửa sổ theo (3.7) đến (3.9) chỉ được gọi ở giai đoạn 1, sau khối Mamba đầu tiên; khi chuyển sang 7 lớp, mô hình nạp checkpoint 8 lớp theo chế độ không nghiêm ngặt và bỏ hai tham số của đầu phân loại; ECA và GRN trong các khối giảm mẫu thuộc Downsample_i.

```
//Input: D_src, D_comb, D_tgt; win = [8, 8, 5, 5]; depths =
[2, 3, 6, 3]; seed ∈ {42, 123, 2024, 3333}.
D_src = MangoLeafBD (8 lớp); D_comb = Bộ dữ liệu kết hợp từ
ba nguồn Bangladesh (8 lớp); D_tgt = VN-MangoLeaf (7 lớp).
//Output: ckpt_phase3, mô hình 7 lớp.
//Phần A: Huấn luyện theo chương trình ba giai đoạn
1: model ← InitMangoMamba(số lớp = 8)
2: ckpt1 ← Train(model, D_src)
3: model.load(ckpt1)
4: ckpt2 ← Train(model, D_comb)
5: model.head ← Đầu phân loại 7 lớp
6: model.load(ckpt2, bỏ head.weight và head.bias)
7: ckpt3 ← Train(model, D_tgt)
8: return ckpt3
```

```
Phần B: Lan truyền thuận (x_rgb: B × 3 × 320 × 320)
x ← GRN(ECA(PatchEmbed(x_rgb)))          # B × 48 × 80 × 80
for i = 1 to 4 do
    x_win ← Chia cửa sổ (x, win_i)
    for j = 1 to depths_i do
        h ← LayerNorm(x_win)
        if j ≤ floor(depths_i / 2) then
            x_win ← x_win + γ1 ⊙ MultiScaleMambaMixer(h)
        else
            x_win ← x_win + γ1 ⊙ LargeKernelAttention(h)
        end if
        x_win ← x_win + γ2 ⊙
SwiGLU_MLP(LayerNorm(x_win))
        if i = 1 and j = floor(depths_1 / 2) then
            x_win ← Trộn liên cửa sổ (x_win)
        end if
    end for
    x ← StageGRN_i(Khôi phục cửa sổ (x_win))
    if i < 4 then
        x ← GRN(ECA(Downsample_i(x)))
    end if
end for
logits ← LinearHead(GlobalAvgPool(BatchNorm(x)))
return {logits, features}
```

Hình 3.4. Giải thuật huấn luyện ba giai đoạn và lan truyền thuận của MangoMamba (điều chỉnh từ Algorithm 1 trong [27])

3.2.6. Cơ sở lựa chọn và phạm vi kết luận của chương trình ba giai đoạn

Chương trình ba giai đoạn được thiết kế dựa trên các nhận định ở mục 2.1.5. Mã huấn luyện không đóng băng tầng nào và dùng cùng tốc độ học ban đầu cho cả ba giai đoạn; warm-up làm giảm độ lớn cập nhật ở các epoch đầu sau khi thay đầu phân loại, nhưng luận văn chưa so sánh với phương án huấn luyện riêng đầu phân

loại trước rồi mới tinh chỉnh toàn mạng (linear probing then fine-tuning - LP-FT) của [19] hoặc với tốc độ học nhỏ hơn cho các giai đoạn tinh chỉnh. Bộ dữ liệu kết hợp có chứa MangoLeafBD, nên giai đoạn 2 mở rộng mức đa dạng của dữ liệu nguồn thay vì tạo một miền hoàn toàn mới. Cách xây dựng này phù hợp về tinh thần với việc mở rộng dần phân bố huấn luyện trong học theo chương trình [6]. Giai đoạn này không chứa ảnh VN-MangoLeaf và không được diễn giải như một miền nội suy giữa miền nguồn và miền đích [18].

$$\theta_1 = \mathcal{A}(D_{src}; \theta_0), \quad \theta_2 = \mathcal{A}(D_{comb}; \theta_1) \tag{3.15}$$

$$\theta_3 = \mathcal{A}(D_{tgt}; (\varphi_2, w_7^0)) \tag{3.16}$$

trong đó θ_0 là khởi tạo ngẫu nhiên, φ_2 là backbone của θ_2 và w_7^0 là đầu phân loại 7 lớp khởi tạo mới. Đối chứng tinh chỉnh trực tiếp ở mục 4.3.5 bỏ giai đoạn 2:

$$\theta_3^{dir} = \mathcal{A}(D_{tgt}; (\varphi_1, w_7^0)) \tag{3.17}$$

So sánh (3.16) với (3.17) cho thấy hai phương án có cùng dữ liệu đích, cùng siêu tham số và cùng cách khởi tạo đầu phân loại; chúng chỉ khác nhau ở điểm khởi tạo backbone, φ_2 so với φ_1. Vì vậy, chênh lệch $\Delta = Acc(\theta_3) - Acc(\theta_3^{dir})$ đo tác động tổng hợp của việc chèn giai đoạn 2, gồm ba yếu tố không tách rời: (i) lượng dữ liệu nguồn lớn hơn khoảng mười lần tính cả ảnh tăng cường (40.000 so với 4.000 ảnh); (ii) mức đa dạng về nguồn thu thập, thiết bị và điều kiện chụp cao hơn; (iii) ngân sách huấn luyện bổ sung trên miền nguồn.

Đối chứng này không tách riêng ảnh hưởng của thứ tự huấn luyện. Để kết luận riêng về thứ tự từ dễ đến khó, cần thêm các đối chứng như huấn luyện giai đoạn 2 từ khởi tạo ngẫu nhiên rồi chuyển sang VN-MangoLeaf, hoặc gộp toàn bộ dữ liệu nguồn trong một giai đoạn với cùng ngân sách huấn luyện. Bảng 3.1 tóm tắt vai trò của từng giai đoạn và cơ sở tương ứng trong tài liệu.

Bảng 3.1. Vai trò của từng giai đoạn trong chương trình huấn luyện

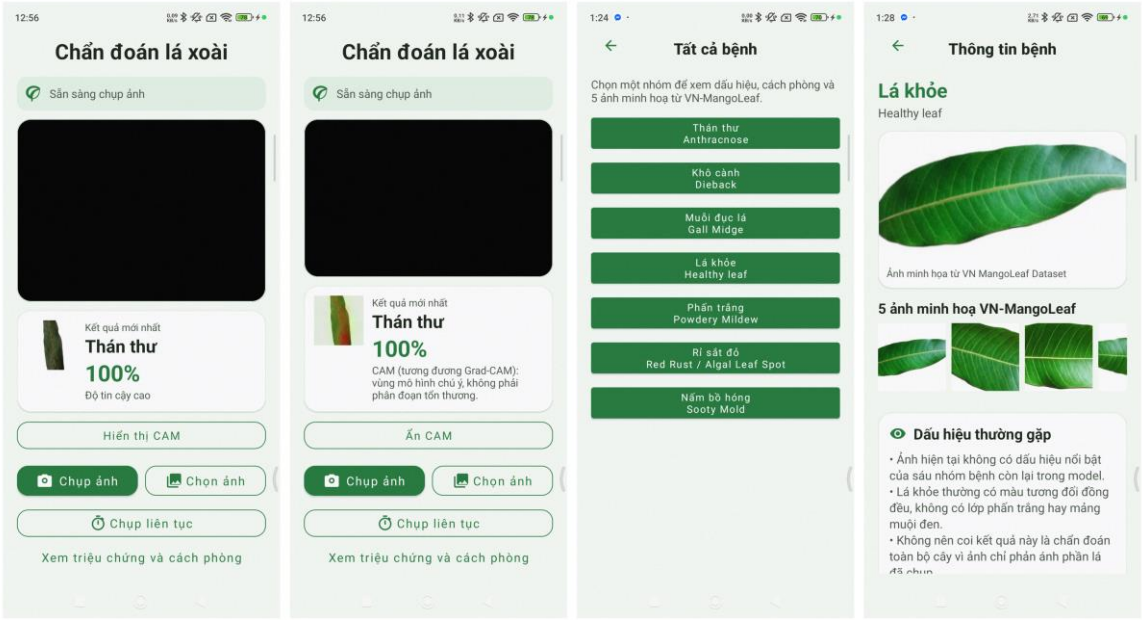
Giai đoạn	Dữ liệu và khởi tạo	Vai trò trong chương trình	Cơ sở tài liệu
1	MangoLeafBD, 8 lớp	Học biểu diễn ban đầu về tổn thương lá xoài trên một bộ dữ liệu nhỏ, cân bằng và có lược đồ nhãn rõ ràng; đóng vai trò giai đoạn khởi đầu của chương trình	Học theo chương trình từ dễ đến khó [6, 45]; đặc trưng lớp nông có tính tổng quát [49]
2	Bộ dữ liệu kết hợp (ba nguồn Bangladesh), 8 lớp	Mở rộng đa dạng nguồn thu thập và điều kiện chụp trong cùng lược đồ nhãn, nhằm giảm phụ thuộc vào đặc điểm riêng của một bộ dữ liệu trước khi chuyển miền	Chuyển tiếp nhiều giai đoạn và miền trung gian [14, 35, 47]; tiền huấn luyện trên dữ liệu cùng lĩnh vực với miền đích hỗ trợ chuyển tiếp [9, 48]
3	VN-MangoLeaf, 7 lớp	Thích ứng với phân bố và lược đồ nhãn của miền đích, trong đó có lớp Red Rust chưa xuất hiện ở miền nguồn	Học chuyển tiếp quy nạp [29]; cận sai số theo khoảng cách miền [5]; nguy cơ biến dạng đặc trưng khi đầu phân loại khởi tạo mới [19]

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

Chương này trình bày môi trường thực nghiệm, kết quả phân loại của MangoMamba trên ba bộ dữ liệu theo ba giai đoạn huấn luyện, phân tích thành phần và chiến lược huấn luyện, so sánh với các mô hình đã công bố, và diễn giải kết quả bằng bản đồ CAM cùng phân tích các trường hợp dự đoán sai. Các thực nghiệm chính dùng cấu hình mô tả ở mục 3.2.1 và được lặp lại với bốn seed; riêng các cấu hình thử nghiệm ở mục 4.3.1 được ghi rõ độ phân giải đầu vào và kích cỡ cửa sổ.

4.1. Ứng dụng chẩn đoán bệnh trên di động

Để đánh giá khả năng triển khai, MangoMamba được tích hợp vào một ứng dụng Android hoạt động ngoại tuyến. Ứng dụng nhận ảnh lá xoài, đưa ảnh về 320 × 320 và xuất nhãn của bảy lớp cùng bản đồ CAM.



Hình 4.1. Giao diện ứng dụng Android trên thiết bị thật: (a) kết quả nhận diện Anthracnose; (b) bản đồ CAM của lớp Anthracnose; (c) danh mục bảy lớp bệnh; (d) trang thông tin bệnh kèm ảnh mẫu từ VN-MangoLeaf

Nhờ đầu ra bản đồ đặc trưng, ứng dụng sinh bản đồ nhiệt chú thích (Class Activation Mapping - CAM) cho lớp dự đoán c theo công thức:

$$\mathrm{CAM}(c, y, x) = \mathrm{ReLU}\left(\Sigma_k F(k, y, x) \cdot W(c, k) \right) \qquad (4.1)$$

trong đó F là bản đồ đặc trưng, W(c, k) là trọng số nối kênh k với lớp c trong đầu phân loại tuyến tính; bản đồ được chuẩn hóa theo giá trị lớn nhất rồi nội suy song tuyến về kích thước ảnh hiển thị.

Ứng dụng được viết bằng Kotlin, cho phép chụp ảnh hoặc chọn ảnh từ thư viện, dự đoán ngoại tuyến và hiển thị CAM kèm nhãn lớp. Thời gian suy luận trên thiết bị được trình bày tại mục 4.4.2.

4.2. Môi trường và thiết lập thực nghiệm

4.2.1. Cấu hình phần cứng và phần mềm

Toàn bộ lượt huấn luyện, đánh giá và đo thời gian suy luận trên máy trạm được thực hiện trên GPU NVIDIA GB10 với 119,6 GiB bộ nhớ hợp nhất, môi trường PyTorch 2.13.0 (CUDA 13.0); thời gian suy luận trên thiết bị di động được đo riêng trên Xiaomi Redmi Note 11S (mục 4.4.2). Huấn luyện dùng trộn độ chính xác tự động; đo thời gian suy luận dùng FP32. Bảng 4.1 liệt kê cấu hình chính.

Bảng 4.1. Cấu hình phần cứng và phần mềm cho thực nghiệm

Thành phần	Cấu hình
GPU	NVIDIA GB10, 119,6 GiB bộ nhớ hợp nhất
Framework	PyTorch 2.13.0+cu130, CUDA 13.0
Thư viện mô hình	timm, mamba-ssm, einops
Độ chính xác	Automatic Mixed Precision (AMP) khi huấn luyện; FP32 khi đo thời gian suy luận
Hệ điều hành	Linux

4.2.2. Các siêu tham số huấn luyện và kỹ thuật điều chuẩn

Cả ba giai đoạn dùng chung một cấu hình siêu tham số, nhất quán với mô tả ở mục 3.2.1. Checkpoint tốt nhất của mỗi lượt được chọn theo loss kiểm định. Bảng 4.2 tổng hợp các giá trị chính.

Bảng 4.2. Siêu tham số huấn luyện và kỹ thuật điều chuẩn

Nhóm	Siêu tham số	Giá trị
Bộ tối ưu	Optimizer	AdamW
	Tốc độ học ban đầu / cuối (cosine)	5×10^{-4} / 5×10^{-6}
	Warm-up / số epoch tối đa	5 epoch / 100 epoch
	Weight decay / epsilon / chuẩn gradient	0,01 / 10^{-8} / 1,0
Dữ liệu	Batch vật lý / tích lũy gradient	64 / 4 bước (hiệu dụng 256)
	Tăng cường dữ liệu	Random resized crop (diện tích 0,8-1,0; tỷ lệ 3/4-4/3), lật ngang 0,5, lật dọc 0,2, color jitter 0,05, random erasing 0,05
	Chuẩn hóa	mean (0,485; 0,456; 0,406), std (0,229; 0,224; 0,225)
Thiết lập	Seed	42, 123, 2024, 3333
	Early stopping	Patience 20 epoch

Mỗi cấu hình chính được lặp lại với bốn seed để đánh giá độ ổn định. Các kết quả dưới đây được báo cáo trên tập kiểm tra chưa tham gia quá trình tối ưu; Precision, Recall và F1 được tính theo trung bình có trọng số (weighted) theo số mẫu của từng lớp; do các tập kiểm tra được cân bằng theo lớp, giá trị này trùng với trung bình macro ở mức chính xác báo cáo, nhưng luận văn gọi tên theo đúng loại độ đo đã tính.

4.3. Đánh giá kết quả phân loại trên các bộ dữ liệu

Mục này trình bày kết quả phân loại trên ba bộ dữ liệu theo thứ tự ba giai đoạn huấn luyện: MangoLeafBD (8 lớp, 400 ảnh kiểm tra), Bộ dữ liệu kết hợp (8 lớp, 4.000 ảnh kiểm tra) và VN-MangoLeaf (7 lớp, 700 ảnh kiểm tra). Mỗi bộ dữ liệu được huấn luyện với bốn seed; các bảng báo cáo kết quả từng seed cùng giá trị trung bình và độ lệch chuẩn mẫu (n = 4).

Việc chia tập được thực hiện phân tầng ở mức bộ dữ liệu; luận văn chưa kiểm chứng việc tách nhóm ảnh gốc - ảnh tăng cường giữa các tập.

4.3.1. Kết quả trên bộ dữ liệu MangoLeafBD

Bảng 4.3 trình bày kết quả bốn seed trên MangoLeafBD. Trên tập kiểm tra, độ chính xác trung bình đạt 99,69 ± 0,24% và F1 weighted đạt 99,69 ± 0,24%. Checkpoint seed 3333 phân loại đúng toàn bộ 400 ảnh kiểm tra (100%) với Precision, Recall và F1 bằng 1,0 trên cả tám lớp; lượt này kết thúc sau 87 epoch do dừng sớm, với checkpoint tốt nhất tại epoch 67 và thời gian huấn luyện khoảng 1 giờ 43 phút.

Bảng 4.3. Kết quả trên bộ dữ liệu MangoLeafBD (tập kiểm tra 400 ảnh)

Seed	Val Acc (%)	Val F1 (%)	Test Acc (%)	Test F1 (%)	Số epoch	Epoch tốt nhất
42	99,25	99,25	99,50	99,50	100	82
123	99,50	99,50	99,50	99,50	90	70
2024	99,75	99,75	99,75	99,75	98	78
3333	99,25	99,25	100,00	100,00	87	67
Trung bình	-	-	99,69 ± 0,24	99,69 ± 0,24	-	-

Trên MangoLeafBD, tôi khảo sát ECA, GRN và SwiGLU trong năm cấu hình lai có LKA giữ cố định, ở đầu vào 224 × 224, kích cỡ cửa sổ [8, 8, 14, 7] và bốn seed. Với từng seed, mô hình tại epoch có validation loss nhỏ nhất được chọn. Kết quả tổng hợp ở Bảng 4.4.

Bảng 4.4. Kết quả khảo sát thành phần trên MangoLeafBD (đầu vào 224 × 224, tập kiểm tra 400 ảnh, n = 4)

Chỉ số	Full	Không ECA	Không GRN	Không SwiGLU	Hybrid tối giản
Thành phần thay đổi	Đủ ECA, GRN, LKA và SwiGLU	Bỏ ECA	Bỏ GRN	Bỏ SwiGLU	Bỏ ECA, GRN và SwiGLU
Epoch tốt nhất	72,5 ± 21,2	53,5 ± 10,5	69,0 ± 5,6	75,0 ± 25,2	76,8 ± 5,4
Validation loss	0,0250 ± 0,0073	0,0194 ± 0,0075	0,0248 ± 0,0069	0,0244 ± 0,0074	0,0170 ± 0,0021
Test Accuracy (%)	99,44 ± 0,24	98,88 ± 0,52	99,31 ± 0,31	99,31 ± 0,38	99,25 ± 0,20
Test F1 (%)	99,44 ± 0,24	98,88 ± 0,52	99,31 ± 0,32	99,31 ± 0,38	99,25 ± 0,20
Test Recall (%)	99,44 ± 0,24	98,88 ± 0,52	99,31 ± 0,31	99,31 ± 0,38	99,25 ± 0,20
Test Precision (%)	99,45 ± 0,24	98,89 ± 0,51	99,33 ± 0,31	99,33 ± 0,37	99,26 ± 0,20

Trong năm cấu hình lai có LKA giữ cố định, cấu hình đầy đủ ECA, GRN và SwiGLU có trung bình cao nhất ở cả bốn chỉ số phân loại trên tập kiểm tra (Accuracy 99,44 ± 0,24%). So với cấu hình này, bỏ ECA làm Accuracy trung bình giảm 0,56 điểm phần trăm; bỏ GRN hoặc SwiGLU giảm 0,13 điểm phần trăm; cấu hình tối giản giảm 0,19 điểm phần trăm. Các mức chênh lệch này chỉ tương ứng khoảng 0,5 đến 2,2 ảnh trên 400 ảnh kiểm tra và không lớn hơn đáng kể độ lệch chuẩn giữa các seed (0,20 đến 0,52 điểm phần trăm). Theo cột validation loss, cấu hình tối giản (0,0170) và cấu hình không ECA (0,0194) còn thấp hơn cấu hình đầy đủ (0,0250).

Vì vậy, khảo sát này không cho thấy thành phần nào tạo khác biệt rõ rệt và không được dùng để chọn cấu hình. Cấu hình đầy đủ ECA, GRN, LKA và SwiGLU được giữ làm cấu hình chính vì đây là thiết kế của MangoMamba đã công bố trong [27]; số liệu tập kiểm tra ở Bảng 4.4 chỉ mang tính mô tả. Vì LKA được giữ cố định trong các phép so sánh, kết quả này cũng không ước lượng riêng đóng góp biên của LKA.

4.3.2. Kết quả trên Bộ dữ liệu kết hợp

Trên Bộ dữ liệu kết hợp 40.000 ảnh, Bảng 4.5 cho thấy độ chính xác kiểm tra đạt 99,48 ± 0,07% và F1 weighted đạt 99,48 ± 0,07%. Kết quả này được đo trên tập kiểm tra khác với giai đoạn 1 nên không so sánh trực tiếp với Bảng 4.3. Độ lệch chuẩn giữa các seed nhỏ (± 0,07), cho thấy huấn luyện ổn định trên bộ dữ liệu lớn hơn. Cả bốn lượt chạy đủ 100 epoch, checkpoint tốt nhất nằm trong khoảng epoch 84 đến 96.

Bảng 4.5. Kết quả trên Bộ dữ liệu kết hợp (tập kiểm tra 4.000 ảnh)

Seed	Val Acc (%)	Val F1 (%)	Test Acc (%)	Test F1 (%)	Số epoch	Epoch tốt nhất
42	99,75	99,75	99,48	99,47	100	84
123	99,78	99,77	99,58	99,57	100	94
2024	99,85	99,85	99,45	99,45	100	92
3333	99,80	99,80	99,43	99,42	100	96
Trung bình	-	-	99,48 ± 0,07	99,48 ± 0,07	-	-

4.3.3. Kết quả trên bộ dữ liệu VN-MangoLeaf

Bảng 4.6 tổng hợp kết quả trên VN-MangoLeaf sau khi chuyển đầu phân loại sang 7 lớp và tinh chỉnh toàn bộ mô hình. Độ chính xác kiểm tra trung bình đạt 98,89 ± 0,29% và F1 weighted đạt 98,89 ± 0,29%. Checkpoint seed 42 đạt 99,29% (695/700 ảnh) với F1 weighted 99,28%.

Bảng 4.6. Kết quả trên bộ dữ liệu VN-MangoLeaf (tập kiểm tra 700 ảnh)

Seed	Val Acc (%)	Val F1 (%)	Test Acc (%)	Test F1 (%)	Số epoch	Epoch tốt nhất
42	99,00	99,00	99,29	99,28	60	40
123	99,00	99,00	98,86	98,86	63	43
2024	98,86	98,85	98,57	98,57	53	33
3333	98,57	98,57	98,86	98,86	47	27
Trung bình	-	-	98,89 ± 0,29	98,89 ± 0,29	-	-

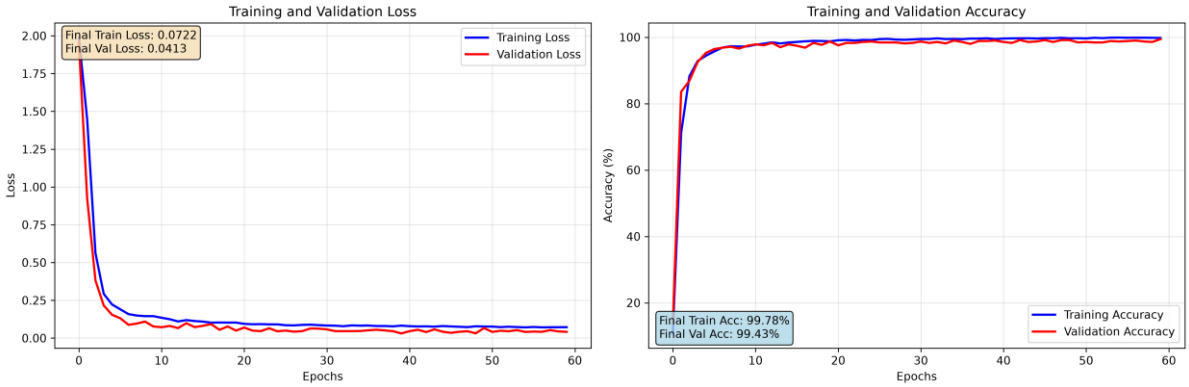
Bảng 4.7 trình bày chỉ số theo từng lớp của checkpoint seed 42. Cả bảy lớp đạt

recall từ 0,95 đến 1,00. Riêng Red Rust, lớp bổ sung đặc thù của Việt Nam, đạt recall 1,00 và precision 0,98: mô hình nhận diện đủ các mẫu rỉ sắt trong tập kiểm tra, chỉ nhầm một số mẫu của lớp khác thành Red Rust.

Bảng 4.7. Chỉ số theo từng lớp trên VN-MangoLeaf của checkpoint seed 42

Lớp	Precision	Recall	F1-score
Healthy	1,00	1,00	1,000
Anthracnose	1,00	0,95	0,974
Gall Midge	0,990	1,00	0,995
Powdery Mildew	1,00	1,00	1,000
Sooty Mold	0,980	1,00	0,990
Dieback	1,00	1,00	1,000
Red Rust	0,980	1,00	0,990

Hình 4.2 minh hoạ đường cong huấn luyện của lượt seed 42: loss huấn luyện và loss kiểm định giảm ổn định, độ chính xác kiểm định hội tụ quanh mức 99%. Trên đường cong được minh hoạ, luận văn xét hai tiêu chí: validation loss không tăng trở lại về cuối quá trình huấn luyện và khoảng cách giữa validation loss với training loss không mở rộng liên tục; theo hai tiêu chí này, lượt seed 42 không cho thấy dấu hiệu overfitting rõ rệt, và nhận định này chỉ áp dụng cho lượt seed 42 được vẽ.

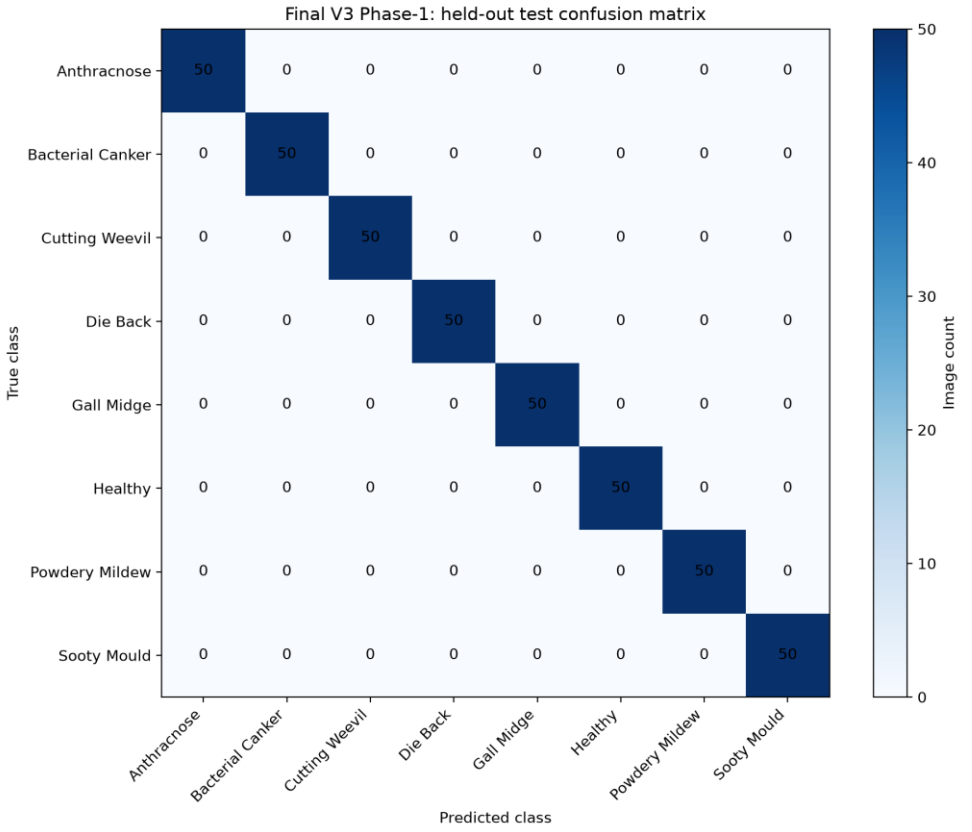


Hình 4.2. Đường cong huấn luyện của MangoMamba trên VN-MangoLeaf (giai đoạn 3, seed 42)

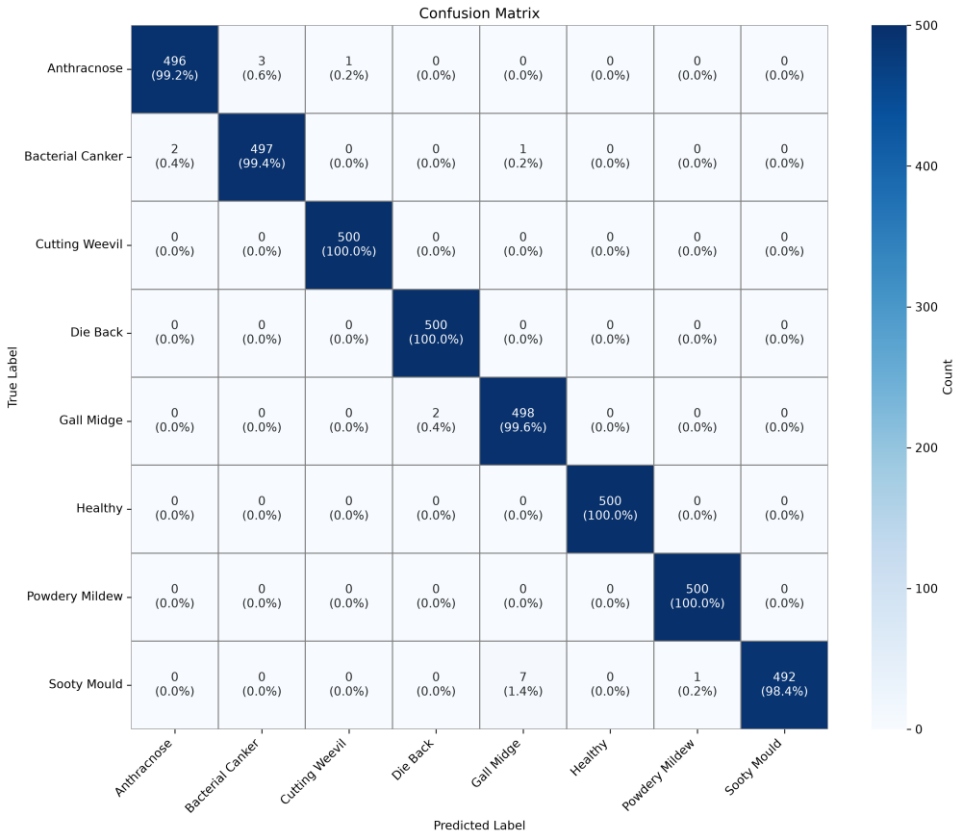
4.3.4. Phân tích ma trận nhầm lẫn

Hình 4.3, Hình 4.4 và Hình 4.5 trình bày ma trận nhầm lẫn trên tập kiểm tra của checkpoint tốt nhất từng bộ dữ liệu. Checkpoint này được chọn hậu nghiệm theo độ chính xác kiểm tra cao nhất trong bốn seed (seed 3333 cho MangoLeafBD, seed 123 cho Bộ dữ liệu kết hợp và seed 42 cho VN-MangoLeaf); đây là lựa chọn mô tả cho trực quan hóa, không phải giao thức chọn không thiên lệch, do đó các nhận định về cấu trúc lỗi chỉ áp dụng cho checkpoint được nêu. Trên MangoLeafBD, checkpoint seed 3333 phân loại đúng cả 400 ảnh nên ma trận thuần đường chéo. Trên Bộ dữ liệu kết hợp, ma trận của seed 123 (99,58%) tập trung gần trọn vẹn ở đường chéo với số lỗi rải rác nhỏ giữa một số cặp lớp có triệu chứng tương đồng.

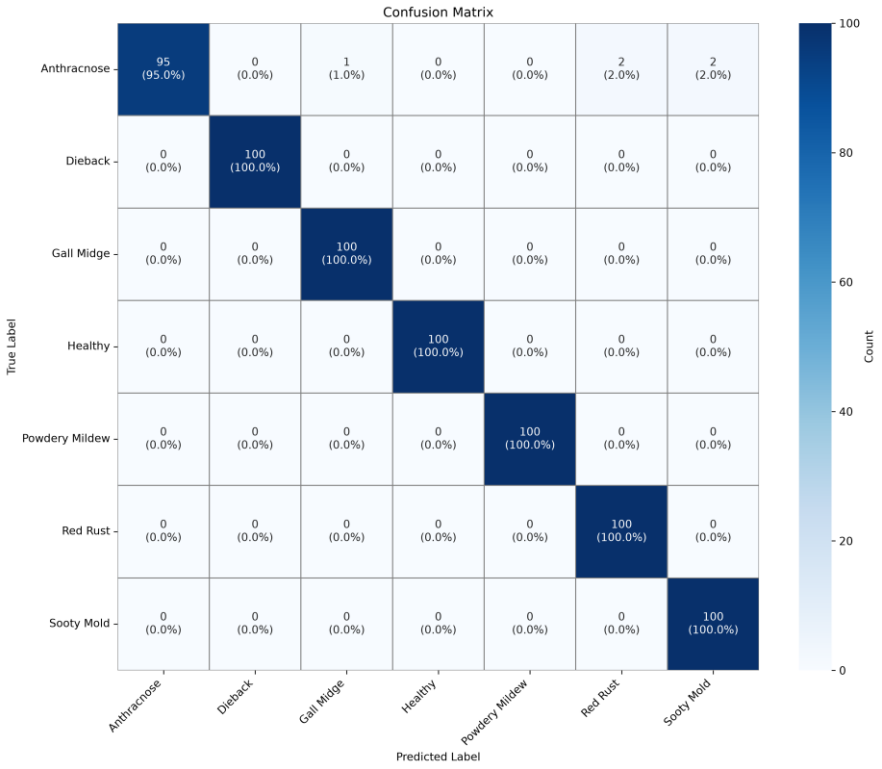
Trên VN-MangoLeaf (Hình 4.5), seed 42 dự đoán đúng 695/700 ảnh; cả năm lỗi đều có nhãn thật là Anthracnose, trong đó một ảnh bị dự đoán thành Gall Midge, hai ảnh thành Sooty Mold và hai ảnh thành Red Rust. Hệ quả là precision của Gall Midge, Sooty Mold và Red Rust giảm nhẹ xuống 0,98-0,99 do nhận thêm dự đoán sai từ lớp Anthracnose, trong khi recall của các lớp này vẫn giữ 1,00; recall của Anthracnose là 0,95. Nhận định định tính về các trường hợp sai được phân tích ở mục 4.5.2.



Hình 4.3. Ma trận nhầm lẫn tập kiểm tra MangoLeafBD của checkpoint seed 3333



Hình 4.4. Ma trận nhầm lẫn tập kiểm tra Bộ dữ liệu kết hợp của checkpoint seed



Hình 4.5. Ma trận nhầm lẫn tập kiểm tra VN-MangoLeaf của checkpoint seed 42

4.3.5. Đánh giá chiến lược huấn luyện theo chương trình

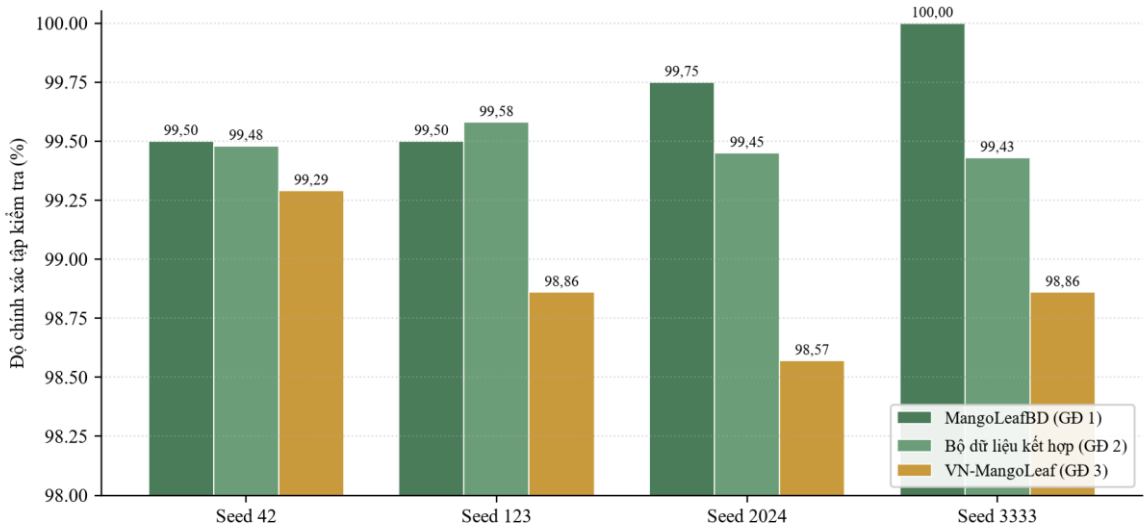
Đối chứng curriculum nạp checkpoint giai đoạn 1 từ MangoLeafBD, thay đầu phân loại 8 lớp bằng đầu 7 lớp rồi tinh chỉnh trực tiếp trên VN-MangoLeaf. Vì đối chứng chỉ bỏ giai đoạn 2 trên Bộ dữ liệu kết hợp, đây không phải phép huấn luyện từ đầu. Theo Bảng 4.8, curriculum ba giai đoạn đạt Accuracy 98,89 ± 0,29%, cao hơn 0,79 điểm phần trăm so với tinh chỉnh trực tiếp (98,11 ± 0,55%); F1 weighted lần lượt là 98,89 ± 0,29% và 98,09 ± 0,57%. Curriculum cao hơn ở cả bốn seed tương ứng, với chênh lệch từ 0,43 đến 1,29 điểm phần trăm. Vì hai phương án của cùng một seed xuất phát từ cùng một checkpoint giai đoạn 1, kết quả được so sánh theo cặp seed. Chênh lệch theo cặp là 0,57; 1,29; 0,86 và 0,43 điểm phần trăm, tương ứng 4, 9, 6 và 3 ảnh trên 700 ảnh kiểm tra; trung bình 0,79 điểm phần trăm, độ lệch chuẩn 0,38 điểm phần trăm. Kiểm định t ghép cặp cho $t(3) = 4,16$ và $p \approx 0,025$ (hai phía); khoảng tin cậy 95% của chênh lệch trung bình là [0,18; 1,39] điểm phần trăm. Kiểm định dấu, vốn không giả định phân phối chuẩn, cho $p = 0,0625$ (một phía) vì cả bốn chênh lệch đều dương; với bốn cặp, đây là mức p nhỏ nhất mà kiểm định dấu có thể

đạt. Với n = 4, giả định phân phối chuẩn của kiểm định t không kiểm tra được và các seed dùng chung một tập kiểm tra, nên các giá trị thống kê này chỉ mang tính tham khảo. Bộ dữ liệu kết hợp chỉ gồm ba nguồn Bangladesh và không có ảnh VN-MangoLeaf; giai đoạn 2 nằm giữa dữ liệu nguồn và dữ liệu mục tiêu trong lộ trình huấn luyện. Luận văn chưa đo khoảng cách miền hoặc độ chính xác trung gian giữa các giai đoạn.

Bảng 4.8. So sánh học theo chương trình ba giai đoạn và tinh chỉnh trực tiếp trên VN-MangoLeaf từ checkpoint giai đoạn 1 (độ chính xác tập kiểm tra, %)

Phương án	Seed 42	Seed 123	Seed 2024	Seed 3333	Trung bình ± độ lệch chuẩn
Ba giai đoạn (curriculum)	99,29	98,86	98,57	98,86	98,89 ± 0,29
Trực tiếp trên VN-MangoLeaf	98,71	97,57	97,71	98,43	98,11 ± 0,55

Hình 4.6 tổng hợp độ chính xác tập kiểm tra theo từng seed trên ba bộ dữ liệu của lộ trình ba giai đoạn. Trục tung bắt đầu từ 98% để phân biệt được khoảng cách giữa các cột. Biến động giữa các seed nhỏ ở giai đoạn 1 và 2, và lớn hơn ở giai đoạn 3 trên dữ liệu mục tiêu, phù hợp với độ lệch chuẩn đã nêu ở Bảng 4.3, Bảng 4.5 và Bảng 4.6.



Hình 4.6. Độ chính xác tập kiểm tra theo seed trên ba bộ dữ liệu của lộ trình ba giai đoạn (trục tung bắt đầu từ 98%)

Đối chiếu với tài liệu ở mục 2.1.5, chiều của kết quả phù hợp với các nghiên cứu ghi nhận lợi ích của chuyển tiếp nhiều giai đoạn hoặc có miền trung gian [14, 35, 47, 48]: ở cả bốn seed, lộ trình có giai đoạn 2 đạt độ chính xác kiểm tra cao hơn tinh chỉnh trực tiếp. Mức chênh lệch trung bình 0,79 điểm phần trăm tương ứng khoảng 5,5 ảnh được phân loại đúng thêm trên tập kiểm tra 700 ảnh ở mỗi seed. Vì đối chứng cũng được khởi tạo từ checkpoint giai đoạn 1, kết quả này phù hợp với giả thuyết rằng giai đoạn 2 mang lại lợi ích so với chuyển tiếp một giai đoạn. Tuy nhiên, với bốn seed dùng chung một tập kiểm tra, chênh lệch này chỉ là bằng chứng sơ bộ; theo phân tích ở mục 3.2.6, nó phản ánh tác động tổng hợp của dữ liệu nguồn lớn hơn, đa dạng hơn và ngân sách huấn luyện bổ sung, không tách riêng được tác động của thứ tự huấn luyện.

4.4. So sánh tham chiếu với các mô hình đã công bố

4.4.1. So sánh về độ chính xác

Bảng 4.9 so sánh MangoMamba với các mô hình đã được báo cáo trên MangoLeafBD. Số liệu của DeiT, Swin Transformer, MobileNetV2 và các mô hình còn lại được trích từ Hossain và cộng sự [17] (thời gian suy luận đo trên CPU Intel Xeon 2,20 GHz, chia dữ liệu 8:1:1); số liệu của ViX-MangoEFormer trích từ Noman và cộng sự [28] (thời gian suy luận dưới 120 ms trên GPU tầm trung, đánh giá bằng kiểm định chéo phân tầng 10 lần); số liệu của MangoMamba là kết quả thực nghiệm của luận văn này (trung bình bốn seed, thời gian đo trên GPU NVIDIA GB10, FP32, batch 1, chi tiết ở mục 4.4.2). Bộ số liệu này là kết quả chạy lại với cấu hình cuối cùng và thay thế các con số đã công bố trong [27] (99,75%; 98,71%; 1,46 ms trên T4); khi hai nguồn khác nhau, luận văn dùng bộ số đã kiểm chứng mới nhất nêu tại đây, còn phân tích theo từng lớp, đối chứng tinh chỉnh trực tiếp và các phần trình bày sau đây được mở rộng mới trong luận văn. Luận văn không tái huấn luyện lại các baseline do giới hạn tài nguyên tính toán; số liệu đối chứng được trích nguyên trạng từ nguồn. Do điều kiện phần cứng và cách chia dữ liệu không đồng nhất giữa các nghiên cứu, cột thời gian suy luận chỉ mang tính tham chiếu.

Bảng 4.9. So sánh với các mô hình đã công bố trên MangoLeafBD

Mô hình	Accuracy (%)	Thời gian suy luận	Tham số	Kích thước	Chia dữ liệu
DeiT [17]	99,75	346,87 ms (CPU)	85,8 M	1,03 GB	8:1:1
Optimized DeiT [17]	99,75	315,78 ms (CPU)	781 K	88,47 MB	8:1:1
Swin Transformer [17]	99,50	206,02 ms (CPU)	27,58 M	110,30 MB	8:1:1
ShuffleNet [17]	99,50	40,76 ms (CPU)	5,36 M	64,72 MB	8:1:1
MobileNetV2 [17]	98,79	42,28 ms (CPU)	2,23 M	27,19 MB	8:1:1
EfficientNet [17]	98,75	45,01 ms (CPU)	4,01 M	48,70 MB	8:1:1
DenseNet121 [17]	96,75	119,34 ms (CPU)	6,96 M	84,47 MB	8:1:1
SqueezeNet [17]	96,50	20,80 ms (CPU)	726,6 K	8,78 MB	8:1:1
ViX-MangoEFormer [28]	99,43	<120 ms (GPU tầm trung)	không công bố	không công bố	10-fold CV
MangoMamba (luận văn này)	**99,69 ± 0,24**	**4,57 ms (GPU)**	**5,82 M**	**25,29 MB (ONNX)**	**8:1:1**

Về độ chính xác, MangoMamba đạt 99,69%, thấp hơn hai biến thể DeiT 0,06 điểm phần trăm. Trong các mô hình có dung lượng file dưới 30 MB (SqueezeNet, MobileNetV2 và MangoMamba), MangoMamba có độ chính xác cao nhất. Optimized DeiT có ít tham số hơn (781 K) và độ chính xác cao hơn 0,06 điểm phần trăm, nhưng dung lượng là 88,47 MB, gấp khoảng 3,5 lần MangoMamba.

4.4.2. So sánh về hiệu quả tính toán (Tốc độ suy luận, kích thước mô hình)

Thời gian suy luận trên máy trạm được đo với giao thức thống nhất: batch 1, FP32, đầu vào 320 × 320, 100 lần warm-up sau đó 500 lần đo đồng bộ bằng CUDA event. Kết quả trên GPU NVIDIA GB10 (checkpoint tốt nhất trên MangoLeafBD, seed 3333) là 4,566 ± 0,130 ms/ảnh (trung vị 4,545 ms; p95 4,594 ms), tương đương

khoảng 219 ảnh/giây. Checkpoint này có đầu phân loại 8 lớp; mô hình 7 lớp dùng trong ứng dụng chỉ ít hơn 385 tham số ở đầu phân loại nên có thể xem là tương đương về chi phí tính toán. Trên thiết bị di động Xiaomi Redmi Note 11S (MediaTek Helio G96, Android 13), ứng dụng Android mô tả ở mục 4.1 đạt trung bình 612,80 ms/ảnh (độ lệch chuẩn 48,47 ms; trung vị 598,48 ms; p95 718,51 ms; trung bình trên 50 lần suy luận liên tiếp cùng một ảnh kiểm tra), tương đương khoảng 1,63 ảnh/giây, hoạt động hoàn toàn ngoại tuyến không cần kết nối máy chủ.

MangoMamba có 5,82 triệu tham số (dung lượng tham số FP32 khoảng 22,22 MiB); file mô hình ONNX dùng khi triển khai có dung lượng 25,29 MB. Thời gian suy luận trên Xiaomi Redmi Note 11S dưới một giây mỗi ảnh, phù hợp với phân loại ngoại tuyến; luận văn chưa đánh giá chính thức trải nghiệm người dùng.

4.5. Diễn giải mô hình và phân tích hạn chế

4.5.1. Trực quan hóa kết quả bằng Grad-CAM

Phần này dùng Grad-CAM để trực quan hóa các vùng của ảnh có đóng góp lớn vào dự đoán. Hình 4.7 gồm bảy hàng, tương ứng bảy lớp của VN-MangoLeaf; mỗi hàng lần lượt thể hiện ảnh gốc, bản đồ nhiệt Grad-CAM và ảnh phủ bản đồ nhiệt. Các hình dùng checkpoint giai đoạn 3, seed 42; ảnh của mỗi lớp là ảnh đầu tiên của lớp đó trong bộ nạp dữ liệu kiểm định (validation), không được chọn thủ công, và bản đồ được tính cho lớp thật của ảnh.

Grad-CAM được tính tại lớp BatchNorm2d cuối, ngay trước gộp trung bình toàn cục và lớp tuyến tính. Gọi $F_k(i, j)$ là giá trị kênh k tại vị trí (i, j) của bản đồ đặc trưng $H \times W$ sau lớp này; logit của lớp c là:

$$y_c = \Sigma_k W_{ck} \cdot (1/(HW)) \Sigma_{i,j} F_k(i, j) + b_c \quad (4.2)$$

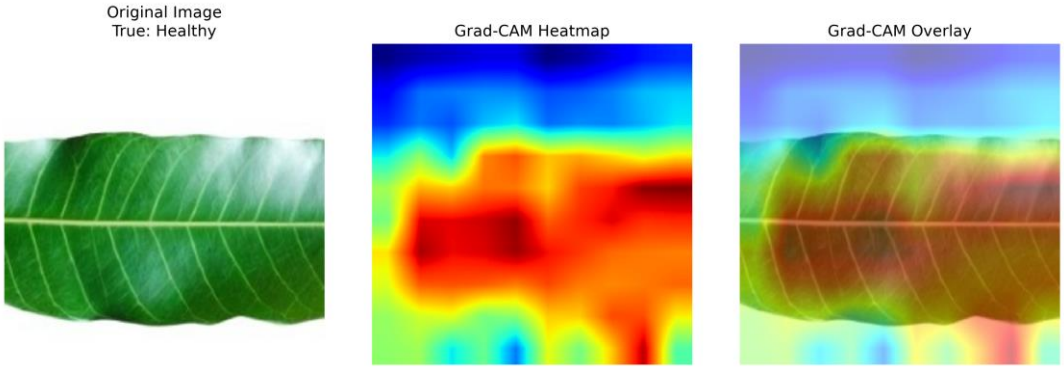
Do đó $\partial y_c/\partial F_k(i, j) = W_{ck}/(HW)$ không phụ thuộc vị trí, nên trọng số Grad-CAM là $\alpha_k^c = W_{ck}/(HW)$ và bản đồ Grad-CAM của lớp c là:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}(\Sigma_k \alpha_k^c F_k) = (1/(HW)) \cdot \text{CAM}(c) \quad (4.3)$$

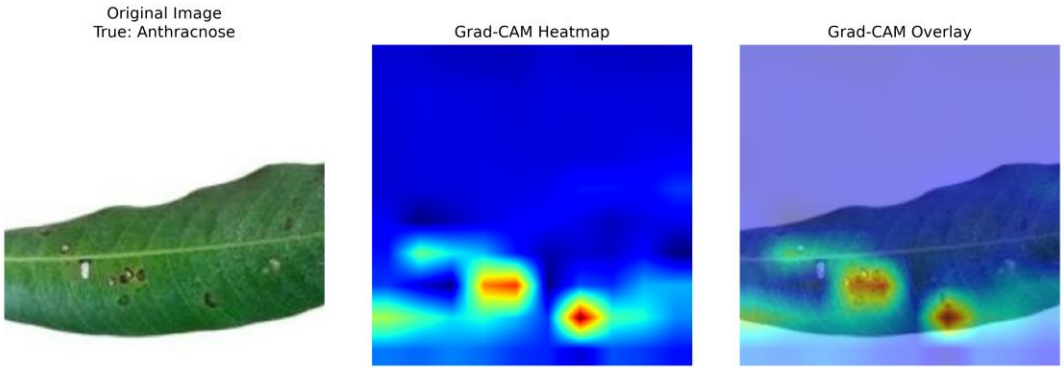
tức là bằng bản đồ CAM ở công thức (4.1) nhân với hằng số dương 1/(HW). Hai bản đồ vì vậy chỉ khác nhau ở bước chuẩn hóa để hiển thị: thư viện pytorch-grad-cam

chuẩn hóa theo min-max, còn ứng dụng Android và các hình ở mục 4.5.2 chia cho giá trị lớn nhất.

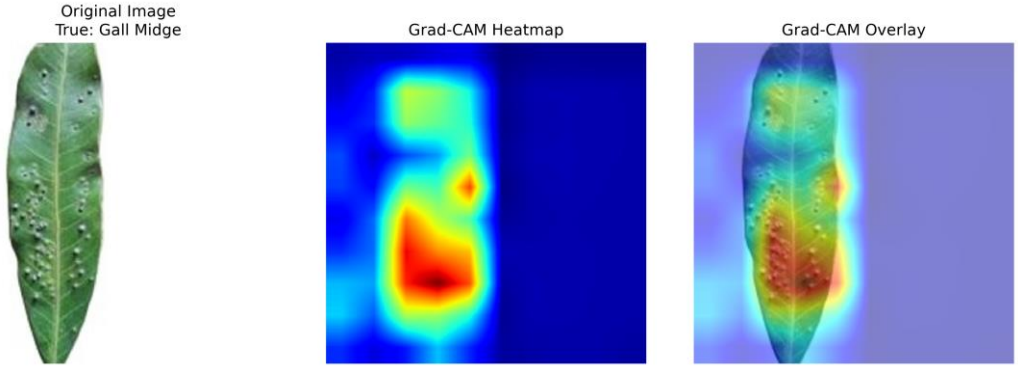
Ở các ví dụ minh hoạ, vùng kích hoạt cao thường trùng với tổn thương đặc trưng của từng lớp: mảng bột trắng trên mô non, lớp phủ đen dạng muội, nốt đỏ nâu hơi nổi, nốt u phồng trên lá non, đốm hoại tử nâu đen và vùng chồi hoặc lá cháy héo. Với mẫu Healthy, Grad-CAM vẫn có vùng kích hoạt mạnh và phân bố rộng trên phiến lá. Các quan sát này không đủ để kết luận mô hình định vị đúng tổn thương hoặc không sử dụng bối cảnh nền.



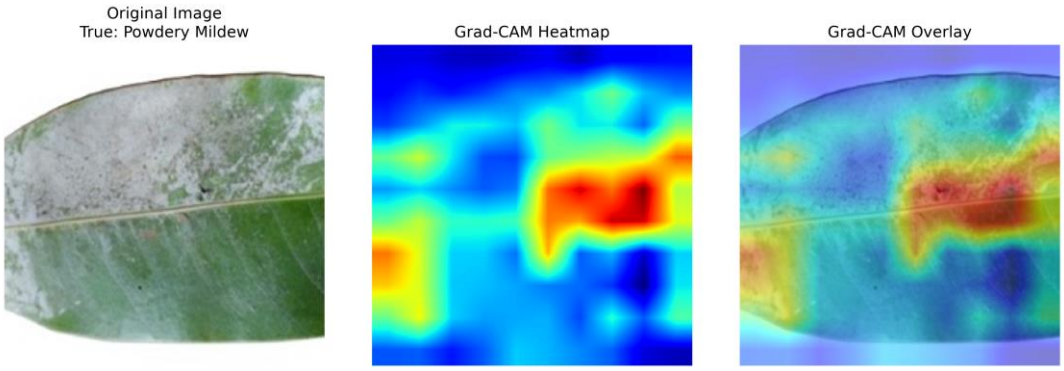
(a) Healthy



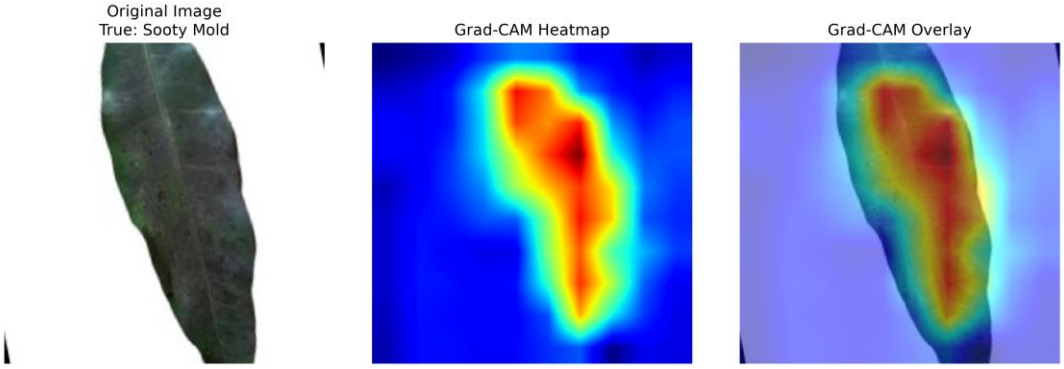
(b) Anthracnose



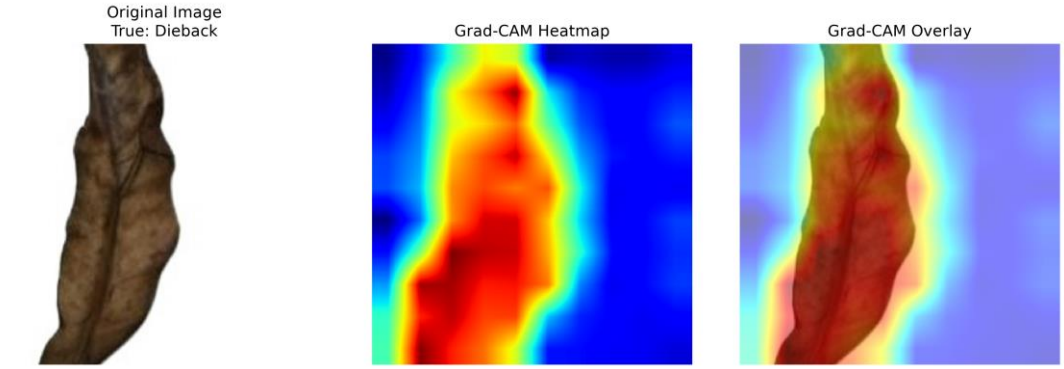
(c) Gall Midge



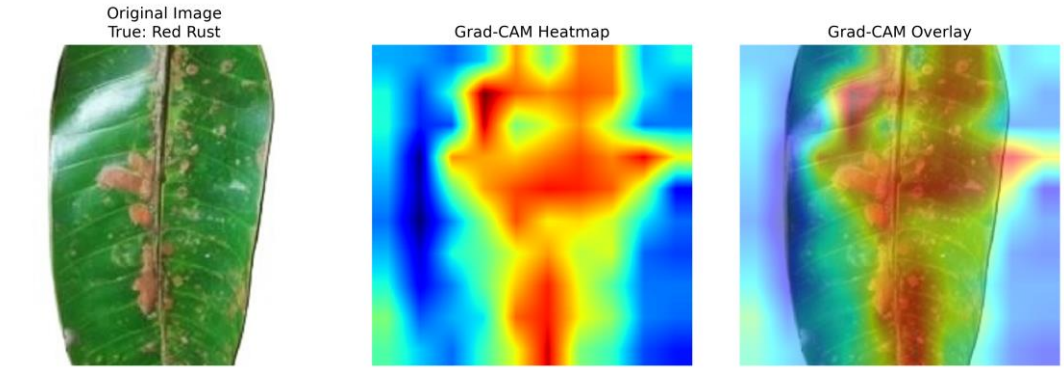
(d) Powdery Mildew



(e) Sooty Mold



(f) Dieback



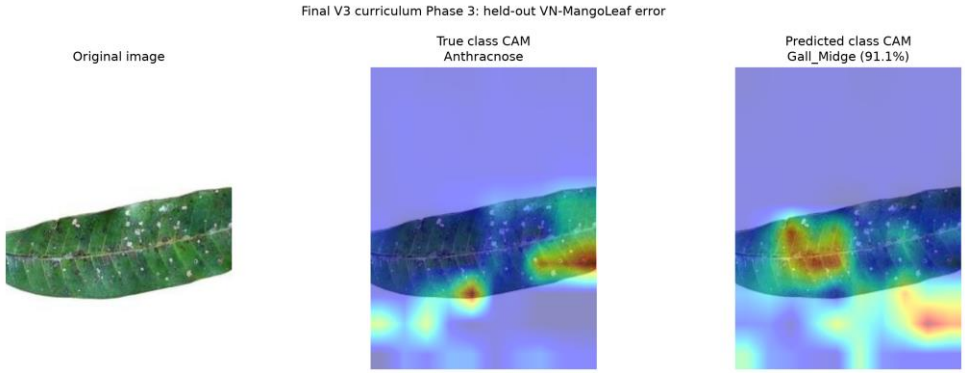
(g) Red Rust

Hình 4.7. Bản đồ Grad-CAM cho bảy lớp của VN-MangoLeaf (checkpoint giai đoạn 3, seed 42). Mỗi hàng gồm ảnh gốc, bản đồ nhiệt Grad-CAM và ảnh phủ.

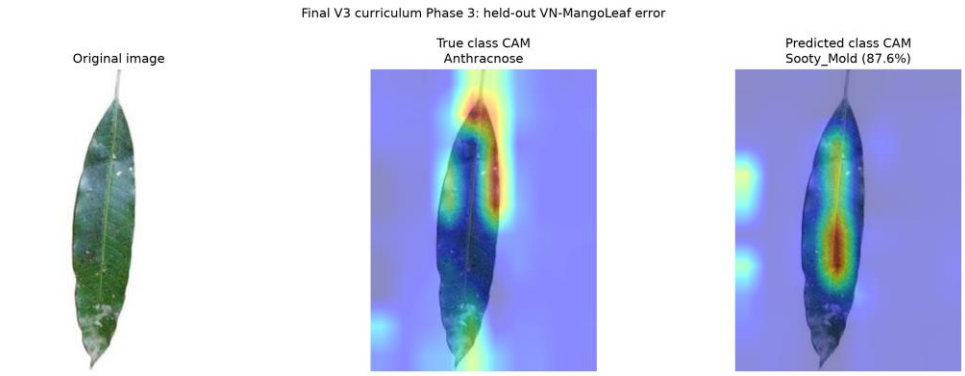
4.5.2. Phân tích các trường hợp dự đoán sai

Checkpoint seed 42 mắc 5 lỗi trên 700 ảnh kiểm tra (0,71%). Tất cả các ảnh bị sai đều có nhãn thật là Anthracnose, được dự đoán thành Gall Midge (một ảnh, độ tin cậy 0,911), Sooty Mold (hai ảnh, 0,876 và 0,657) và Red Rust (hai ảnh, 0,872 và 0,455).

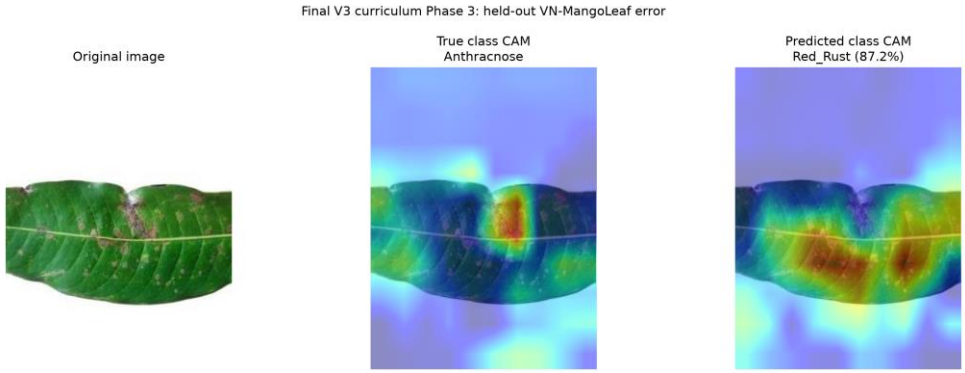
Trong checkpoint seed 42, cả năm dự đoán sai đều có nhãn thật là Anthracnose. Một số mẫu có màu nâu đỏ gần với Red Rust; một số có mảng tối gần với Sooty Mold; và một số có tổn thương phồng gần với Gall Midge. Trong các ví dụ minh hoạ, CAM có vùng kích hoạt trùng với tổn thương, nhưng quan sát này không chứng minh quan hệ nhân quả. Hai trong năm dự đoán có độ tin cậy dưới 0,7 (0,657 và 0,455). Một ngưỡng bất định có thể được đánh giá sau khi kiểm tra toàn bộ phân bố confidence của tập kiểm tra. Phân tích này chỉ áp dụng cho checkpoint seed 42; chưa có tổng hợp lỗi trên bốn seed.



Trường hợp 1: nhãn thật Anthracnose, dự đoán Gall Midge (0,911)



Trường hợp 2: nhãn thật Anthracnose, dự đoán Sooty Mold (0,876)



Trường hợp 3: nhãn thật Anthracnose, dự đoán Red Rust (0,872)

Hình 4.8. Ba trường hợp dự đoán sai đại diện trên VN-MangoLeaf (seed 42); mỗi khối gồm ảnh gốc, bản đồ CAM của lớp thật và bản đồ CAM của lớp dự đoán kèm độ tin cậy

Cuối cùng, hai điểm cần lưu ý thêm khi đọc kết quả: dữ liệu mục tiêu thu thập chủ yếu tại khu vực Nam Bộ nên khả năng tổng quát cho miền khí hậu khác chưa được kiểm chứng; và mô hình phân loại đóng với bảy lớp nên bệnh ngoài tập nhãn sẽ được gán vào một trong bảy lớp thay vì được báo là không xác định. Các hạn chế này là cơ sở cho các hướng phát triển trình bày ở Chương 5.

CHƯƠNG 5. KẾT LUẬN VÀ KHUYẾN NGHỊ

5.1. Kết luận

Luận văn hoàn thành các mục tiêu nghiên cứu nêu ở Chương 1, dựa trên kết quả thực nghiệm ở Chương 4. Về dữ liệu, tôi xây dựng và công bố VN-MangoLeaf gồm 7.000 ảnh cân bằng trong 7 lớp, gồm 4.900 ảnh gốc đã xác minh và 2.100 ảnh tăng cường; bộ dữ liệu bổ sung lớp rỉ sắt Red Rust, chưa có trong các bộ dữ liệu Bangladesh nguồn.

Về mô hình, tôi đề xuất MangoMamba, một kiến trúc lai kết hợp selective scan của Mamba với Large-Kernel Attention trong cấu trúc phân cấp theo cửa sổ. Trên tập kiểm tra, mô hình đạt 99,69 ± 0,24% trên MangoLeafBD, 99,48 ± 0,07% trên Bộ dữ liệu kết hợp và 98,89 ± 0,29% trên VN-MangoLeaf, với 5,82 triệu tham số (dung lượng tham số FP32 khoảng 22,22 MiB; file ONNX dùng khi triển khai 25,29 MB).

Trên MangoLeafBD, các khảo sát thành phần ở cấu hình 224 × 224 cho thấy việc loại ECA, GRN hoặc SwiGLU chỉ làm Accuracy trung bình trên tập kiểm tra giảm từ 0,13 đến 0,56 điểm phần trăm, xấp xỉ độ lệch chuẩn giữa các seed (0,20 đến 0,52 điểm phần trăm); khảo sát vì vậy chưa xác lập đóng góp riêng của từng thành phần. Trên dữ liệu mục tiêu, huấn luyện ba giai đoạn đạt Accuracy cao hơn 0,79 điểm phần trăm so với tinh chỉnh trực tiếp từ checkpoint giai đoạn 1 (kiểm định t ghép cặp, $p \approx 0{,}025$, $n = 4$; giá trị này chỉ mang tính tham khảo vì số seed nhỏ và các seed dùng chung một tập kiểm tra). Thời gian suy luận là 4,57 ms/ảnh trên GPU NVIDIA GB10 và 612,80 ms/ảnh trên Xiaomi Redmi Note 11S. Với checkpoint seed 42, vốn được chọn hậu nghiệm theo độ chính xác kiểm tra cao nhất, lớp rỉ sắt đạt recall 1,00; kết quả theo lớp của các seed còn lại chưa được tổng hợp.

5.2. Hạn chế của luận văn

Các hạn chế chính của nghiên cứu gồm:

- Các mô hình đối chứng trong Bảng 4.9 được trích từ các nghiên cứu dùng phần cứng và cách chia dữ liệu khác nhau. Luận văn chưa tái huấn luyện các mô hình này trong cùng điều kiện dữ liệu và phần cứng.

- Ma trận nhầm lẫn, CAM và phân tích lỗi dùng checkpoint có độ chính xác trên tập kiểm tra cao nhất trong bốn seed. Các checkpoint khác có thể có cấu trúc lỗi khác.
- Thời gian suy luận trên điện thoại dùng FP32, chưa lượng tử hóa. Luận văn chưa đo thời gian toàn trình từ lúc chụp ảnh đến khi nhận kết quả, chưa thử NNAPI hoặc GPU delegate và chưa đánh giá trải nghiệm người dùng.
- Nhãn dữ liệu chưa được đánh giá chính thức bởi chuyên gia bệnh học thực vật. Một số bệnh cũng còn ít mẫu.
- Các thí nghiệm thành phần ECA, GRN và SwiGLU hiện chỉ được thực hiện ở cấu hình 224 × 224 trên MangoLeafBD; chưa đánh giá các thành phần này trên VN-MangoLeaf.

5.3. Hướng phát triển trong tương lai

Nghiên cứu tiếp theo nên mở rộng VN-MangoLeaf bằng ảnh thu thập tại nhiều địa điểm, mùa vụ, giống xoài và giai đoạn bệnh. Cần ghi nhận thiết bị cùng điều kiện chụp. Khi chia tập, nên nhóm ảnh theo cây, vườn hoặc đợt thu thập để ảnh gần nhau không đồng thời xuất hiện trong huấn luyện và kiểm tra. Một tập kiểm tra độc lập từ địa điểm khác sẽ cung cấp thêm bằng chứng về khả năng áp dụng ngoài phạm vi dữ liệu hiện có.

Cần đánh giá chất lượng nhãn và các trường hợp dự đoán sai với chuyên gia bệnh học thực vật. Nên báo cáo kết quả theo từng lớp, ma trận nhầm lẫn và phân tích ảnh khó hoặc có triệu chứng tương tự. Có thể kiểm tra độ tin cậy của xác suất dự đoán, rồi xác định ngưỡng để ứng dụng đề nghị chụp lại hoặc chuyển ca chưa rõ cho chuyên gia. Cách này làm rõ mức độ tin cậy của từng dự đoán bên cạnh độ chính xác trung bình.

Về phương pháp, cần so sánh tinh chỉnh trực tiếp với chuyển tiếp nhiều giai đoạn trên cùng phép chia dữ liệu và nhiều seed; đồng thời khảo sát độ phân giải đầu vào, từng thành phần kiến trúc và chi phí tính toán trên dữ liệu mục tiêu. Với ứng dụng di động, có thể đánh giá lượng tử hóa số nguyên 8 bit (INT8), NNAPI và GPU delegate trên nhiều thiết bị. Cần đo độ chính xác theo lớp sau tối ưu, thời gian từ lúc chụp hoặc chọn ảnh đến khi hiển thị kết quả, bộ nhớ và năng lượng tiêu thụ. Các phép

đo này giúp xác định tốc độ tăng lên có đi kèm giảm chất lượng hay không trước khi xem xét sử dụng thực tế.

DANH MỤC CÔNG BỐ KHOA HỌC CỦA TÁC GIẢ

Trong quá trình thực hiện luận văn này, tác giả đã có công bố khoa học liên quan trực tiếp đến đề tài nghiên cứu như sau:

1. Nguyen-Tat, T.B., Pham-Thanh, B. (2026), “Mango-Mamba and VN-MangoLeaf: A lightweight Mamba model and New Dataset for Mango leaf disease classification”, *Computers and Electrical Engineering* Vol. 133, Article 111033. DOI: 10.1016/j.compeleceng.2026.111033.

TÀI LIỆU THAM KHẢO

Tiếng Việt

1. Vũ Hường (08-2018), "Phòng trị bệnh đốm rong trên cây ăn trái trong mùa mưa", Trung tâm Khuyến nông Quốc gia, https://khuyennongvn.gov.vn/khoa-hoc-cong-nghe/khcn-trong-nuoc/phong-tri-benh-dom-rong-tren-cay-an-trai-trong-mua-mua-17309.html

Tiếng Anh

2. Ahmad, M., Abdullah, M., Moon, H., Han, D. (2021), "Plant Disease Detection in Imbalanced Datasets Using Efficient Convolutional Neural Networks With Stepwise Transfer Learning", *IEEE Access* Vol. 9, pp. 140565-140580. DOI: 10.1109/ACCESS.2021.3119655.
3. Ahmed, S.I., Ibrahim, M., Nadim, M., Rahman, M.M., Shejunti, M.M., Jabid, T., Ali, M.S. (2023), "MangoLeafBD: A comprehensive image dataset to classify diseased and healthy mango leaves", *Data in Brief* Vol. 47, Article 108941. DOI: 10.1016/j.dib.2023.108941.
4. Alamri, F.S., Sadad, T., Almasoud, A.S., Aurangzeb, R.A., Khan, A. (2025), "Mango Disease Detection Using Fused Vision Transformer with ConvNeXt Architecture", *Computers, Materials & Continua* Vol. 83 (1), pp. 1023-1039. DOI: 10.32604/cmc.2025.061890.
5. Ben-David, S., Blitzer, J., Crammer, K., Kulesza, A., Pereira, F., Vaughan, J.W. (2010), "A theory of learning from different domains", *Machine Learning* Vol. 79 (1-2), pp. 151-175. DOI: 10.1007/s10994-009-5152-4.
6. Bengio, Y., Louradour, J., Collobert, R., Weston, J. (2009), "Curriculum learning", *Proceedings of the 26th Annual International Conference on Machine Learning (ICML 2009)*, pp. 41-48. DOI: 10.1145/1553374.1553380.
7. Business Queensland (Queensland Government) (08-2025), "Mango leaf gall midge", https://www.business.qld.gov.au/industries/farms-fishing-forestry/agriculture/biosecurity/plants/priority-pest-disease/mango-leaf-gall-midge
8. Chowdhery, A., Narang, S., Devlin, J., et al. (2023), "PaLM: Scaling Language

Modeling with Pathways", *Journal of Machine Learning Research* Vol. 24 (240), pp. 1-113.

9. Cui, Y., Song, Y., Sun, C., Howard, A., Belongie, S. (2018), "Large Scale Fine-Grained Categorization and Domain-Specific Transfer Learning", *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4109-4118. DOI: 10.1109/CVPR.2018.00432.
10. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N. (2021), "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", *International Conference on Learning Representations (ICLR)*.
11. Gu, A., Goel, K., Ré, C. (2022), "Efficiently Modeling Long Sequences with Structured State Spaces", *International Conference on Learning Representations (ICLR)*.
12. Gu, A., Dao, T. (2024), "Mamba: Linear-Time Sequence Modeling with Selective State Spaces", *Proceedings of the First Conference on Language Modeling (COLM 2024)*. arXiv:2312.00752.
13. Guo, M.H., Lu, C.Z., Liu, Z.N., Cheng, M.M., Hu, S.M. (2023), "Visual attention network", *Computational Visual Media* Vol. 9 (4), pp. 733-752. DOI: 10.1007/s41095-023-0364-2.
14. Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., Smith, N.A. (2020), "Don't Stop Pretraining: Adapt Language Models to Domains and Tasks", *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 8342-8360. DOI: 10.18653/v1/2020.acl-main.740.
15. Hatamizadeh, A., Kautz, J. (2025), "MambaVision: A Hybrid Mamba-Transformer Vision Backbone", *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 25261-25270. DOI: 10.1109/CVPR52734.2025.02352.
16. Henn, A. (2025), *The Plant Doctor: Sooty Mold*, Publication 4035 (POD-08-25), Mississippi State University Extension Service, Mississippi, USA,

https://extension.msstate.edu/publications/the-plant-doctor-sooty-mold

17. Hossain, M.A., Sakib, S., Abdullah, H.M., Arman, S.E. (2024), "Deep learning for mango leaf disease identification: A vision transformer perspective", *Heliyon* Vol. 10 (17), Article e36361. DOI: 10.1016/j.heliyon.2024.e36361.
18. Kumar, A., Ma, T., Liang, P. (2020), "Understanding Self-Training for Gradual Domain Adaptation", *Proceedings of the 37th International Conference on Machine Learning, PMLR* Vol. 119, pp. 5468-5479.
19. Kumar, A., Raghunathan, A., Jones, R., Ma, T., Liang, P. (2022), "Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution", *International Conference on Learning Representations (ICLR)*.
20. Li, Z., Liu, F., Yang, W., Peng, S., Zhou, J. (2022), "A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects", *IEEE Transactions on Neural Networks and Learning Systems* Vol. 33 (12), pp. 6999-7019. DOI: 10.1109/TNNLS.2021.3084827.
21. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B. (2021), "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows", *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9992-10002. DOI: 10.1109/ICCV48922.2021.00986.
22. Mahmud, B.U., Al Mamun, A., Hossen, M.J., Hong, G.Y., Jahan, B. (2024), "Light-Weight Deep Learning Model for Accelerating the Classification of Mango-Leaf Disease", *Emerging Science Journal* Vol. 8 (1), pp. 28-42. DOI: 10.28991/ESJ-2024-08-01-03.
23. Mohanty, S.P., Hughes, D.P., Salathé, M. (2016), "Using deep learning for image-based plant disease detection", *Frontiers in Plant Science* Vol. 7, Article 1419. DOI: 10.3389/fpls.2016.01419.
24. National Horticulture Board (Ấn Độ) (không ghi ngày, truy cập 09-2026), "Mango Diseases", https://nhb.gov.in/pdf/fruits/mango/man002.pdf
25. Nelson, S.C. (2008), *Mango Anthracnose (Colletotrichum gloeosporioides)*, Plant Disease PD-48, College of Tropical Agriculture and Human Resources, University of Hawai'i at Mānoa, Honolulu, USA, https://www.ctahr.hawaii.edu/oc/freepubs/pdf/PD-48.pdf

26. Neyshabur, B., Sedghi, H., Zhang, C. (2020), "What is being transferred in transfer learning?", *Advances in Neural Information Processing Systems* Vol. 33, pp. 512-523.

27. Nguyen-Tat, T.B., Pham-Thanh, B. (2026), "Mango-Mamba and VN-MangoLeaf: A lightweight Mamba model and New Dataset for Mango leaf disease classification", *Computers and Electrical Engineering* Vol. 133, Article 111033. DOI: 10.1016/j.compeleceng.2026.111033.

28. Noman, A.A., Hossain, A., Sakib, A., Debnath, J., Fardin, H., Al Sakib, A., Haque, R., Ahmed, M.R., Reza, A.W., Dewan, M.A.A. (2025), "ViX-MangoEFormer: An Enhanced Vision Transformer–EfficientFormer and Stacking Ensemble Approach for Mango Leaf Disease Recognition with Explainable Artificial Intelligence", *Computers* Vol. 14 (5), Article 171. DOI: 10.3390/computers14050171.

29. Pan, S.J., Yang, Q. (2010), "A Survey on Transfer Learning", *IEEE Transactions on Knowledge and Data Engineering* Vol. 22 (10), pp. 1345-1359. DOI: 10.1109/TKDE.2009.191.

30. Patrice, N.D.J., Alain, H., Bertrand, M.S., Norbert, K.T.W., Nourou, K.N.A., Brice, T.T.D., Honorine, A.G., Dolaris, J.A., Bouba, D., Zachee, A. (2020), "First Report of Red Rust Disease caused by *Cephaleuros virescens* on Mango (*Mangifera indica*) Tree in Cameroon", *International Journal of Phytopathology* Vol. 9 (3), pp. 187-193. DOI: 10.33687/phytopath.009.03.3432.

31. Qin, J., Deng, L., Li, C., He, J., Pen, H., Wang, Z. (2025), "CropCapsNet: Enhanced capsule network for crop disease classification", *Computers and Electrical Engineering* Vol. 128 (Part A), Article 110635. DOI: 10.1016/j.compeleceng.2025.110635.

32. Rahman, M.S., Hasan, R., Mojumdar, M.U. (2024), *Mango Leaf Disease Dataset*, Mendeley Data, Version 1. DOI: 10.17632/7ghdbftp54.1.

33. Rao, Y., Zhao, W., Liu, B., Lu, J., Zhou, J., Hsieh, C.-J. (2021), "DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification", *Advances in Neural Information Processing Systems* Vol. 34, pp. 13937-13949.

34. Rizvee, R.A., Orpa, T.H., Ahnaf, A., Kabir, M.A., Rashid, M.R.A., Islam, M.M., Islam, M., Jabid, T., Ali, M.S. (2023), "LeafNet: A proficient convolutional neural network for detecting seven prominent mango leaf diseases", *Journal of Agriculture and Food Research* Vol. 14, Article 100787. DOI: 10.1016/j.jafr.2023.100787.

35. Samala, R.K., Chan, H.-P., Hadjiiski, L., Helvie, M.A., Richter, C.D., Cha, K.H. (2019), "Breast Cancer Diagnosis in Digital Breast Tomosynthesis: Effects of Training Sample Size on Multi-Stage Transfer Learning Using Deep Neural Nets", *IEEE Transactions on Medical Imaging* Vol. 38 (3), pp. 686-696. DOI: 10.1109/TMI.2018.2870343.

36. Shakib, M.M.H., Mustofa, S., Ahad, M.T. (2024), *MLD24: An image dataset for mango leaf disease detection*, Mendeley Data, Version 1. DOI: 10.17632/6dvpywm2m2.1.

37. Shazeer, N. (2020), "GLU Variants Improve Transformer", *arXiv preprint* arXiv:2002.05202.

38. Simone, G.W. (1999), *Disease Control in Mango (Mangifera indica)*, Plant Disease Management Guide Vol. 3 (PDMG-V3-22), Florida Cooperative Extension Service, Institute of Food and Agricultural Sciences, University of Florida, Gainesville, USA, https://ufdcimages.uflib.ufl.edu/IR/00/00/52/68/00001/PG01600.pdf

39. Tamil Nadu Agricultural University (không ghi ngày, truy cập 09-2026), "TNAU Agritech Portal :: Crop Protection", mục bệnh rỉ sắt trên xoài, https://agritech.tnau.ac.in/crop_protection/mango_5.html

40. Touvron, H., Lavril, T., Izacard, G., et al. (2023), "LLaMA: Open and Efficient Foundation Language Models", *arXiv preprint* arXiv:2302.13971.

41. Truong, G.-B., Tran, T.-T., Than, N.-L., Nguyen, V.Q., Nguyen, T.H., Pham, V.-T. (2025), "SC-MambaFew: Few-shot learning based on Mamba and selective spatial-channel attention for bearing fault diagnosis", *Computers and Electrical Engineering* Vol. 123, Article 110004. DOI: 10.1016/j.compeleceng.2024.110004.

42. Varma, T., Mate, P., Azeem, N.A., Sharma, S., Singh, B. (2025), "Automatic

mango leaf disease detection using different transfer learning models", *Multimedia Tools and Applications* Vol. 84 (11), pp. 9185-9218. DOI: 10.1007/s11042-024-19265-x.

43. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I. (2017), "Attention is All you Need", *Advances in Neural Information Processing Systems* Vol. 30, pp. 5998-6008.
44. Wang, Q., Wu, B., Zhu, P., Li, P., Zuo, W., Hu, Q. (2020), "ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks", *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11531-11539. DOI: 10.1109/CVPR42600.2020.01155.
45. Wang, X., Chen, Y., Zhu, W. (2022), "A Survey on Curriculum Learning", *IEEE Transactions on Pattern Analysis and Machine Intelligence* Vol. 44 (9), pp. 4555-4576. DOI: 10.1109/TPAMI.2021.3069908.
46. Woo, S., Debnath, S., Hu, R., Chen, X., Liu, Z., Kweon, I.S., Xie, S. (2023), "ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders", *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16133-16142. DOI: 10.1109/CVPR52729.2023.01548.
47. Xu, H., Ebner, S., Yarmohammadi, M., White, A.S., Van Durme, B., Murray, K. (2021), "Gradual Fine-Tuning for Low-Resource Domain Adaptation", *Proceedings of the Second Workshop on Domain Adaptation for NLP, Association for Computational Linguistics*, pp. 214-221.
48. Xu, M., Yoon, S., Jeong, Y., Park, D.S. (2022), "Transfer learning for versatile plant disease recognition with limited data", *Frontiers in Plant Science* Vol. 13, Article 1010981. DOI: 10.3389/fpls.2022.1010981.
49. Yosinski, J., Clune, J., Bengio, Y., Lipson, H. (2014), "How transferable are features in deep neural networks?", *Advances in Neural Information Processing Systems* Vol. 27, pp. 3320-3328.
50. Zhang, H., Li, S., Xie, J., Chen, Z., Chen, J., Guo, J. (2025), "VMamba for plant leaf disease identification: design and experiment", *Frontiers in Plant Science* Vol. 16, Article 1515021. DOI: 10.3389/fpls.2025.1515021.

51. Zhu, L., Liao, B., Zhang, Q., Wang, X., Liu, W., Wang, X. (2024), "Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model", *Proceedings of the 41st International Conference on Machine Learning, PMLR* Vol. 235, pp. 62429-62442.

Mango-Mamba and VN-MangoLeaf: A Lightweight Mamba Model and New Dataset for Mango Leaf Disease Classification

Thien B. Nguyen-Tat[a,b,*], Binh Pham-Thanh[a,b]

[a] University of Information Technology, Ho Chi Minh City, Vietnam
[b] Vietnam National University, Ho Chi Minh City, Vietnam

ARTICLE INFO

Keywords:
Deep Learning
Mango Leaf Classification
Mamba Models
Image processing
Smart agriculture

ABSTRACT

Mango leaf disease represents a significant threat to fruit quality and yield, necessitating highly accurate, real-time detection systems. However, existing Deep Learning approaches, particularly Transformer-based models, often suffer from prohibitive computational complexity (quadratic scaling), limiting their deployment on resource-constrained edge devices. To address this challenge, this study introduces MangoMamba, a novel lightweight hybrid architecture specifically optimized for mobile deployment. The proposed model integrates Multi-Scale Mamba Mixers with Large-Kernel Attention mechanisms within a hierarchical four-stage framework, enabling linear computational complexity while preserving global receptive fields. Experimental evaluations were conducted on the MangoLeafBD dataset and the newly curated VN-MangoLeaf dataset, which comprises 7,000 images of Vietnamese mango varieties. Results demonstrate that MangoMamba achieves competitive classification accuracies of 99.75% and 98.71% on the respective datasets. Crucially, the model exhibits exceptional efficiency with only 5.8 million parameters and an inference latency of 1.46 ms per image on T4 GPU, approximately 80 times faster than recent ViX-MangoEFormer architectures. Furthermore, the practical feasibility of the proposed approach is validated through a functional Android application capable of offline inference (100-300 ms latency) on standard smartphones. These findings confirm that MangoMamba establishes a new competitive trade-off between accuracy and efficiency for smart agriculture applications.

1. Introduction

Mango (*Mangifera indica* L.) cultivation is a vital component of the global agricultural economy, yet it faces substantial production risks due to foliar pathogens. Diseases such as Anthracnose, Powdery Mildew, and Red Rust can cause yield losses exceeding 20-30% if not managed effectively [1]. Traditional disease management relies on manual visual inspection by pathologists, a process that is labor-intensive, subjective, and difficult to scale in large orchards or resource-limited regions [9]. Consequently, the development of automated, accurate, and real-time disease detection systems has become a priority in smart agriculture.

Recent advancements in deep learning have significantly improved automated plant disease diagnosis. Convolutional Neural Networks (CNNs) [14, 8] and Transformer-based architectures [6, 2] have achieved high classification accuracy on benchmark datasets. However, the practical deployment of these models on edge devices (e.g., smartphones, embedded sensors) is hindered by computational constraints. While Vision Transformers (ViTs) offer superior global context modeling compared to CNNs, they suffer from a quadratic computational complexity ($O(N^2)$) with respect to image resolution due to the self-attention mechanism [7]. This results in high memory consumption and latency, rendering them unsuitable for real-time applications in rural environments where internet connectivity for cloud processing is often unreliable [9].

Additionally, recent studies have demonstrated the potential of advanced hybrid architectures in addressing performance and computational challenges. For instance, Qin et al. [11] introduced CropCapsNet, which integrates an enhanced capsule network with Squeeze-and-Excitation Inception modules. This approach effectively captures multi-scale features for crop disease classification, achieving remarkable accuracy by overcoming the spatial information loss inherent in traditional CNNs. Similarly, Truong et al. [17] proposed SC-MambaFew, utilizing the Mamba architecture coupled with selective spatial-channel attention for bearing fault diagnosis. Their work highlights the capability of Mamba-based hybrids to model long-range dependencies efficiently, particularly in few-shot learning

thienntb@uit.edu.vn (T.B. Nguyen-Tat)
ORCID(s): 0000-0002-4809-7126 (T.B. Nguyen-Tat)

scenarios. However, while these pioneering works validate the efficacy of hybrid models, they do not explicitly target the optimization of Mamba-Attention mechanisms for the subtle, fine-grained visual characteristics of mango leaf diseases on strictly resource-constrained edge hardware. Furthermore, the issue of cross-domain generalization against geographical bias- a critical requirement in global agriculture-remains largely unaddressed in these architectural studies. This research builds upon these advancements to bridge that specific gap.

To address the limitations of quadratic complexity while ensuring high diagnostic precision, State Space Models (SSMs), particularly the Mamba architecture [3], have emerged as a promising alternative. Mamba introduces a selective scan mechanism that allows for modeling long-range dependencies with linear computational complexity ($O(N)$). Recent adaptations of Mamba for computer vision, such as MambaVision [5] and Vision Mamba [22], have demonstrated that SSMs can rival Transformers in performance while significantly reducing inference costs. Despite these advances, the specific optimization of hybrid Mamba-Attention architectures for lightweight agricultural disease classification remains underexplored. Existing lightweight models often sacrifice accuracy for speed or fail to capture the multi-scale features required to distinguish between morphologically similar mango leaf diseases [21].

Furthermore, a critical gap exists in the geographical diversity of available mango leaf disease datasets. The majority of public datasets, including MangoLeafBD [1] and MLD24 [15], originate primarily from Bangladesh. This geographical bias limits the generalization capability of models when applied to regions with different mango cultivars and disease prevalence, such as Vietnam, where distinct pathogens like Red Rust are more common [10].

To bridge these gaps, this study proposes **MangoMamba**, a lightweight architecture specifically engineered for efficient mango leaf disease classification on edge devices. The architecture integrates Multi-Scale Mamba Mixers for linear-complexity global context modeling with Large-Kernel Attention (LKA) mechanisms for local feature refinement. Additionally, the **VN-MangoLeaf** dataset is introduced to address regional data scarcity, complemented by a curriculum learning strategy to enhance cross-domain generalization.

The primary contributions of this research are summarized as follows:

1. A specialized hybrid architecture, MangoMamba, is developed, combining the efficiency of Selective State Space Models with the spatial precision of Large-Kernel Attention. This design achieves linear computational complexity, significantly reducing inference latency compared to Transformer-based counterparts.
2. The VN-MangoLeaf dataset is introduced, comprising 7,000 images collected from Vietnamese orchards. This dataset mitigates geographical bias by including regionally specific classes such as Red Rust, enabling more robust model evaluation.
3. A three-phase curriculum learning methodology is implemented to facilitate effective knowledge transfer between datasets with varying class distributions, enhancing the model's robustness against morphological variations.
4. Extensive experimental validation demonstrates that the proposed model achieves competitive accuracy (99.75% on MangoLeafBD and 98.14% on VN-MangoLeaf) while maintaining an inference time of approximately 1.5 ms per image, verifying its suitability for real-time mobile deployment.

2. Materials and Methods

Datasets overview

Existing public datasets for mango leaf disease detection, such as MangoLeafBD [1] and MLD24 [15], have provided valuable benchmarks for the research community. However, these datasets originate predominantly from Bangladesh, which limits the generalization capability of models when applied to other geographical regions with different environmental conditions and pathogen distributions. To address this geographical bias and the lack of diverse regional data, this study introduces the **VN-MangoLeaf** dataset, collected extensively across southern Vietnam.

A critical distinction of the VN-MangoLeaf dataset is the specific disease distribution observed in Vietnam compared to Bangladesh (Figure 1). While diseases such as 'Cutting Weevil' and 'Bacterial Canker' are common in previous datasets, they are infrequently observed in Vietnamese mango orchards. Conversely, 'Red Rust', caused by the algal plant pathogen *Cephaleuros virescens* [10], is highly prevalent in this region but underrepresented in existing benchmarks. Red Rust manifests as orange-brown, rust-like spots, typically on the upper leaf surface [10], as illustrated in Figure 2.

The dataset comprises seven distinct classes: Healthy, Anthracnose, Gall Midge, Powdery Mildew, Sooty Mold, Dieback, and Red Rust. Images were acquired using a Xiaomi Redmi Note 11S smartphone camera. To ensure high

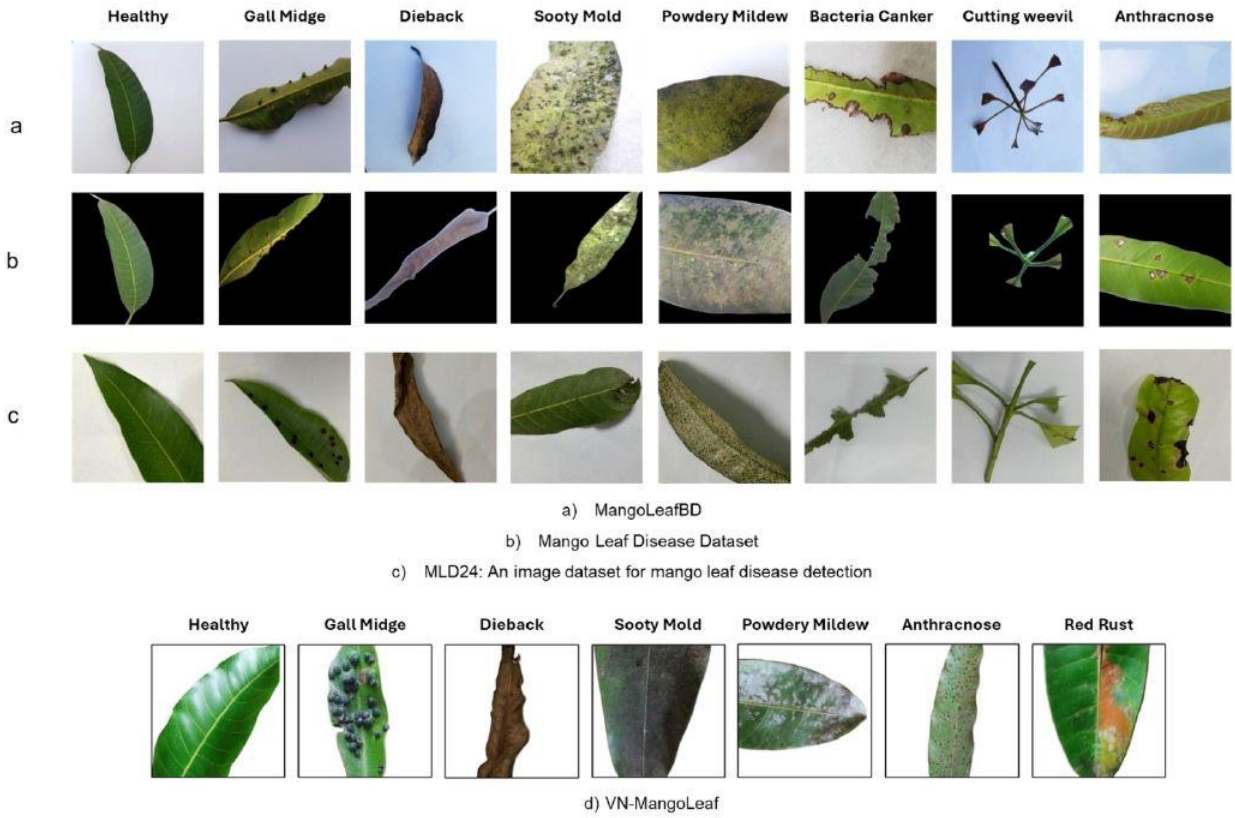


Figure 1: Comparison of disease classes: 8 classes from existing Bangladesh-based datasets versus 7 classes from the proposed VN-MangoLeaf dataset.

fidelity, images were captured at a native resolution of 3000×4000 pixels under natural lighting conditions between 7:00 AM and 4:00 PM, capturing variations in sunlight intensity and shadowing. The collection sites covered diverse locations in Southern Vietnam, including Binh Thanh District, District 12, District 9 (Ho Chi Minh City), Cao Lanh City (Dong Thap Province), and Giong Trom District (Ben Tre), as described in Figure 3.

To ensure label accuracy and dataset consistency, a two-stage verification process was implemented. Initial labeling was performed by the research team, followed by cross-verification by experienced mango orchard owners, who acted as domain experts. This process resulted in the removal of approximately 2,000 images identified as ambiguous, mislabeled, or of poor quality. The final curated dataset comprises 7,000 images (post-balancing), with at least 700 verified original images per class prior to augmentation. A comparison between VN-MangoLeaf and existing datasets is presented in Table 1.

2.1. Data augmentation

To facilitate the proposed Curriculum Learning strategy, this study utilizes two distinct categories of data serving complementary roles: a large-scale **Combined Dataset** serving as the source domain for general feature learning, and the proprietary **VN-MangoLeaf** dataset serving as the target domain for regional adaptation. Three publicly available datasets originating from Bangladesh: MangoLeafBD [1], MLD24 [15], and the Mendeley Mango Leaf Disease Dataset [12] were consolidated. This merged corpus, referred to as the *Combined Dataset*, provides a diverse range of 8 disease classes but suffers from significant class imbalance (Figure 4). It's primarily used in the early training phases to expose the model to a wide variety of visual symptoms.

In contrast, the **VN-MangoLeaf** dataset (described in Section 2) represents the specific environmental conditions and pathogen distributions of Vietnam. It is carefully verified by experts, contains the unique 'Red Rust' class, and excludes classes not prevalent in the region (Cutting Weevil and Bacterial Canker). This dataset is reserved for the



Figure 2: Representative samples of Mango leaves infected with Red Rust (*Cephaleuros virescens*) from the VN-MangoLeaf dataset.



Figure 3: The mango orchards locations of VN-MangoLeaf

final fine-tuning phase and performance benchmarking to validate the model's real-world applicability in the target geographical region.

As illustrated in Figure 4, the initial distribution of the three Bangladesh mango leaf datasets exhibited significant imbalance, with classes such as Bacterial Canker being overrepresented compared to Powdery Mildew. Such disparities can lead to model overfitting on the majority classes.

Table 1
Comparison between mango leaf disease datasets from Bangladesh and our proposed dataset from Vietnam

Dataset	Data format	Resolution	Image amount	Type of disease	Class	Image per class	Locations of images
VN-MangoLeaf	JPG	240x320	7000 (4900 original + 2100 augmented)	Healthy; Anthracnose; Powdery Mildew; Dieback; Sooty Mold; Gall Midge; Red Rust	7	1000	Giong Trom (Ben Tre); Cao Lanh (Dong Thap); District 12 (TPHCM); Binh Thanh (TPHCM); District 9 (TPHCM)
MangoLeafBD	JPG	240x320	4000 (1800 original + 2200 augmented)	Healthy; Anthracnose; Powdery Mildew; Dieback; Sooty Mold; Gall Midge; Bacterial Canker; Cutting Weevil	8	500	Sher-e-Bangla Agricultural University orchard; Jahangir Nagar University orchard; Udaypur village orchard; Itakhola village orchard (Bangladesh)
MLD24	JPG	240x240	6400 (all original)	Healthy; Anthracnose; Powdery Mildew; Dieback; Sooty Mold; Gall Midge; Bacterial Canker; Cutting Weevil	8	800	Kushtia; Dhaka (Bangladesh)

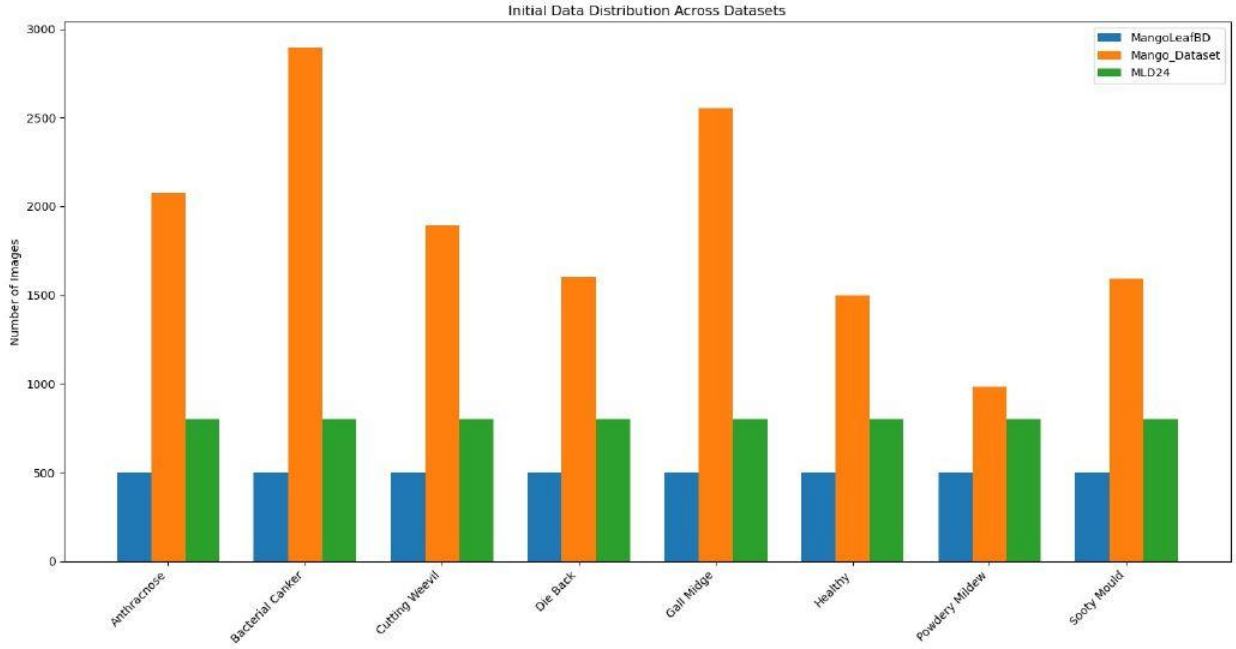


Figure 4: The initial class distribution across the component datasets before balancing, highlighting significant imbalances.

To mitigate this, augmentation strategies adapted from Noman et al. [9] were employed. Geometric transformations (rotation, flipping) and photometric adjustments (brightness, scaling) were applied specifically to underrepresented classes to achieve a target balance. Table 2 details the augmentation parameters and preprocessing steps.

Following these procedures, the Combined Dataset achieved a balanced distribution with approximately 5,000 images per class, totaling 40,000 images. Figure 5 depicts the class distribution post-augmentation.

Table 2
Data augmentation parameters and preprocessing specifications.

Processing Step	Description & Parameters
1. Augmentation	Applied stochastically to achieve target counts
Rotation	Range: $[-45°, 45°]$
Flipping	Vertical and Horizontal
Brightness	Factor range: $[0.8, 1.2]$
Scaling	Factor range: $[0.5, 1.5]$
2. Resizing	Uniform resizing to 224×224 pixels (JPG format)
3. Normalization	Min-Max scaling to the range $[0, 1]$ for stable convergence.

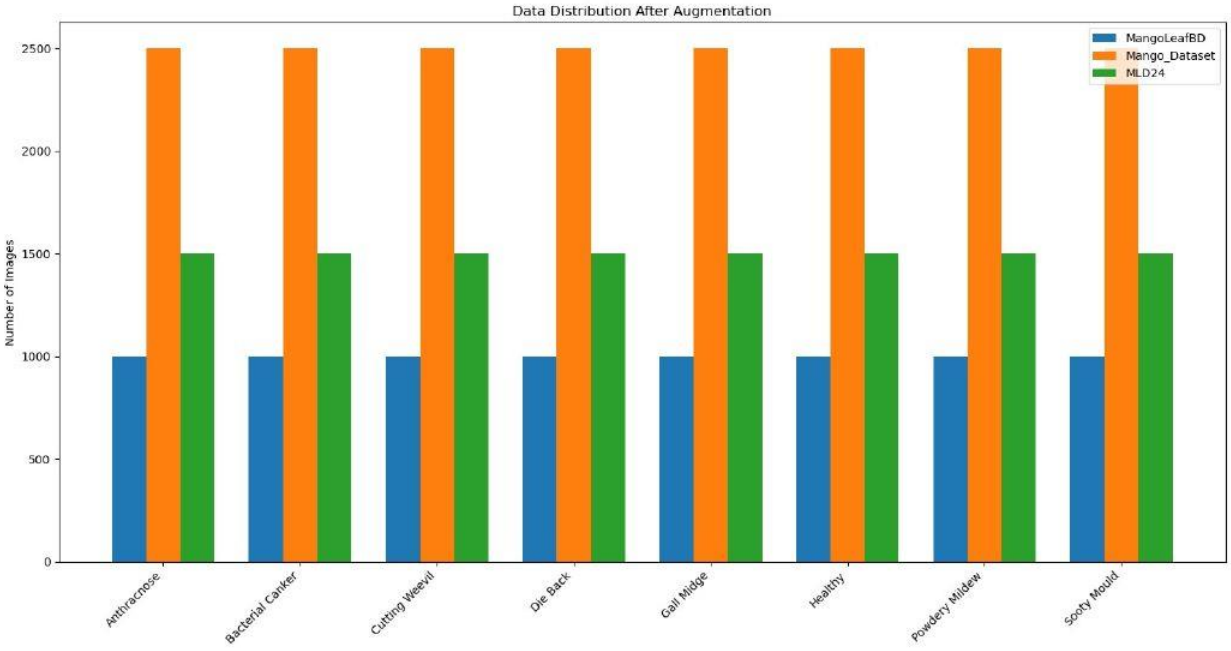


Figure 5: Balanced class distribution of the Combined Dataset following the application of augmentation strategies.

To ensure consistent feature scaling and accelerate gradient descent convergence, all images underwent Min-Max normalization. Each pixel value x was scaled to x' using Equation 1:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{1}$$

where $x_{\max}$ and $x_{\min}$ represent the maximum (255) and minimum (0) pixel intensities, respectively.

To strictly evaluate generalization performance and prevent data leakage, the datasets were partitioned using a stratified random split with a ratio of 80:10:10 for training, validation, and testing, respectively. The specific distribution of images across these subsets for the MangoLeafBD, Combined, and VN-MangoLeaf datasets is summarized in Table 3.

2.2. MangoMamba model's architecture

The proposed MangoMamba architecture is a lightweight, hierarchical vision backbone specifically engineered for mango leaf disease recognition on edge devices. It prioritizes a compact model size and rapid inference capabilities

Table 3
Distribution of images across Training, Validation, and Test sets for each experimental dataset.

Dataset	Train (80%)	Validation (10%)	Test (10%)
MangoLeafBD	3,200	400	400
Combined Dataset	32,000	4,000	4,000
VN-MangoLeaf	5,600	700	700

while maintaining high classification accuracy. The architecture adopts a four-stage structure with progressively decreasing spatial resolutions ($H/4 \rightarrow H/32$) and increasing channel dimensions ($C \rightarrow 8C$). To maintain linear computational complexity, each stage processes features through non-overlapping windows.

Unlike standard hybrid architectures that combine State Space Models (SSM) with computationally intensive Transformer blocks, MangoMamba introduces a hybrid integration of Mamba Mixers and Large-Kernel Attention (LKA). This design strategically utilizes the global receptive field of SSMs for sequence modeling and the spatial adaptability of LKA for capturing local disease textures, effectively eliminating the quadratic complexity associated with traditional Self-Attention.

The model integrates several advanced components, selected based on extensive ablation studies to maximize the trade-off between efficiency and performance:

- **Global Response Normalization (GRN):** Utilized to enhance the contrast and selectivity of channels, thereby preventing feature collapse and stabilizing training dynamics. This component, adapted from ConvNeXt V2 [20], ensures that feature diversity is maintained across the deep layers of the network.
- **Efficient Channel Attention (ECA):** Implemented to provide channel-wise feature enhancement with negligible parameter overhead [19]. By emphasizing informative channels, ECA allows the model to focus on distinct disease characteristics amidst complex leaf backgrounds.
- **Windowed Processing:** Inspired by the hierarchical design of the Swin Transformer [7], feature maps are partitioned into local windows. This approach facilitates efficient local attention computation and hierarchical feature representation, eliminating the heavy computational burden associated with global attention.
- **Multi-Scale Mamba Mixers:** Inspired by the MambaVision architecture [5], this component is adapted to model multi-resolution features. It preserves the linear scaling superiority of SSMs [3], enabling the model to effectively capture long-range dependencies across the leaf surface.
- **Large-Kernel Attention (LKA):** Incorporated to replace the Transformer component found in typical hybrid models. LKA decomposes large kernel convolutions to capture spatial correlations and long-range dependencies with lower computational cost and parameter count compared to standard self-attention mechanisms [4].
- **SwiGLU Activation:** Employed within the Multi-Layer Perceptron (MLP) blocks, SwiGLU offers enhanced representational capacity compared to standard GELU or ReLU activations [16]. It operates as a gated linear unit with a sigmoid-activated component, improving the model's ability to learn complex non-linear disease patterns.

As visualized in Figure 6, the input processing commences with a lightweight patch embedding module. This module efficiently transforms raw images into patch-level representations using two consecutive stride-2 convolutions, reducing the spatial dimension to a quarter of the input resolution while projecting to the base channel dimension.

To ensure robust feature initialization, this embedding is immediately refined by the ECA and GRN mechanisms. The ECA module generates channel-wise attention weights by applying a parameter-efficient 1D convolution over global average pooling features, followed by a sigmoid activation. These weights are then element-wise multiplied with the input features to recalibrate channel importance. Subsequently, the GRN mechanism stabilizes feature responses by normalizing the spatial L2-norm across height and width dimensions, followed by a learnable affine transformation. This process emphasizes spatially important regions and suppresses redundant information.

The core of the architecture employs a hierarchical four-stage design utilizing windowed processing. In each stage, feature maps are partitioned into non-overlapping windows of size $S \times S$. This partitioning transforms the input tensor

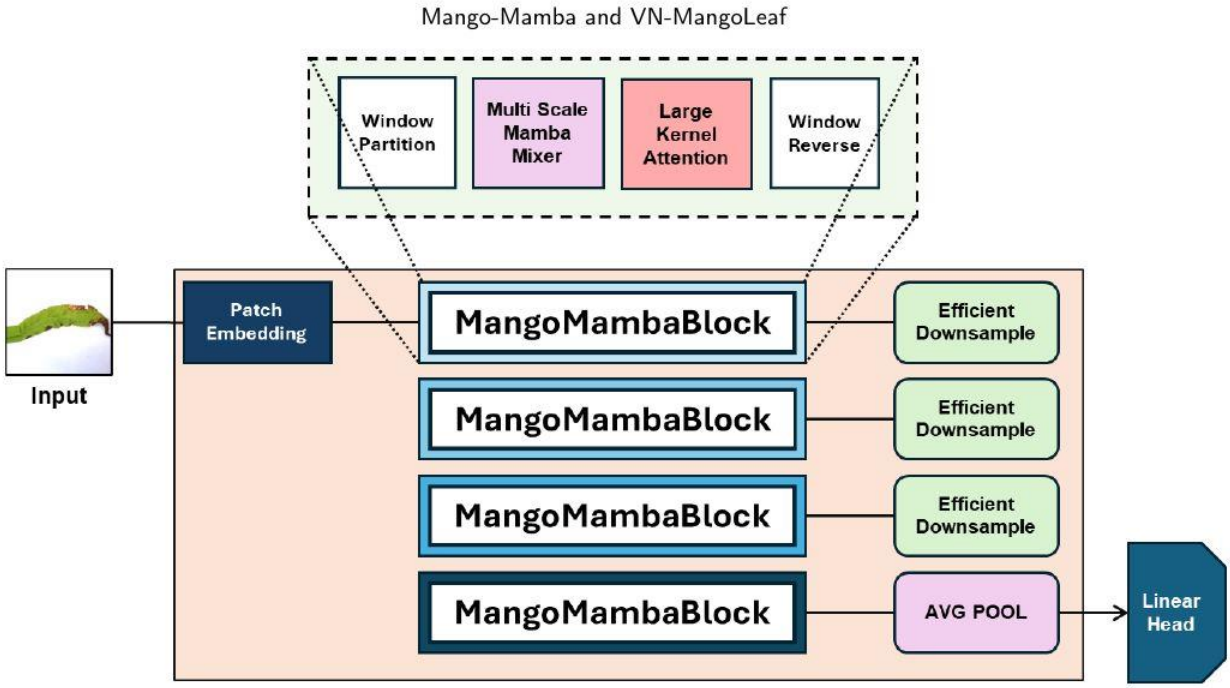


Figure 6: The architectural overview of the proposed MangoMamba model

into a windowed sequence, allowing the model to perform computations on smaller, fixed-size token sets rather than the entire image at once. This strategy significantly reduces memory usage and computational complexity.

Within the processing stages, the architecture utilizes a specific intra-stage hybrid strategy. The initial blocks of each stage employ Mamba Mixers to scan the windowed sequences and establish global context, while the subsequent blocks utilize LKA to refine spatial details and texture information. This order ensures that the model first understands the global structure of the leaf and lesion distribution before focusing on the fine-grained visual patterns of the disease. Between stages, depthwise separable convolutions with a stride of 2 are employed for efficient downsampling, followed by additional ECA and GRN modules to preserve feature distinctiveness during resolution reduction.

2.3. MangoMambaBlock

The MangoMambaBlock serves as the foundational computational unit within each stage of the architecture, designed to efficiently extract and refine features for mango leaf disease classification. As illustrated in Figure 7, this block utilizes a dual-pathway residual structure that integrates advanced mixing mechanisms with a robust feed-forward network. The design is engineered to balance the capture of local textural details-essential for identifying specific lesions like Anthracnose-with global context, which is required to detect broader stress symptoms such as Dieback.

To achieve this balance while minimizing computational overhead, the architecture employs an intra-stage hybrid strategy. The initial blocks of a stage utilize the Multi-Scale Mamba Mixer to model long-range dependencies and global sequence patterns with linear complexity. Conversely, the latter blocks of the stage switch to Large-Kernel Attention (LKA) to refine explicit spatial attention and local correlations. This alternation is determined by the block's depth index; empirically, activating LKA in the second half of the stage enhances the model's ability to discriminate between visually similar disease classes by focusing on fine-grained spatial textures after the global context has been established.

The signal propagation within the MangoMambaBlock follows a standard pre-normalization residual design. The input token sequence first undergoes Layer Normalization before entering the designated Mixer module (either Mamba or LKA). The output of the mixer is scaled by a learnable layer-scaling coefficient and subjected to stochastic depth (DropPath) regularization before being added back to the original input. This process is repeated for the second half of the block, which consists of a Layer Normalization step followed by a SwiGLU-based Feed-Forward Network (FFN). The inclusion of learnable layer scaling is critical for stabilizing the training of deep networks by mitigating potential gradient distinctness issues.

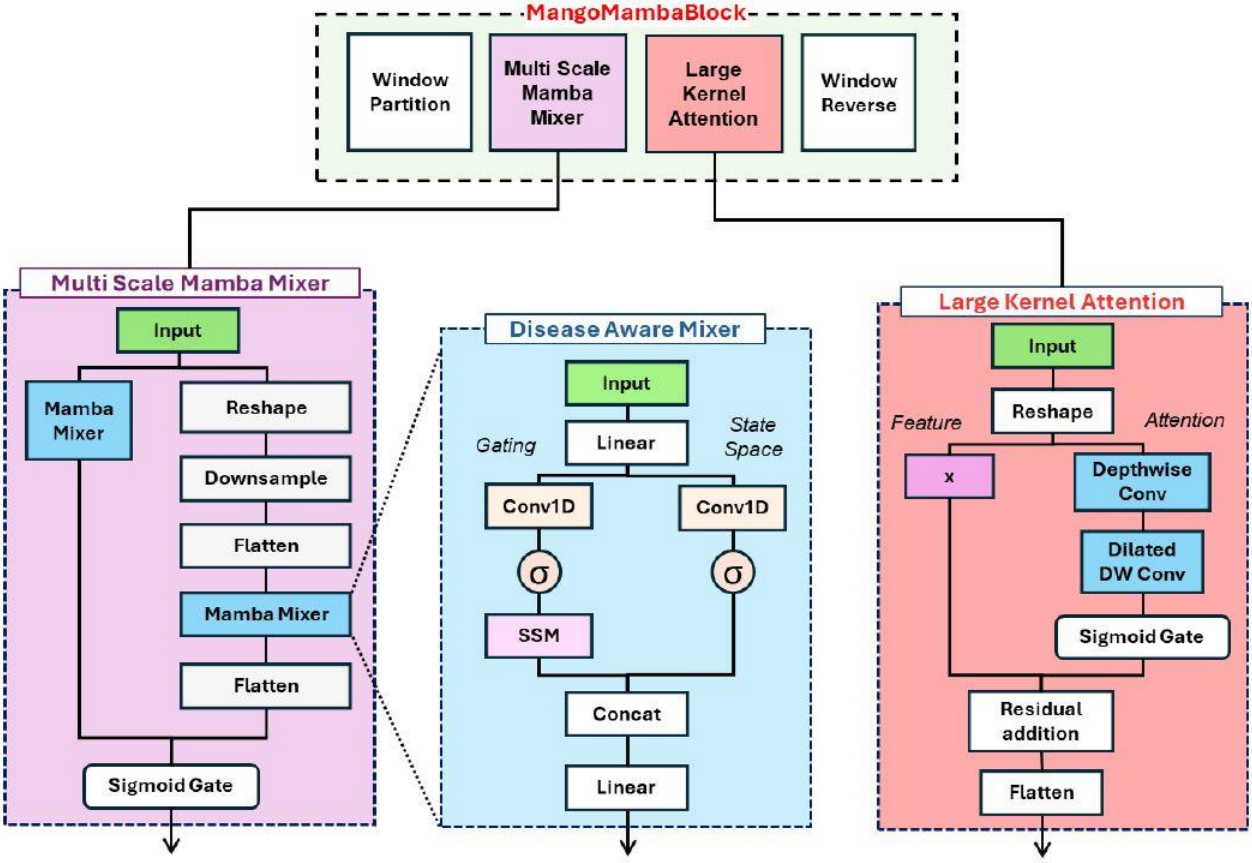


Figure 7: The architectural diagram of the MangoMambaBlock, illustrating the dual-pathway design.

To address the challenge of scale variance in disease symptoms, the Multi-Scale Mamba Mixer processes windowed tokens through two parallel branches. A high-resolution branch processes the native token sequence to preserve fine details, while a low-resolution branch operates on a spatially downsampled version to capture broader semantic features. The low-resolution input is generated via bilinear interpolation, processed, and then upsampled back to the original size. Both branches utilize the Disease Aware Mixer (DAM), and their outputs are fused via a learnable, channel-wise sigmoid gate. This gating mechanism allows the network to adaptively weigh the importance of fine versus coarse features for each channel, optimizing the representation without the parameter explosion typical of dense fusion methods.

The DAM module is a lightweight adaptation of the mixer found in MambaVision [5], specifically optimized for disease pattern recognition. It reduces the state dimension and convolution kernel size to prioritize inference speed on edge devices while preserving the selective reasoning capabilities of State Space Models (SSM). Internally, the input projection is split into two streams. The first stream undergoes a 1D depthwise convolution to capture local context, followed by a selective state-space operation where the discretized state-transition parameters are derived dynamically from the input. This enables the model to selectively propagate or forget information along the sequence, effectively modeling the interconnected nature of disease symptoms across leaf veins. The output is then gated by the second stream (the z-stream) to regulate information flow before the final output projection.

In the later layers of the stage, the block utilizes Large-Kernel Attention to generate spatial attention maps. Unlike the Mamba mixer, LKA is designed to capture spatial correlations directly without the quadratic cost of self-attention. It decomposes a large kernel convolution into a sequence of smaller operations: a standard depthwise convolution to capture local dependencies, followed by a dilated depthwise convolution to expand the receptive field. In this

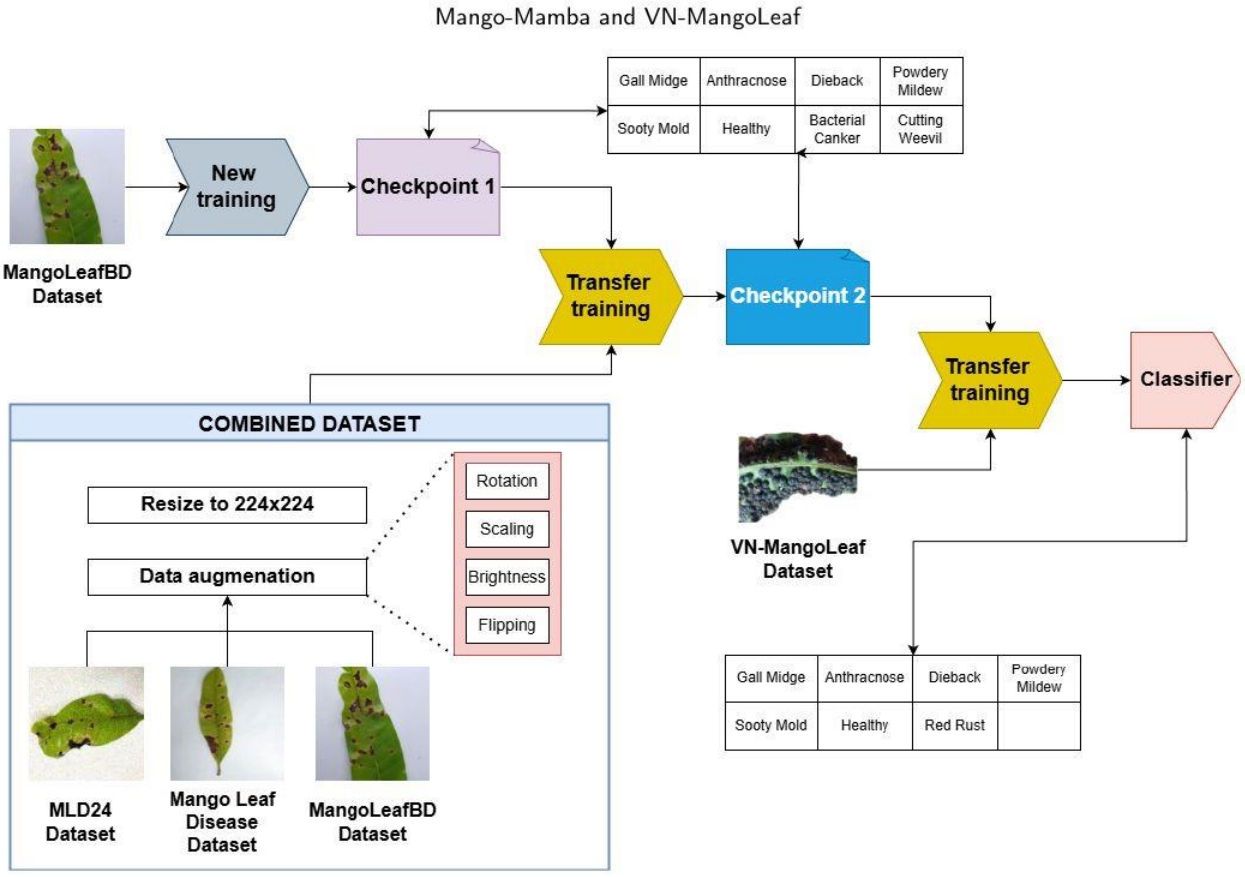


Figure 8: The proposed three-phase training curriculum, progressing from the source dataset (MangoLeafBD) to a combined robust dataset, and finally adapting to the target domain (VN-MangoLeaf).

implementation, a kernel size of 3×3 is used for the local convolution, followed by a 5×5 kernel with a dilation rate of 2. This combination effectively simulates a large receptive field, allowing the model to encompass larger irregular patterns, such as red rust pustules, while remaining computationally efficient. The resulting attention map is applied to the input via element-wise multiplication.

To enhance the non-linear representational capacity of the block, a Position-wise Feed-Forward Network with SwiGLU activation is employed. Selected based on ablation results demonstrating its superiority over standard GELU, the SwiGLU mechanism projects the input into two parallel linear layers. One path is activated by the SiLU function and then element-wise multiplied by the output of the second path. This gating interaction allows the network to learn more complex feature transformations, contributing to the high classification accuracy observed in the experiments.

2.4. Proposed methodology

To address data scarcity and adapt to the specific label-set variations across different agricultural datasets, a three-phase curriculum learning strategy is implemented. This approach, visualized in Figure 8, progressively broadens the data diversity before narrowing down to the specific target domain.

- **Phase 1: Source Domain Initialization (MangoLeafBD).** The MangoMamba backbone is initialized with an 8-class linear projection head. Training is conducted end-to-end on the MangoLeafBD dataset. This phase establishes a foundational representation of canonical mango lesion features, creating a baseline checkpoint.
- **Phase 2: Robust Feature Learning (Combined Dataset).** The checkpoint from Phase 1 is loaded and fine-tuned on a composite dataset constructed from MangoLeafBD, the Mango Leaf Disease Dataset, and MLD24. This phase exposes the model to significant variations in illumination, capture devices, and lesion morphology while maintaining the original 8-way classification scheme. Lower layers are optionally frozen during the initial epochs

to preserve low-level feature extractors, followed by full-model adaptation. This step is critical for learning invariance to environmental conditions.

- **Phase 3: Target Domain Adaptation (VN-MangoLeaf).** In the final phase, the linear classifier is replaced to accommodate the 7-class label set of the target VN-MangoLeaf dataset. The backbone retains weights from the Combined checkpoint. Since the target dataset excludes classes such as 'Cutting Weevil' and 'Bacterial Canker' but emphasizes 'Red Rust', the adaptation process biases the updates toward high-level semantic features while retaining the low-level textural invariances learned in Phase 2. This strategy effectively manages the domain shift and class imbalance inherent in the target environment.

Algorithm 1 formally describes the complete training procedure and the internal forward pass mechanism of the proposed model.

Algorithm 1 MangoMamba Training Curriculum and Inference Procedure

Require: Source Data $\mathcal{D}_{src}$, Combined Data $\mathcal{D}_{comb}$, Target Data $\mathcal{D}_{tgt}$
Ensure: Final Optimized Model $\mathcal{M}_{final}$

Part I: Three-Phase Training Curriculum

$\mathcal{M} \leftarrow \text{InitializeMangoMamba}(classes = 8)$
$\theta_A \leftarrow \text{Train}(\mathcal{M}, \mathcal{D}_{src})$ ▷ Phase 1: Feature Initialization
$\mathcal{M}.\text{load_weights}(\theta_A)$
$\theta_B \leftarrow \text{FineTune}(\mathcal{M}, \mathcal{D}_{comb})$ ▷ Phase 2: Robustness to Variations
$\mathcal{M}.\text{load_weights}(\theta_B)$
$\mathcal{M}.\text{head} \leftarrow \text{Reinitialize}(in = 384, out = 7)$
$\mathcal{M}_{final} \leftarrow \text{FineTune}(\mathcal{M}, \mathcal{D}_{tgt})$ ▷ Phase 3: Domain Adaptation

Part II: Model Inference (Forward Pass)

function FORWARD(x_{rgb})
    $x \leftarrow \text{PatchEmbed}(x_{rgb})$
    $x \leftarrow \text{ECA}(x) + \text{GRN}(x)$
    for $i = 1$ **to** 4 **do**
        $x_{win} \leftarrow \text{WindowPartition}(x, \text{size} = W_i)$
        for $j = 1$ **to** $Depth_i$ **do**
            $x_{norm} \leftarrow \text{LayerNorm}(x_{win})$
            if $j < \lfloor Depth_i/2 \rfloor$ **then**
                $x_{win} \leftarrow x_{win} + \text{MambaMixer}(x_{norm})$ ▷ Global Context
            else
                $x_{win} \leftarrow x_{win} + \text{LKAMixer}(x_{norm})$ ▷ Local Texture
            end if
            $x_{win} \leftarrow x_{win} + \text{SwiGLU_MLP}(\text{LayerNorm}(x_{win}))$
        end for
        $x \leftarrow \text{WindowReverse}(x_{win})$
        $x \leftarrow \text{StageGRN}(x)$
        if $i < 4$ **then**
            $x \leftarrow \text{Downsample}(x)$
        end if
    end for
    $y \leftarrow \text{LinearHead}(\text{GlobalAvgPool}(x))$
    return y
end function

Table 4
Comprehensive ablation analysis of MangoMamba components evaluated on the MangoLeafBD dataset. To ensure statistical reliability and reproducibility, each experimental case was executed across four independent runs using distinct random seeds {42, 123, 2024, 3333}. All models were trained using a consistent hyperparameter configuration: 100 epochs, batch size of 256, initial learning rate of 0.0005 with 5 warmup epochs, and an early stopping patience of 20. The results are reported as Mean ± Standard Deviation. Model complexity is denoted in millions of parameters (M), while Validation and Test metrics (Accuracy and F1 Score) are measured in percentage. The final selected configuration, achieving the optimal trade-off between stability and performance, is highlighted in bold.
Note: (↑) indicates higher is better; (↓) indicates lower is better.

No.	Model Configuration (Ablation Case)	Params (M)↓	Val Accuracy ↑	Val F1 Score ↑	Test Accuracy ↑	Test F1 Score ↑
1	LKA without Mamba, ECA , GRN, SwiGLU	**3.01**	99.09 ± 0.49	99.22 ± 0.40	98.88 ± 0.63	99.19 ± 0.47
2	Mamba without LKA, ECA , GRN, SwiGLU	6.62	99.09 ± 0.21	99.12 ± 0.20	99.25 ± 0.35	99.12 ± 0.32
3	LKA without Mamba	4.41	98.63 ± 0.77	98.78 ± 0.70	98.44 ± 1.45	98.50 ± 1.34
4	Hybrid without ECA , GRN, SwiGLU	4.41	99.09 ± 0.45	99.22 ± 0.36	99.12 ± 0.63	99.19 ± 0.75
5	Hybrid without ECA	5.81	98.59 ± 0.99	98.78 ± 1.01	98.56 ± 1.52	98.75 ± 1.34
6	Hybrid without SwiGLU	4.41	99.25 ± 0.27	99.31 ± 0.16	99.31 ± 0.38	99.44 ± 0.12
7	Hybrid without GRN	5.81	99.56 ± 0.16	99.56 ± 0.16	**99.50 ± 0.20**	99.62 ± 0.32
8	Hybrid with fixed Window [7, 7, 7, 7]	5.81	**99.62 ± 0.10**	**99.62 ± 0.10**	99.44 ± 0.31	**99.69 ± 0.24**
9	Hybrid Baseline (Original) [8, 8, 14, 7]	5.81	98.59 ± 0.91	98.66 ± 0.85	98.56 ± 0.97	98.57 ± 1.02
10	Hybrid on VN-MangoLeaf without Curriculum	5.81	97.22 ± 0.43	97.28 ± 0.39	98.07 ± 0.34	97.63 ± 0.57

3. Results and Discussion

3.1. Ablation study

To validate the effectiveness of the proposed MangoMamba architecture and justify the inclusion of each specific component, a comprehensive ablation study was conducted on the MangoLeafBD dataset. The experiments were performed using consistent hyperparameters across four independent runs for each configuration to ensure statistical reliability. The results, summarizing the impact of the Mixer core, enhancement modules (ECA, GRN, SwiGLU), and windowing strategies, are presented in Table 4.

The core hypothesis of MangoMamba is that combining State Space Models with Large-Kernel Attention offers a balanced trade-off between global context modeling and parameter efficiency compared to pure architectures. As shown in Table 4, the **Pure LKA** model (Case 1) is the most lightweight (3.01M parameters) but lacks the sequence modeling capabilities required for complex disease patterns, resulting in lower stability. Conversely, the **Pure Mamba** model (Case 4) achieves competitive accuracy (99.25%) but incurs a significant parameter penalty (6.62M), which is less ideal for edge deployment. The **Hybrid** architecture without additional features (Case 3) strikes a balance, achieving 99.12% accuracy with 4.41M parameters. This confirms that integrating Mamba Mixers in early layers to capture global dependencies and LKA in later layers for spatial refinement provides an optimal foundation for the model.

The necessity of the integrated components—Efficient Channel Attention (ECA), Global Response Normalization (GRN), and SwiGLU—was evaluated by selectively removing them from the hybrid baseline.

- **Impact of ECA (Case 6):** Removing the ECA module resulted in a significant increase in variance (±1.52%) and a drop in accuracy to 98.56%. This indicates that the channel-wise attention mechanism is critical for stabilizing feature selection and focusing the model on relevant disease symptoms amidst background noise.
- **Impact of GRN (Case 7):** While removing GRN maintained high accuracy (99.50%), the inclusion of GRN in the final configuration (Case 8) contributed to a higher F1-score (99.69% vs. 99.62%). GRN is retained to prevent feature collapse in deep layers and ensure the model prioritizes high-contrast features, which is essential for generalization across diverse datasets.
- **Impact of SwiGLU (Case 5):** Replacing SwiGLU with standard activations resulted in a slight decrease in F1-score compared to the optimal configuration. The SwiGLU activation enhances the non-linear representational capacity of the MLP blocks, allowing the model to capture more intricate decision boundaries between similar disease classes.

Table 5
Hyperparameter configuration for training MangoMamba.

Hyperparameter	Value
Training Epochs	100
Batch Size	256
Base Learning Rate	5×10^{-4}
Warmup Epochs	5
Optimizer	AdamW
Learning Rate Scheduler	Cosine Annealing
Early Stopping Patience	20 epochs

A critical finding from the ablation study is the impact of window size configuration. The initial design utilized a multi-scale window strategy ([8, 8, 14, 7]) to mimic hierarchical receptive fields (Case 9). However, empirical results demonstrate that a **Fixed Window** strategy of [7, 7, 7, 7] (Case 8) yields better performance, achieving the highest F1-score of 99.69% with the lowest standard deviation ($\pm$ 0.24%). This suggests that a consistent local context size of 7×7 aligns better with the typical spatial extent of mango leaf lesions at the processed feature resolutions, minimizing boundary artifacts often introduced by varying window sizes. Consequently, the Fixed Window configuration (Case 8) was selected as the final architecture for MangoMamba.

3.2. Training setup

The experimental validation of the MangoMamba model was conducted within a standardized computational environment to ensure reproducibility and accessibility for resource-constrained agricultural applications. All training and evaluation procedures were executed on the Google Colab platform (Free Tier), utilizing a single NVIDIA Tesla T4 GPU (15 GB VRAM) and 12.7 GB of system RAM. The software stack was built upon Python 3.12 and PyTorch 2.6 with CUDA 12.6 support. Key dependencies included NumPy 2.2.2, Matplotlib 3.10.3, and Scikit-learn 1.7.1 for data handling and analysis; Transformers 4.50, TIMM 1.0.9, and Einops 0.8.1 for model architecture components; and Mamba-SSM 2.2.4 for the state-space modeling operations.

As delineated in Section 2.1, the datasets were stratified into distinct subsets: 80% for training, 10% for validation, and 10% for testing. To ensure assessment of generalization capability, all performance metrics reported in subsequent sections are derived exclusively from the held-out test set, which comprises images completely unseen during the optimization process.

The training configuration was standardized across all experiments to maintain consistency. The specific hyperparameters are detailed in Table 5. The model was optimized using the AdamW optimizer with a cosine annealing learning rate scheduler, initialized at a learning rate of 5×10^{-4} following a 5-epoch linear warmup period. This warmup strategy is critical for stabilizing the early training dynamics of the hybrid Mamba-LKA architecture.

Despite the substantial volume of training data, a comprehensive suite of regularization techniques was employed to mitigate overfitting and simulate the noise characteristics of real-world deployment. Label smoothing was applied to prevent the model from becoming over-confident in its predictions, thereby encouraging softer probability distributions. Weight decay was utilized to penalize large weights and promote parameter sparsity. Stochastic depth (DropPath) was implemented to randomly drop residual paths during training, effectively training an implicit ensemble of sub-networks. Additionally, gradient clipping was enforced to stabilize the optimization of the recurrent state-space components, and an early stopping mechanism with a patience of 20 epochs was used to halt training when validation performance plateaued. These measures collectively ensure that the final deployed model maintains robust performance on edge devices.

3.3. Evaluation metrics

To assess the performance of our proposed model for classifying multiple mango leaf diseases on edge devices, MangoMamba was evaluated on four widely adopted evaluation metrics: Accuracy, Precision, Recall, and F1 Score. These metrics provide a comprehensive evaluation of the model's classification performance across diverse disease classes.

Accuracy quantifies the proportion of correctly classified samples relative to the total number of samples. It is defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{2}$$

where TP (True Positives) is the number of correctly predicted positive samples, TN (True Negatives) is the number of correctly predicted negative samples, FP (False Positives) is the number of negative samples incorrectly predicted as positive, and FN (False Negatives) is the number of positive samples incorrectly predicted as negative.

Precision measures the proportion of correctly predicted positive samples among all samples predicted as positive, emphasizing the model's ability to minimize false positives. It is calculated as:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3}$$

Recall evaluates the model's ability to identify all positive samples, reflecting its capacity to detect all instances of a specific disease. It is defined as:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{4}$$

F1 Score provides a balanced measure of Precision and Recall, beneficial for imbalanced datasets where one class may dominate. It is also the harmonic mean of Precision and Recall, computed as:

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5}$$

3.4. Confusion Matrix

To evaluate the classification consistency across different data distributions, the confusion matrices for the MangoLeafBD, Combined, and VN-MangoLeaf test sets are presented in Figure 9 and Figure 10.

Figures 9(a) and 9(b) illustrate the performance on the MangoLeafBD and Combined datasets, respectively, which both encompass the standard eight-class schema (Anthracnose, Bacterial Canker, Cutting Weevil, Die Back, Gall Midge, Healthy, Powdery Mildew, and Sooty Mold). A distinct dominance of diagonal elements is observed in both matrices, indicating high precision and recall rates. The minimal off-diagonal values suggest that the model successfully extracted discriminative features for each class, maintaining high separability even when the dataset size and diversity were expanded in the Combined set.

In the case of the VN-MangoLeaf dataset (Figure 10), which operates on a seven-class schema (excluding Bacterial Canker and Cutting Weevil while introducing Red Rust), the diagonal structure remains prominent. However, minor misclassifications are observable, particularly between 'Red Rust' and 'Anthracnose'. This phenomenon can be attributed to the morphological similarities between the early-stage necrotic spots of Anthracnose and the reddish-brown pustules of Red Rust, which can pose a challenge for feature extraction under varying lighting conditions. Despite these subtle inter-class similarities, the overall concentration of predictions along the diagonal confirms the reliability of the MangoMamba architecture for practical disease diagnosis.

3.5. Grad-CAM visualization

To validate the interpretability of the model and ensure that predictions rely on pathological features rather than background noise, Gradient-weighted Class Activation Mapping (Grad-CAM) was employed. This technique generates a coarse localization map highlighting the varying intensity of regions within the input image that most significantly influence the decision-making process of the final convolutional layer.

Consistent with the visual analysis framework employed in recent studies [9], the activation heatmaps were overlaid on representative samples from each class to verify semantic alignment. As depicted in Figure 11, the visualizations demonstrate a strong correlation between high-activation regions (red/yellow zones) and the biological symptoms of the diseases:

- For **Powdery Mildew**, the heatmap accurately localizes the white, dusty fungal patches spread across the leaf surface.

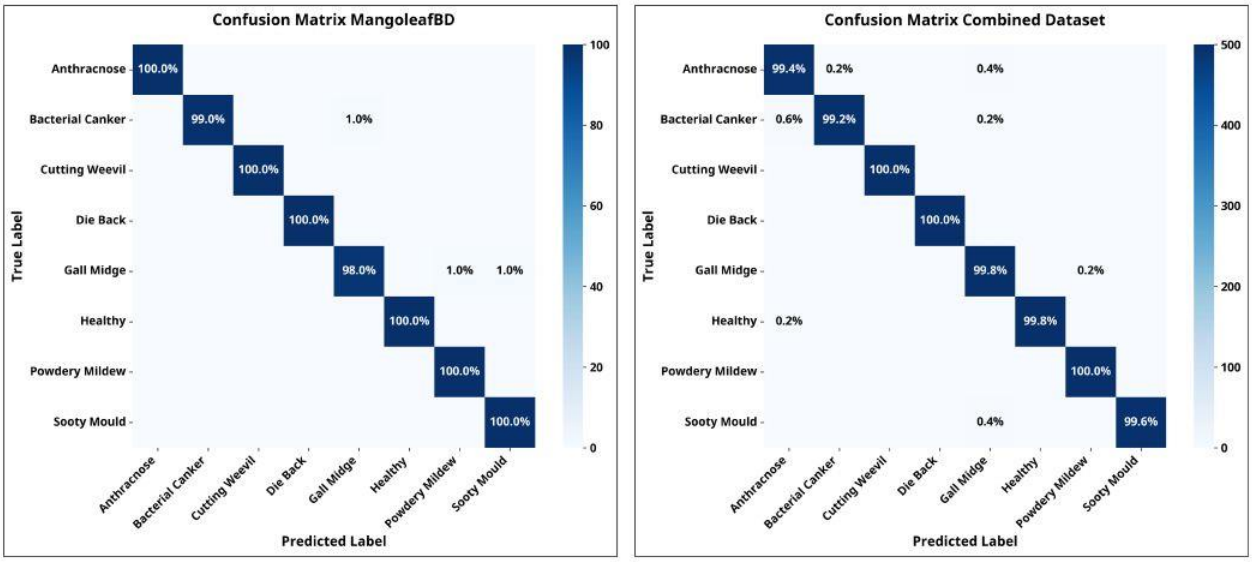


Figure 9: Confusion matrices evaluating classification performance on (a) the MangoLeafBD dataset and (b) the Combined dataset, sharing the same eight-class structure.

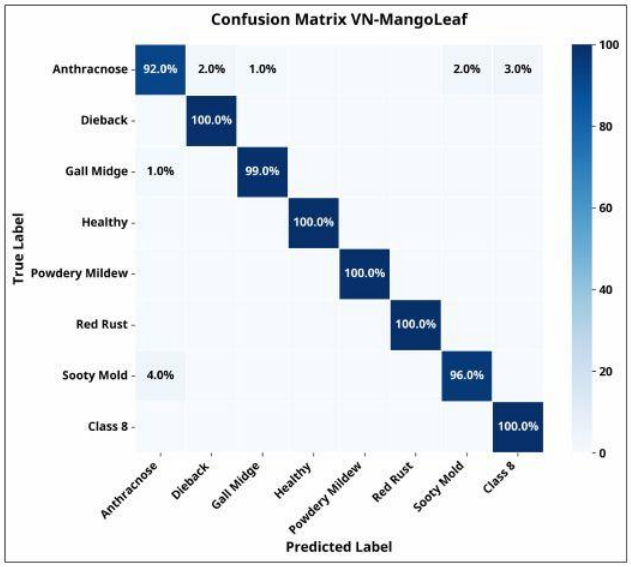


Figure 10: Confusion matrix for the VN-MangoLeaf dataset, illustrating performance across its specific seven-class schema.

- In cases of **Bacterial Canker** and **Anthracnose**, the model focuses intensely on the characteristic necrotic lesions and dark spots, ignoring the healthy green tissues.
- The visualizations for **Sooty Mold** and **Red Rust** confirm the model's attention to the black fungal coatings and reddish-brown pustules, respectively.
- For the **Healthy** class, the activation maps are characteristically diffuse with lower intensity, reflecting the absence of specific lesion patterns.

These qualitative results confirm that the proposed MangoMamba architecture effectively learns distinct disease markers and does not overfit to irrelevant background artifacts.

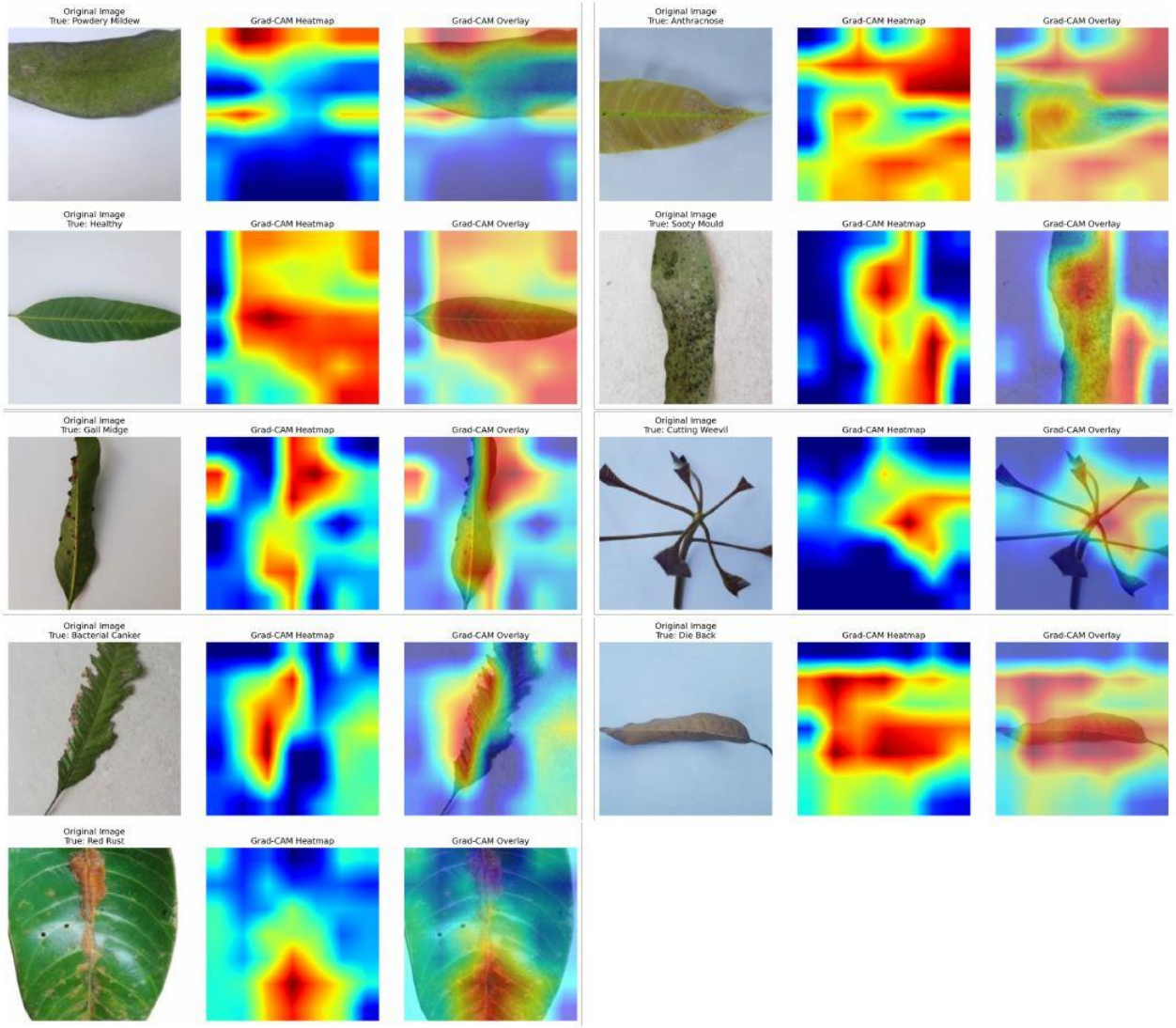


Figure 11: Grad-CAM visualization of representative images from each class. The columns display the original image, the activation heatmap, and the superposition of the heatmap on the original image, demonstrating the model's focus on disease-specific symptoms.

3.6. Performance comparison

Table 6 presents a comprehensive comparison of the proposed MangoMamba model against existing state-of-the-art (SOTA) approaches evaluated on mango leaf disease datasets. The comparison encompasses classification accuracy, computational efficiency (inference time and parameter count), and storage requirements suitable for edge deployment. It is important to note that direct comparisons of inference speed must account for hardware heterogeneity. As the source code and deployed mobile applications for the comparative studies were not publicly available, the performance metrics for these methods are cited directly from their respective publications. Specifically, the models analyzed by Hossain et al. (2024) [citation needed] were evaluated on an Intel Xeon CPU @ 2.20GHz, whereas the ViX-MangoEFormer by Noman et al. (2025) [9] reported inference speeds on mid-tier GPUs (e.g., NVIDIA Jetson Xavier NX or T4 - which is the same type as Google Colab's T4 GPU).

The proposed MangoMamba model demonstrates competitive classification performance, achieving an accuracy of 99.75% on the MangoLeafBD dataset, 99.65% on the Combined dataset, and 98.14% on the newly introduced



Figure 12: Demonstration of the MangoMamba model deployed on a mobile device via an Android application. The model runs locally using the ONNX runtime, enabling real-time disease classification without internet dependency.

VN-MangoLeaf dataset. While certain heavy architectures such as Fused ViT-ConvNeXt [2] or InceptionV3 [18] may report marginally higher accuracies (e.g., 99.87% on MangoLeafBD), these gains often come at the cost of significantly increased computational complexity, rendering them less suitable for on-device deployment. A substantial advantage of the proposed architecture lies in its inference speed. On a standard GPU environment, MangoMamba requires only 1.46-1.52 ms per image. When compared to the recent ViX-MangoEFormer [9], which reports an inference time of approximately 120 ms on GPU hardware, MangoMamba achieves an approximate 80× reduction in latency while maintaining comparable accuracy.

Unlike many existing studies that primarily utilize models as remote backends accessed via Web APIs (e.g., ViX-MangoEFormer), the proposed framework explicitly supports offline, on-device execution. To validate this capability, the trained MangoMamba model was converted to the ONNX format and integrated into a native Android application using Android Studio.

Figure 12 illustrates the real-world deployment of the application. The system successfully processes images directly on the mobile device, with an inference latency of 100-300 ms (depending on the specific mobile chipset), ensuring functionality even in areas with limited or no internet connectivity. This contrasts with server-dependent solutions that suffer from latency induced by network transmission.

Regarding model size, MangoMamba strikes an optimal balance with 5.8 million parameters and a file size of 22.2 MB. This is significantly more compact than transformer-based alternatives, such as DeiT (85.8M parameters, 1.03

Table 6
Comparison between the proposed model and other studies that also evaluated on the MangoLeafBD dataset. Note: (↑) indicates higher is better; (↓) indicates lower is better.

Model	Accuracy(%) ↑	Dataset	Image Amount ↑	Inference Time (Per image) ↓	Inference Time on mobile ↓	Params ↓	Model Size ↓	Train/Test split
DeiT [6]	**99.75**	MangoLeafBD	4,000	346.87 ms (CPU)	Not provided	85.8 M	1.03 GB	8:1:1
Optimized DeiT [6]	**99.75**	MangoLeafBD	4,000	315.78 ms (CPU)	Not provided	781 K	88.47 MB	8:1:1
SwinTransformer [6]	99.5	MangoLeafBD	4,000	206.02 ms (CPU)	Not provided	27.58 M	110.30 MB	8:1:1
ShuffleNet [6]	99.5	MangoLeafBD	4,000	40.76 ms (CPU)	Not provided	5.36 M	64.72 MB	8:1:1
MobileNetV2 [6]	98.79	MangoLeafBD	4,000	42.28 ms (CPU)	Not provided	2.23 M	27.19 MB	8:1:1
EfficientNet [6]	98.75	MangoLeafBD	4,000	45.01 ms (CPU)	Not provided	4.01 M	48.7 MB	8:1:1
DenseNet121 [6]	96.75	MangoLeafBD	4,000	119.34 ms (CPU)	Not provided	6.96 M	84.47 MB	8:1:1
SqueezeNet [6]	96.5	MangoLeafBD	4,000	20.80 ms (CPU)	Not provided	**726.6 K**	**8.78 MB**	8:1:1
ViX-MangoEFormer [9]	99.43	MangoLeafBD	4,000	120 ms (GPU)	Not provided	Not provided	Not provided	9:1
	98.02	Mango Leaf Disease	12,730	120 ms (GPU)	Not provided	Not provided	Not provided	9:1
	98.63	MLD24	6,400	120 ms (GPU)	Not provided	Not provided	Not provided	9:1
	99.78	Combined	25,530	120 ms (GPU)	Not provided	Not provided	Not provided	9:1
MangoMamba	99.75	MangoLeafBD	4000	**1.46 ms (GPU)**	**100-300 ms**	5.8 M	22.2 & 68.8 MB	8:1:1
Our proposed model	99.72	Combined	**40,000**	1.52 ms (GPU)	**100-300 ms**	5.8 M	22.2 & 68.8 MB	8:1:1
	98.14	VN-MangoLeaf	7000	1.48 ms (GPU)	**100-300 ms**	5.8 M	22.2 & 68.8 MB	8:1:1

GB) and SwinTransformer (27.58M parameters, 110.30 MB). Although SqueezeNet offers a smaller footprint (8.78 MB), its accuracy (96.5%) is notably lower than that of the proposed method (99.75%).

In summary, the experimental results indicate that MangoMamba achieves an effective trade-off between accuracy and efficiency, validating its suitability for real-time edge computing in smart agriculture.

3.7. Error analysis

While the MangoMamba model achieves high overall classification accuracy, a granular analysis of misclassified samples reveals specific limitations in distinguishing phenotypically similar diseases. Figure 13 illustrates representative error cases accompanied by Grad-CAM visualizations, which serve to diagnose whether failures stem from incorrect spatial localization or feature ambiguity.

Two primary misclassification patterns are observed. The first involves confusion between *Anthracnose* and *Red Rust* (Rows 1, 2, and 4 in Figure 13). In these instances, the model incorrectly predicts the class with moderate confidence (e.g., 0.511-0.665). The Grad-CAM heatmaps demonstrate that the attention mechanism correctly activates on the necrotic lesions, indicating that the model successfully localizes the pathology. However, the decision boundary remains ambiguous due to the shared reddish-brown chromatic characteristics of both diseases. This suggests that the feature extractor prioritizes dominant color features over the subtle textural differences (e.g., the powdery pustules of Red Rust versus the sunken necrosis of Anthracnose).

The second pattern involves the misclassification of *Gall Midge* as *Sooty Mold* (Row 3). Both conditions manifest as dark, clustered masses on the leaf surface. Although the model accurately focuses on the gall structures, the morphological similarity in low-contrast regions leads to erroneous predictions.

These findings imply that while the hybrid Mamba-Attention architecture excels at capturing global context and localizing symptoms (as evidenced by precise Grad-CAM activation), it occasionally struggles with fine-grained texture discrimination when inter-class color similarity is high. The relatively low confidence scores in these misclassified cases suggest that integrating an uncertainty threshold mechanism could effectively flag such ambiguous samples for expert review in practical deployment.

3.8. Limitations and Future Research Directions

While the experimental results demonstrate that the proposed MangoMamba architecture achieves a favorable trade-off between computational efficiency and classification accuracy, several limitations regarding the study's scope and deployment validity are acknowledged.

First, regarding the hardware deployment claims, although the model was designed for edge devices, the current empirical validation of real-time inference is limited to a specific mid-range smartphone configuration (Xiaomi Redmi Note 11S). While this device confirms the feasibility of the architecture with an inference latency suitable for interactive use, the performance on lower-end devices or different mobile chipsets (e.g., older Snapdragon series or MediaTek entry-level chips) has not been comprehensively benchmarked. Furthermore, the current deployment relies on the raw PyTorch mobile runtime. The potential gains from hardware-specific optimizations, such as quantization (INT8) or conversion to ONNX/TensorFlow Lite formats, which are critical for broad cross-platform compatibility, remain an open area for investigation.

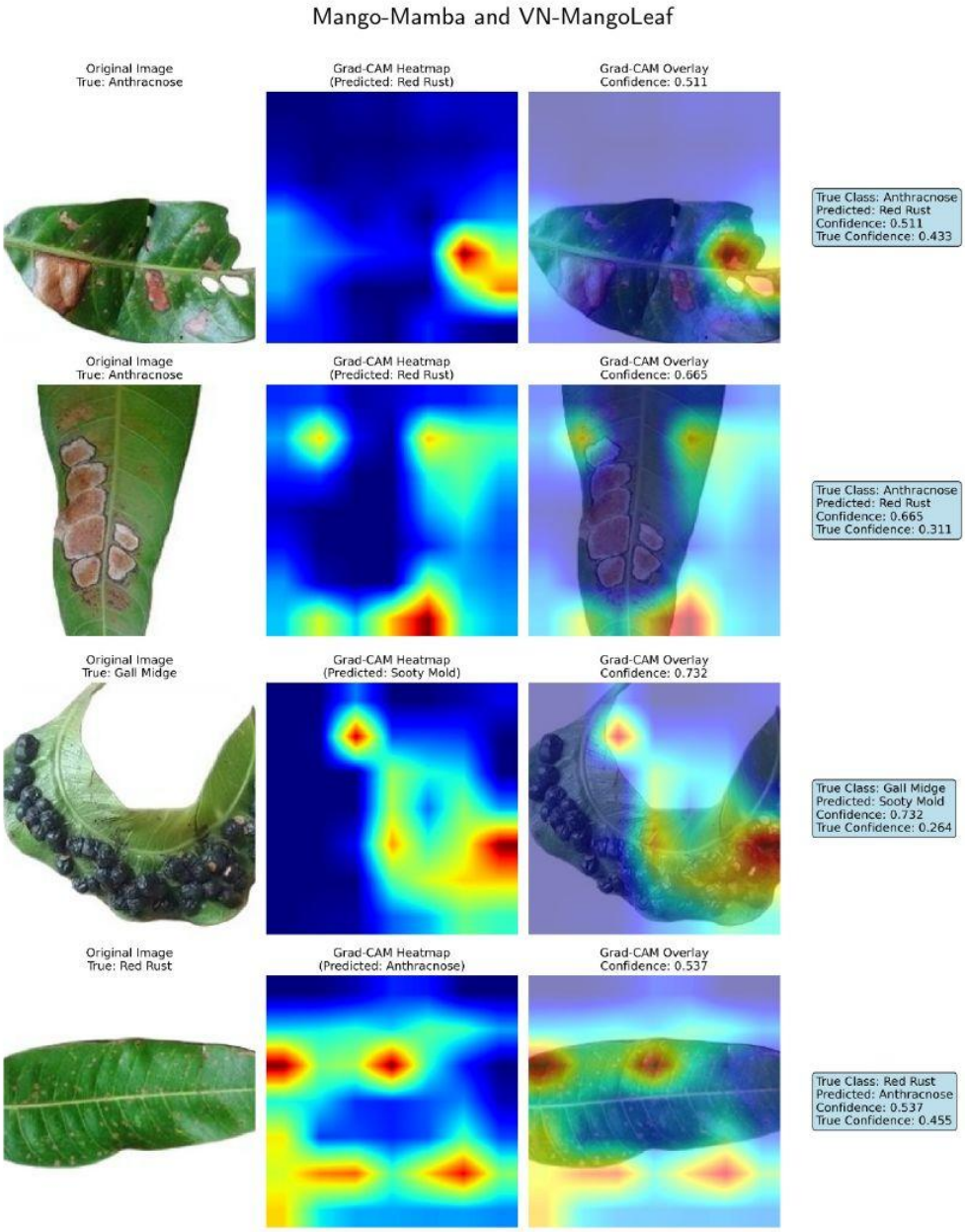


Figure 13: Incorrect predicted images

Second, the generalization capability of the VN-MangoLeaf dataset presents geographical constraints. Although the dataset successfully addresses the bias of existing Bangladesh-centric repositories by introducing Vietnamese-specific disease variants like Red Rust, the data collection was primarily concentrated in Southern Vietnam. Consequently, the model may exhibit reduced sensitivity to disease manifestations specific to Northern Vietnam or other tropical regions with differing climatic conditions. Additionally, the current taxonomy includes seven distinct classes; however, other economically significant mango pathologies, such as Bacterial Black Spot (*Xanthomonas campestris*) or Malformation disease, are absent from the training distribution. This limitation implies that the system classifies unseen pathologies as one of the existing seven categories rather than identifying them as "unknown" or "out-of-distribution."

Third, while data augmentation and curriculum learning were employed to mitigate environmental variability, the robustness of the model under extreme field conditions—such as varying sunlight angles at dusk/dawn, severe occlusion

by overlapping foliage, or motion blur caused by wind—requires further rigorous field testing. The current evaluation relies heavily on static images, whereas real-world agricultural diagnosis often involves dynamic video streams.

Future research directions will focus on three key objectives to address these shortcomings.

- **Cross-Platform ptimization:** Future work will implement and evaluate model quantization and pruning techniques to establish a comprehensive benchmark across a diverse fleet of Android and iOS devices, ensuring consistent real-time performance regardless of hardware tier.
- **Dataset expansion:** The VN-MangoLeaf dataset will be expanded through a citizen science approach, incorporating samples from diverse geographical latitudes to capture broader symptom variations. Concurrently, the architecture will be adapted to support open-set recognition, enabling the model to flag unknown diseases rather than forcing a misclassification.
- **Integrated pest management system:** Beyond singular leaf disease classification, future iterations aim to integrate the MangoMamba backbone into a multi-modal system capable of analyzing fruit quality and pest infestation simultaneously, providing a holistic diagnostic tool for precision agriculture.

4. Conclusion

This study introduced MangoMamba, a lightweight hybrid architecture that effectively balances classification accuracy with computational efficiency for mango leaf disease detection. By integrating Multi-Scale Mamba Mixers with Large-Kernel Attention, the proposed model achieved competitive accuracies of 99.75% on the MangoLeafBD dataset and 98.71% on the newly curated VN-MangoLeaf dataset (7,000 images). With a compact footprint of 5.8 million parameters and a file size of 22.2 MB, the model demonstrates linear complexity, enabling an inference speed of 1.46 ms per image on GPU, representing an approximate 80× speedup compared to recent Transformer-based approaches like ViX-MangoEFormer. Crucially, practical deployment feasibility was validated via an ONNX-based Android application, achieving stable inference latencies of 100-300 ms on mobile devices without internet dependency. Ablation studies confirmed that the synergistic combination of Mamba mixers and attention mechanisms significantly minimizes feature redundancy while maximizing diagnostic precision. Future work will focus on integrating uncertainty quantification to handle ambiguous cases and expanding the dataset to include diverse environmental conditions for broader agro-ecological generalization.

Conflicts of interest

The authors have no conflicts of interest to declare that are relevant to the content of this article.

Data and code availability

To promote transparency and facilitate future research in the domain of agricultural plant pathology, the VN-MangoLeaf dataset has been made publicly available. It is released under the **Creative Commons Attribution 4.0 International (CC BY 4.0)** license, allowing for unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. The full dataset, including raw and augmented images, can be accessed via the Kaggle repository at: `https://www.kaggle.com/datasets/binhpt310/viet-nam-mango-diseases-leaves-dataset`.
The source code can also be found at `https://github.com/binhpt310/Mango-Mamba-and-VN-MangoLeaf`.
The corresponding author can provide the datasets and codes generated and/or analyzed during the current work upon reasonable request.

Acknowledgements

This research was supported by The VNUHCM-University of Information Technology's Scientific Research Support Fund.

References

[1] Sarder Iftekhar Ahmed, Muhammad Ibrahim, Md. Nadim, Md. Mizanur Rahman, Maria Mehjabin Shejunti, Taskeed Jabid, and Md. Sawkat Ali. MangoLeafBD: A comprehensive image dataset to classify diseased and healthy mango leaves. *Data in Brief*, 47:108941, April 2023.

[2] Faten S. Alamri, Tariq Sadad, Ahmed S. Almasoud, Raja Atif Aurangzeb, and Amjad Khan. Mango disease detection using fused vision transformer with convnext architecture. *Computers, Materials & Continua*, 83(1):1023–1039, 2025.
[3] Albert Gu and Tri Dao. Mamba: linear-time sequence modeling with selective state spaces, 2023.
[4] Meng-Hao Guo, Cheng-Ze Lu, Zheng-Ning Liu, Ming-Ming Cheng, and Shi-Min Hu. Visual attention network, December 2023.
[5] Ali Hatamizadeh and Jan Kautz. Mambavision: A hybrid mamba-transformer vision backbone. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 25261–25270, June 2025.
[6] Md. Arban Hossain, Saadman Sakib, Hasan Muhammad Abdullah, and Shifat E. Arman. Deep learning for mango leaf disease identification: A vision transformer perspective. *Heliyon*, 10(17):e36361, September 2024.
[7] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9992–10002, Montreal, QC, Canada, October 2021. IEEE.
[8] Bahar Uddin Mahmud, Abdullah Al Mamun, Md Jakir Hossen, Guan Yue Hong, and Busrat Jahan. Light-Weight Deep Learning Model for Accelerating the Classification of Mango-Leaf Disease. *Emerging Science Journal*, 8(1):28–42, February 2024.
[9] Abdullah Al Noman, Amira Hossain, Anamul Sakib, Jesika Debnath, Hasib Fardin, Abdullah Al Sakib, Rezaul Haque, Md. Redwan Ahmed, Ahmed Wasif Reza, and M. Ali Akber Dewan. Vix-mangoeformer: an enhanced vision transformer–efficientformer and stacking ensemble approach for mango leaf disease recognition with explainable artificial intelligence. *Computers*, 14(5):171, May 2025.
[10] Ngoh D. J. Patrice, Heu Alain, Mboussi S. Bertrand, Kuate T. W. Norbert, Kone N. A. Nourou, Tchoupou T. D. Brice, Amani G. Honorine, Jeutsa A. Dolaris, Djile Bouba, and Ambang Zachee. First report of red rust disease caused by cephaleuros virescens on mango (Mangifera indica) tree in cameroon. *International Journal of Phytopathology*, 9(3):187–193, December 2020.
[11] Juan Qin, Linfan Deng, Cong Li, Junjie He, Haibo Pen, and Zhaoxia Wang. CropCapsNet: Enhanced capsule network for crop disease classification. *Computers and Electrical Engineering*, 128:110635, December 2025.
[12] Md Shajedur Rahman, Rakibul Hasan, and Mayen Uddin Mojumdar. Mango leaf disease dataset, November 2024.
[13] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. Dynamicvit: efficient vision transformers with dynamic token sparsification. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, NIPS '21, Red Hook, NY, USA, 2021. Curran Associates Inc.
[14] Redwan Ahmed Rizvee, Tasnim Hossain Orpa, Adil Ahnaf, Md Ahsan Kabir, Mohammad Rifat Ahmmad Rashid, Mohammad Manzurul Islam, Maheen Islam, Taskeed Jabid, and Md Sawkat Ali. LeafNet: A proficient convolutional neural network for detecting seven prominent mango leaf diseases. *Journal of Agriculture and Food Research*, 14:100787, December 2023.
[15] Md. Mahir Hasan Shakib, Sumaya Mustofa, and Md Taimur Ahad. MLD24: An image dataset for mango leaf disease detection, August 2024.
[16] Noam Shazeer. Glu variants improve transformer, 2020.
[17] Gia-Bao Truong, Thi-Thao Tran, Nhu-Linh Than, Van Quang Nguyen, Thi Hue Nguyen, and Van-Truong Pham. SC-MambaFew: Few-shot learning based on Mamba and selective spatial-channel attention for bearing fault diagnosis. *Computers and Electrical Engineering*, 123:110004, April 2025.
[18] Teena Varma, Prajwal Mate, Noamaan Abdul Azeem, Sanjeev Sharma, and Bhupendra Singh. Automatic mango leaf disease detection using different transfer learning models. *Multimedia Tools and Applications*, May 2024.
[19] Qilong Wang, Banggu Wu, Pengfei Zhu, Peihua Li, Wangmeng Zuo, and Qinghua Hu. Eca-net: efficient channel attention for deep convolutional neural networks. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11531–11539, Seattle, WA, USA, June 2020. IEEE.
[20] Sanghyun Woo, Shoubhik Debnath, Ronghang Hu, Xinlei Chen, Zhuang Liu, In So Kweon, and Saining Xie. Convnext v2: Co-designing and scaling convnets with masked autoencoders, June 2023.
[21] Hewei Zhang, Shengzhou Li, Jialong Xie, Zihan Chen, Jiyang Chen, and Jianwen Guo. VMamba for plant leaf disease identification: design and experiment. *Frontiers in Plant Science*, 16:1515021, April 2025.
[22] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: efficient visual representation learning with bidirectional state space model. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

Appendix A: Model Architecture Equations

This appendix provides the detailed mathematical formulations for the core components of the proposed **MangoMamba** architecture, specifically the LayerNorm-SwiGLU-Residual pathway, the Large-Kernel Attention (LKA) mechanism, and the Selective State Space Model (SSM) components.

A.1. Efficient Channel Attention (ECA)

ECA mechanism [19] generates channel-wise attention weights s via a parameter-efficient 1D convolution over global average pooled (GAP) features according to Equation 6:

$$\begin{aligned} s &= \sigma(\text{Conv1D}(\text{GAP}(x))), \\ \hat{x} &= x \odot s, \end{aligned} \tag{6}$$

where sigmoid activation σ is applied on the result of a 1D convolution over the GAP features, and $\hat{x}$ applies these weights to the original input x through element-wise multiplication $\odot$.

A.2. Global Response Normalization (GRN)

GRN mechanism [20] stabilizes feature responses by normalizing the spatial L2-norm (energy) across the height and width dimensions, followed by a learnable affine transformation. This is applied according to Equation 7:

$$\begin{aligned} n_x &= \frac{\|x\|_2}{\left(\text{mean}_C\left(\|x\|_2\right)+\epsilon\right)}, \\ \text{GRN}(x) &= x+\gamma\left(x \odot n_x\right)+\beta, \end{aligned} \tag{7}$$

where $\|\cdot\|_2$ denotes the L 2 -norm computed over the spatial dimensions (H, W) for each channel, $\text{mean}_C(\cdot)$ is the mean across channels, ϵ is a small constant for numerical stability (typically 10^{-6}), $\odot$ represents element-wise multiplication, x and γ are learnable parameters initialized to zero.

A.3. Windowed processing

In each stage, the feature maps are partitioned into non-overlapping windows of size $S \times S$ to facilitate efficient local computation. This process transforms the input tensor $x \in \mathbb{R}^{B\times C\times H\times W}$ into a windowed representation $X \in \mathbb{R}^{B_{\text{win}}\times L\times C}$ ($L = S^2$ where L represents the number of tokens within each window) and according to Equation 8:

$$B_{\text{win}} = B \cdot \left(\frac{H}{s}\right) \cdot \left(\frac{W}{s}\right), \tag{8}$$

where B_{win} is the total number of windows, with B being the batch size, while $\frac{H}{s}$ and $\frac{W}{s}$ indicate the number of windows along the height and width dimensions, respectively.

A.4. Dual-pathway residual architecture

Structurally, the MangoMambaBlock employs a dual-pathway residual architecture, comprising a LayerNorm-Mixer-Residual pathway followed by a LayerNorm-SwiGLU-Residual pathway. For an input token sequence $x \in \mathbb{R}^{B\times L\times C}$ the forward pass is formalized as Equation 9:

$$\begin{aligned} x' &= x+\text{DropPath}\left(\gamma_1 \cdot \text{Mixer}(\text{ LayerNorm }(x))\right), \\ y &= x'+\text{DropPath}\left(\gamma_2 \cdot \text{FFN}\left(\text{LayerNorm}\left(x'\right)\right)\right), \end{aligned} \tag{9}$$

where Mixer denotes either the Multi-Scale Mamba Mixer or LKA based on the block position, FFN is the SwiGLU-based feed-forward network, DropPath applies stochastic depth regularization with a stage-specific drop rate (derived from a linear schedule up to 0.15), and γ_1, γ_2 are learnable layer scaling coefficients initialized to 1×10^{-5} to mitigate vanishing gradients and promote stable training, as per modern practices in vision transformers [13].

A.5. Multi-Scale Mamba Mixer

The Multi-Scale Mamba Mixer addresses the challenge of capturing dependencies at varying spatial resolutions by processing windowed tokens through parallel high- and low-resolution branches. The high-resolution branch processes the full token sequence of length $L = S^2$, while the low-resolution branch operates on a spatially downsampled version of length $\frac{L}{4}$, achieved by reshaping to $\mathbb{R}^{B\times C\times S\times S}$, applying bilinear interpolation downsampling to $\frac{S}{2}\times\frac{S}{2}$, and flattening. Each branch utilizes a DiseaseAwareMixer (DAM) module, with the high branch employing an expansion factor of 1.5 for richer representations and the low branch using 1.0 for efficiency. The outputs of $y_{\text{high}} = \text{DAM}(x)$ and $y_{\text{low}} = \text{Upsample}(\text{DAM}(\text{Downsample}(x)))$ are fused using a learnable per-channel gate as Equation 10:

$$\begin{aligned} g &= \sigma(W) \in \mathbb{R}^{1\times 1\times c}, \\ y &= (1-g)y_{\text{high}} + g y_{\text{low}} \end{aligned} \tag{10}$$

where W is a learnable parameter tensor initialized to zero (shape $[1, 1, C]$) and σ is the sigmoid activation. This gating mechanism allows adaptive blending, reducing parameters compared to dense fusion while enhancing multi-scale feature integration.

A.6. DiseaseAwareMixer

The input x is first projected to an inner dimension $d_{\text{inner}} = \lfloor \text{expand} \times C \rfloor$ where $z = \text{in_proj}(x)$. This expanded representation is split into x-stream x_z (first half) and z-stream z_z (second half) for state-space processing and gating, respectively. The x-stream undergoes SiLU activation and depthwise 1D convolution for local context according to Equation 11:

$$\begin{aligned} x_{\text{conv}} &= \text{SiLU}\left(\text{DWConv1D}\left(x_z, \text{kernel} = 3\right)\right), \\ \left(\Delta_t, B_t, C_t\right) &= x_\text{proj}\left(x_{\text{conv}}\right), \end{aligned} \tag{11}$$

where Δ_t is further projected and softplus-activated via *dt_proj* to ensure positive time steps. The selective state-space recurrence is then computed using the selective_scan_fn interface [from mamba_ssm] as follow in Equation 12:

$$\begin{aligned} h_t &= \tilde{A}\left(\Delta_t\right) h_{t-1} + \tilde{B}\left(\Delta_t, B_t\right) x_t \\ o &= h_t C_t \\ y &= o \odot \text{SiLU}\left(z_z\right) \end{aligned} \tag{12}$$

with $\tilde{A}$ and $\tilde{B}$ as discretized matrices, enabling input-dependent evolution of the hidden state $h_t \in \mathbb{R}^{d_{\text{state}}}$ then the output is gated by the SiLU-activated z-stream and projected back to C dimensions via out_proj. This formulation provides efficient modeling of long-range dependencies in windowed tokens, which are crucial for detecting interconnected disease symptoms across leaf veins.

A.7. Large-Kernel Attention (LKA)

In later blocks, the Large-Kernel Attention (LKA) pathway replaces the Mamba Mixer to generate spatial attention maps through cascaded depthwise convolutions, achieving large receptive fields without quadratic complexity. This is advantageous for capturing multi-scale spatial dependencies in mango leaves, such as the irregular patterns of red rust pustules. The input tokens are reshaped to spatial format $x \in \mathbb{R}^{B \times C \times S \times S}$, then processed sequentially. A depthwise convolution with kernel size $k_1 = 3$, padding 1 followed by a dilated depthwise convolution with $k_2 = 5$, dilation 2 will result a residual gate with a learnable per-channel parameter $G \in \mathbb{R}^{1 \times C \times 1 \times 1}$ as described in Equation 13 :

$$\begin{aligned} a &= \text{DWConv}_{k_1=3}(x), \\ b &= \text{DWConv}_{k_2=5, d=2}(a), \\ y &= x + \sigma(G) \odot b, \end{aligned} \tag{13}$$

Where σ is sigmoid and $\odot$ is broadcasted element-wise multiplication. The output is flattened back to token sequence format for downstream processing.

A.8. SwiGLU activation

To further enhance representational capacity, each block incorporates a position-wise feed-forward network using SwiGLU activation. The input is projected to a reduced hidden dimension $h = \lfloor 0.8\times \text{ mlp_ratio} \times C \rfloor$ (mlp_ratio=2.0 typically) via two parallel linear layers V_1 and V_2 followed by gating Equation 14:

$$\begin{aligned} \text{SiLU}(z) &= z \cdot \sigma(z), \\ \text{SwiGLU}(x) &= \left(\text{SiLU}\left(x_{V2}\right) \odot x_{V1}\right) W_o, \end{aligned} \tag{14}$$

where $\odot$ is element-wise, and W_o is the output projection back to C.